VLSI DESIGN VERIFICATION

SystemVerilog Interview Preparation

Module 10: Functional Coverage and Assertions

Detailed study notes for Design Verification roles

Concepts from first principles · complete SystemVerilog examples · line-by-line explanations
comparison tables · common mistakes · interview questions with answers · coding exercises

Contents

Module 10 Overview 4

Part 1: Coverage Fundamentals 6
- 1.1 What Is Coverage? 6
- 1.2 Code Coverage vs Functional Coverage 7
- 1.3 Covergroup 10
- 1.4 Coverpoint 14
- 1.5 Sampling 17

Part 2: Bins 22
- 2.1 Bins and Automatic Bins 22
- 2.2 Explicit Bins 25
- 2.3 Wildcard Bins 28
- 2.4 Ignore Bins 29
- 2.5 Illegal Bins 30
- 2.6 Transition Bins 33

Part 3: Cross Coverage, Options and Coverage-Driven Verification 38
- 3.1 Cross Coverage 38
- 3.2 Coverage Options 43
- 3.3 How the Coverage Percentage Is Computed 48
- 3.4 Coverage-Driven Verification 50
- 3.5 The Complete alu_coverage Class 53

Part 4: Assertion Fundamentals 59
- 4.1 What Is an Assertion? 59
- 4.2 Immediate Assertions 61
- 4.3 Concurrent Assertions 65
- 4.4 Sequences and Properties 68
- 4.5 Clocking Events 73

Part 5: Assertion Operators and System Functions 76
- 5.1 Implication Operators: |-> and |=> 76
- 5.2 Cycle Delays and Repetition 80
- 5.3 Sampled Value Functions: $rose, $fell, $stable, $changed, $past 85
- 5.4 Vector Functions: $onehot, $onehot0, $isunknown, $countones 90
- 5.5 disable iff 92
- 5.6 Assertion Pass and Failure Behaviour 95
- 5.7 Sequence Operators 97
- 5.8 Where Assertions Live, bind, and Assertions vs Scoreboard 100

Part 6: Practical Assertion Sets 105
- 6.1 Assertions for a Synchronous FIFO 105
- 6.2 Assertions for a Valid/Ready Handshake 110
- 6.3 Assertions for a Simple Counter 113
- 6.4 Assertions for the ALU 116
- 6.5 Interview Questions on Practical Assertions 121

Part 7: Module 10 Revision and Practice 122
- 7.1 Revision Summary 122

7.2 Interview Questions and Answers 125
7.3 Output-Prediction Questions 127
7.4 Find-the-Error and Debugging Questions 130
7.5 Coding Exercises with Solutions 133
7.6 Concepts You Must Understand Before Moving Forward 137
7.7 What Comes Next 138

Module 10 Overview

Modules 1 to 9 built a complete class-based testbench: random transactions, a driver, a monitor and a scoreboard. Two questions are still open.

1. **"Have we tested enough?"** The scoreboard says that everything that was tried gave the right answer. It says nothing about what was never tried. **Functional coverage** answers this question.
2. **"Did the DUT obey the protocol rules at every clock cycle?"** The scoreboard compares complete transactions. It does not notice that `valid` dropped one cycle too early, or that a FIFO accepted a write while full. **Assertions** answer this question.

Coverage and assertions are the two measuring instruments of a verification environment. Almost every Design Verification interview contains questions on both, and "write an assertion for this waveform" and "write a covergroup for this field" are standard whiteboard exercises.

What this module covers

Part	Content
1	Coverage fundamentals: code coverage versus functional coverage, covergroup, coverpoint, sampling
2	Bins: automatic, explicit, wildcard, ignore, illegal and transition bins
3	Cross coverage, coverage options, how the percentage is computed, coverage-driven verification, the complete `alu_coverage` class
4	Assertion fundamentals: immediate and concurrent assertions, sequences, properties, clocking, sampling
5	Assertion operators and functions: implication, delays, repetition, `$rose`, `$fell`, `$stable`, `$changed`, `$past`, `$onehot`, `$isunknown`, `disable iff`, action blocks, sequence operators, `bind`
6	Practical assertion sets for a FIFO, a valid/ready handshake, a counter and the ALU
7	Revision summary, interview questions, output-prediction and debugging questions, coding exercises

What you should already know

From	Needed here
Module 1	Four-state values, `$isunknown`, enums, expression width rules
Module 4	Classes, constructors, handles (a covergroup inside a class is created with `new`)
Module 5	Constraints (coverage tells you which constraints to change)
Module 6	The `alu_txn` transaction class and the idea of a verification plan
Module 8	Interfaces and clocking blocks (assertions are usually placed in the interface)
Module 9	Monitor and scoreboard (the coverage collector receives transactions from the monitor)

Simulation-only or synthesizable? Covergroups are simulation-only. Assertions are ignored by synthesis; they are used in simulation and are also the input language of formal verification tools.

> **Note:** As in the earlier modules, the "Expected output" blocks were worked out by hand from the IEEE 1800 rules and were not captured from a simulator run. The stimulus in every example is fixed (not random), so the coverage percentages and the assertion failure times are deterministic. In assertion examples the only text shown as exact output is text printed by the example's own `$display` calls. Messages that a simulator generates by itself (for a failing assertion with no `else` clause, or for an illegal bin) have a different layout in every tool.

How the timing in the examples is arranged

All clocked examples use the same scheme, so that you can check every result by hand:

```
clk starts at 0 and toggles every 5 time units

time     0    5    10   15   20   25   30   35   40   45
clk      ___/‾‾‾‾\____/‾‾‾‾\____/‾‾‾‾\____/‾‾‾‾\____/‾‾‾‾\__
posedge       ^         ^         ^         ^         ^
cycle         0         1         2         3         4      cycle k = posedge at 10k+5

stimulus changes at time 0 and at every negedge (10, 20, 30 ...),
so every signal is stable around each posedge and the sampled value is never in doubt
```

Part 1: Coverage Fundamentals

1.1 What Is Coverage?

In plain English

Imagine testing a lift (elevator) in a ten-storey building. You ride it a thousand times and it never fails. Is the lift fully tested? You cannot say until you know **which** rides you took. If all thousand rides were between the ground floor and floor 1, the answer is no.

Coverage is the record of which rides were taken. It is a measurement of how much of the design, or how much of the design's intended behaviour, the tests have actually exercised.

Technical definition

Coverage is a metric, expressed as a percentage, of the completeness of verification. There are two families:

- **Code coverage** measures how much of the **RTL source code** was exercised. The simulator collects it automatically.
- **Functional coverage** measures how many of the **features and scenarios listed in the verification plan** were exercised. The verification engineer writes it, using `covergroup` constructs (data-oriented coverage) and `cover property` statements (temporal, protocol-oriented coverage).

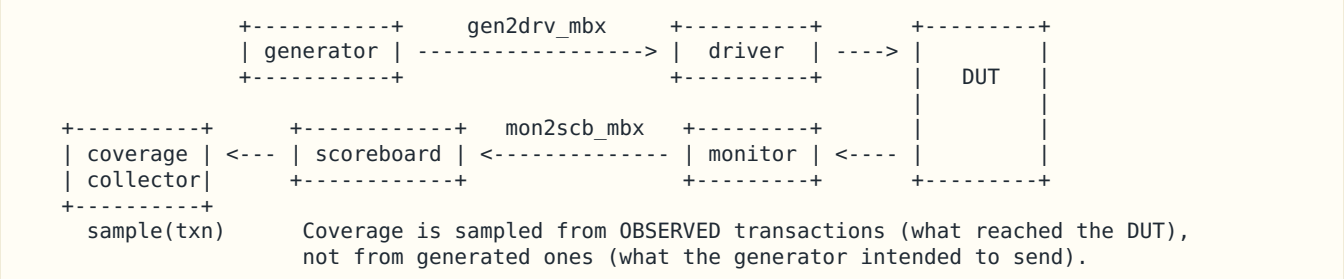
Why it is needed

A constrained-random testbench generates stimulus that nobody wrote by hand, so nobody knows in advance what a given test did. Without coverage:

- you cannot tell whether a corner case (FIFO full and a simultaneous read and write, an addition that produces a carry) ever happened;
- you cannot tell when to stop running tests;
- you cannot tell whether a new test added anything;
- "1000 tests passed" has no meaning.

Coverage turns "we ran a lot of tests" into "we exercised 97 percent of the planned scenarios, and here is the list of the remaining 3 percent".

Where coverage fits in the testbench



> **Key point:** Coverage records that something **happened**. It does not record that the DUT **behaved correctly** when it happened. Correctness is the job of the scoreboard and the assertions. Coverage is only meaningful together with checking: 100 percent coverage in a testbench with no checker proves nothing.

Interview questions

Q1. What is coverage? A measurement of how much of the design (code coverage) or of the planned functionality (functional coverage) has been exercised by the tests. It is used to judge verification progress and completeness.

Q2. Why is coverage needed in constrained-random verification? Because random stimulus is not written by hand, the engineer does not know which scenarios occurred. Coverage is the feedback that shows which planned scenarios were hit, which were missed, and when verification can stop.

Q3. Does 100 percent coverage mean the design has no bugs? No. Coverage shows only that scenarios were exercised. Bugs remain if the checkers are weak, if the coverage model itself is missing scenarios, or if a feature was never implemented (code coverage cannot see missing code).

1.2 Code Coverage vs Functional Coverage

Code coverage

Code coverage answers: **"Which parts of the RTL did the simulation execute?"** The simulator instruments the RTL when a compile or run option is given; no extra code is written.

Type	What it measures	Example of a hole
Statement (line) coverage	Each executable statement was executed at least once	An assignment inside an error-handling branch never ran
Block coverage	Each `begin` ... `end` block was entered	Similar to statement coverage, counted per block
Branch coverage	Each branch of every `if` / `else` and `case` was taken, both the true and the false direction	An `if` without `else` was always true; the false path was never taken
Condition coverage	Each sub-condition of a Boolean condition was seen as both true and false	In `if (a || b)`, `b` was never the reason the condition was true
Expression coverage	The same analysis for expressions on the right of assignments	In `y = a & b`, the case `a=1, b=0` never occurred
Toggle coverage	Each bit of each signal changed 0→1 and 1→0	The top address bit never became 1
FSM coverage	Each state was visited and each state transition was taken	The `ERROR` state was never entered

Why branch coverage is not enough: an example

```
always_comb begin
  if (a || b) y = 1;      // statement 1
  else        y = 0;      // statement 2
end
```

Tests applied (`a`, `b`)	Statement	Branch	Condition
(1,0) only	50% (statement 2 never ran)	50%	Incomplete
(1,0) and (0,0)	100%	100% (true and false both taken)	Incomplete: `b` was never 1
(1,0), (0,0) and (0,1)	100%	100%	Complete: each of `a` and `b` independently made the condition true

Functional coverage

Functional coverage answers: **"Which features of the specification did the simulation exercise?"** It is derived from the verification plan and written by the engineer.

Examples for the ALU:

- Was every opcode executed?
- Was every opcode executed with `a` = 0, with `a` = 255, and with a value in between?
- Did an addition produce a carry? Did a subtraction produce a borrow?
- Did two additions occur back to back?

None of these can be read from code coverage. The line `ALU_ADD: result_nxt = a + b;` is 100 percent covered by statement coverage after a single addition of 1 + 1.

Comparison table

Aspect	Code coverage	Functional coverage
Question answered	Was this code executed?	Was this feature or scenario exercised?
Based on	The RTL implementation	The specification and verification plan
Who creates it	The tool, automatically	The verification engineer, manually
Effort	A compile/run option	Design and coding of a coverage model
SystemVerilog constructs	None	`covergroup`, `coverpoint`, `bins`, `cross`, `cover property`
Can detect a missing feature	No. Code that was never written cannot be uncovered	Yes, if the feature is in the plan
Can detect unreachable or dead code	Yes	No
Risk	Gives false confidence: every line executed does not mean every scenario	Only as good as the plan: a scenario nobody thought of is not measured
Types	Statement, branch, condition, expression, toggle, FSM	Data coverage (covergroups), temporal coverage (cover properties)

Reading the two numbers together

Code coverage	Functional coverage	What it means	Action
High	High	Good position	Review the remaining holes; sign off
High	Low	The code runs, but the planned scenarios are not being produced	Improve stimulus: add constraints, tests, sequences
Low	High	The coverage model is too weak, or the RTL contains code for features not in the plan, or dead code	Review the plan and the coverage model; ask the designer about the uncovered code
Low	Low	Verification is at an early stage	Keep going

Interview trap: "If I have 100 percent code coverage, why do I need functional coverage?" Because code coverage cannot see (1) features that were specified but never implemented, (2) combinations of values and sequences of events: each line can be executed without the important combinations ever occurring. And the reverse: "If I have 100 percent functional coverage, why do I need code coverage?" Because the functional model may be incomplete, and code coverage reveals RTL that no planned scenario reaches.

How to say it in an interview: "Code coverage is implementation-based and automatic: it tells me which lines, branches, conditions, toggles and FSM states of the RTL were exercised. Functional coverage is specification-based and user-written: it tells me which features and corner cases from the verification plan were exercised. Sign-off needs both, plus all tests passing and all assertions passing."

Interview questions

Q1. What are the types of code coverage? Statement (line), block, branch, condition, expression, toggle and FSM (state and transition) coverage.

Q2. What is the difference between branch and condition coverage? Branch coverage checks that each `if` / `case` decision went both ways as a whole. Condition coverage checks each Boolean sub-expression of the decision individually. `if (a || b)` has 100 percent branch coverage with the inputs (1,0) and (0,0), but condition coverage is incomplete because `b` was never 1.

Q3. What is toggle coverage used for? It shows whether every bit of every signal went both 0→1 and 1→0. It is useful for finding stuck or unconnected bits, especially at block boundaries and in gate-level or connectivity checks.

Q4. Functional coverage is 100 percent but code coverage is 80 percent. What do you conclude? The functional coverage model is incomplete, or the RTL contains unreachable/dead code or features that are not in the verification plan. The uncovered code must be analysed: either new coverage items and tests are added, or the code is justified and excluded with the designer's agreement.

Q5. Code coverage is 100 percent but functional coverage is 70 percent. What do you conclude? The tests execute all the code but do not produce all the planned scenarios (value combinations, sequences, corner cases). The stimulus must be improved with new constraints, seeds or directed tests.

Q6. Can functional coverage be collected without writing code? No. It must be modelled by the engineer with covergroups and cover properties. That is its main cost and also its value: it encodes the verification plan in executable form.

1.3 Covergroup

In plain English

A covergroup is a **tally sheet**. You decide which variables you want to watch and which values or value ranges count as separate "boxes". Each time you say "sample now", the covergroup looks at the current values of those variables and puts a tick in the matching boxes. At the end of the test, the fraction of boxes that have at least one tick is the coverage.

Technical definition

A `covergroup` is a user-defined type that encapsulates the specification of a coverage model. It can contain:

- an optional **sampling event** (for example `@(posedge clk)`),
- one or more **coverpoints** (a variable or expression to observe),
- **bins** for each coverpoint (the boxes),
- **cross coverage** between coverpoints,
- **options** that control how coverage is collected and computed,
- optional formal arguments.

Like a class, a covergroup is only a definition. Nothing is collected until an **instance** is created with `new`.

Syntax

```
covergroup <name> [ ( <arguments> ) ] [ <sampling event> ];
  [ option.<name> = <value>; ]
  <label> : coverpoint <expression> [ iff ( <condition> ) ] [ { <bins> } ];
  <label> : cross <coverpoint>, <coverpoint> [ { <cross bins> } ];
endgroup

<name> <instance> = new( <arguments> );       // an instance must be constructed
```

Where a covergroup can be declared

Location	Typical use
Inside a `class` (an "embedded" covergroup)	The normal case in a class-based testbench: coverage of transaction fields
Inside a `module` or `interface`	Coverage of signals, sampled on a clock
Inside a `package`	A reusable covergroup type, usually with arguments
Inside a `program` or `checker`	Less common

How it works internally

```
   covergroup definition (compile time)          instance (run time, after new)
   ------------------------------------          --------------------------------------
   cp_mode : coverpoint mode                  cp_mode bins   auto[0] auto[1] auto[2] auto[3]
             (2-bit variable)                 hit counters      0       0       0       0

   each sample():
      1. evaluate the coverpoint expression            mode = 1
      2. find the bin(s) that contain the value        auto[1]
      3. increment the hit counter of those bins       0 1 0 0

   at any time:
      coverage of cp_mode = (bins whose counter >= at_least) / (number of bins) x 100
```

The simulator stores one hit counter per bin. At the end of simulation the counters are written to a coverage database, and the databases from all the tests of a regression are merged.

Complete example 1: a covergroup in a module

tb_cg_basic.sv

```
module tb_cg_basic;

  bit [1:0] mode;        // 4 possible values
  bit [7:0] data;        // 256 possible values

  covergroup cg_basic;
    cp_mode: coverpoint mode;
    cp_data: coverpoint data;
  endgroup

  cg_basic cg = new();

  initial begin
    mode = 0;  data = 0;    cg.sample();
    mode = 1;  data = 3;    cg.sample();
    mode = 1;  data = 4;    cg.sample();
    mode = 3;  data = 255;  cg.sample();

    $display("cp_mode coverage    = %0.2f%%", cg.cp_mode.get_coverage());
    $display("cp_data coverage    = %0.2f%%", cg.cp_data.get_coverage());
    $display("covergroup coverage = %0.2f%%", cg.get_coverage());
  end

endmodule
```

EXPECTED OUTPUT

```
cp_mode coverage    = 75.00%
cp_data coverage    = 4.69%
covergroup coverage = 39.84%
```

Line by line

Code	Explanation
`covergroup cg_basic;`	Declares a covergroup type with no sampling event. It will be sampled by calling `sample()`.
`cp_mode: coverpoint mode;`	A coverpoint on `mode`, labelled `cp_mode`. No bins are written, so the tool creates **automatic bins**: one per value, four bins.

Code	Explanation
`cp_data: coverpoint data;`	`data` has 256 values, more than the default limit of 64 automatic bins. The tool creates 64 bins of four consecutive values each: `[0:3]`, `[4:7]` ... `[252:255]`. Section 2.1 gives the rule.
`cg_basic cg = new();`	Creates the instance. Without this line no coverage is collected, and calling `cg.sample()` is a null-handle error.
`cg.sample();`	Built-in method: sample every coverpoint now.
Samples of `mode`	0, 1, 1, 3. Bins hit: 0, 1 and 3. Three of four bins: 75 percent.
Samples of `data`	0 and 3 fall in the same bin `[0:3]`; 4 falls in `[4:7]`; 255 falls in `[252:255]`. Three of 64 bins: 4.6875 percent, printed as 4.69.
`cg.cp_mode.get_coverage()`	Coverage of one coverpoint, returned as a `real` between 0 and 100.
`cg.get_coverage()`	Coverage of the whole covergroup: the average of its coverpoints (each has weight 1): (75 + 4.6875) / 2 = 39.84.

Complete example 2: a covergroup in a class

tb_cg_class.sv

```
module tb_cg_class;

  class packet_cov;
    bit [1:0] kind;
    bit [3:0] len;

    covergroup pkt_cg;
      cp_kind: coverpoint kind;
      cp_len : coverpoint len {
        bins small = {[0:3]};
        bins large = {[4:15]};
      }
    endgroup

    function new();
      pkt_cg = new();            // an embedded covergroup must be constructed here
    endfunction

    function void sample(bit [1:0] k, bit [3:0] l);
      kind = k;
      len  = l;
      pkt_cg.sample();
    endfunction
  endclass

  packet_cov cov;

  initial begin
    cov = new();
    cov.sample(2'd0, 4'd2);
    cov.sample(2'd1, 4'd9);
    cov.sample(2'd1, 4'd3);
    $display("pkt_cg coverage = %0.2f%%", cov.pkt_cg.get_coverage());
  end

endmodule
```

EXPECTED OUTPUT

```
pkt_cg coverage = 75.00%
```

Line by line

Code	Explanation
`covergroup pkt_cg; ... endgroup` inside the class	An embedded covergroup. It can read the class properties `kind` and `len` directly. The declaration also creates a class member named `pkt_cg`, which is the instance handle.
`bins small = {[0:3]};`	An explicit bin that covers the values 0 to 3. `cp_len` has exactly two bins. Section 2.2 covers explicit bins.
`pkt_cg = new();` in the constructor	An embedded covergroup must be constructed **in the class constructor** `new`. If the line is forgotten, the covergroup handle is `null`.
`function void sample(...)`	A wrapper method: copy the values into the class properties, then call the covergroup's built-in `sample()`.
Result	`kind` saw 0 and 1: two of four bins, 50 percent. `len` saw 2 (small), 9 (large) and 3 (small): two of two bins, 100 percent. Average: 75 percent.

Rules for covergroups in classes

Rule	Detail
Construction	`cg_name = new();` must be executed in the constructor of the enclosing class
One instance per object	The embedded covergroup name is both the type and the variable. You cannot declare a second variable of that type in the class
Several covergroups	A class may contain several embedded covergroups; construct each of them in `new`
Access to members	Coverpoints may use any property of the class, including properties reached through a handle (`txn.a`)
Null handles	If a coverpoint expression is `txn.a`, the handle `txn` must not be `null` when `sample()` is called

Covergroup with arguments

A covergroup can take arguments, which makes one definition reusable:

```
covergroup cg_range (int lo, int hi) with function sample(int v);
  cp_v: coverpoint v {
    bins below    = {[$:lo-1]};
    bins in_range = {[lo:hi]};
    bins above    = {[hi+1:$]};
  }
endgroup

cg_range cg_len  = new(4, 64);       // two instances with different limits
cg_range cg_gap  = new(1, 10);
```

Arguments are passed once, at construction. A `ref` argument (`covergroup cg (ref bit [7:0] sig) @(posedge clk);`) lets each instance watch a different variable.

Built-in methods of a covergroup

Method	Purpose
`sample()`	Trigger sampling of the covergroup
`get_coverage()`	Coverage of the covergroup **type**, over all instances (0 to 100, `real`)

Method	Purpose
`get_inst_coverage()`	Coverage of this **instance** only
`start()` / `stop()`	Resume / suspend collection for this instance
`set_inst_name(string)`	Give the instance a name for the report

Section 3.3 explains the coverage query methods in detail.

> **Common mistake:** Declaring a covergroup and forgetting to construct it. The definition alone collects nothing. In a module: `cg_basic cg = new();`. In a class: `cg_name = new();` inside the constructor.

> **Common mistake:** Constructing an embedded covergroup outside the class constructor, for example in a `build()` method. The standard requires construction in `new`; tools report an error.

> **Verification application:** In the layered testbench of Module 9, the coverage collector is a small class, `alu_coverage`, that owns the covergroup `alu_cg`. The scoreboard (or a separate subscriber connected to the monitor) calls `cov.sample(txn)` for every observed transaction. Section 3.5 gives the complete class.

Interview questions

Q1. What is a covergroup? A user-defined construct that encapsulates a functional coverage model: coverpoints, bins, crosses, options and an optional sampling event. It must be instantiated with `new` before it collects anything.

Q2. Where can a covergroup be defined? In a class, module, interface, package, program or checker.

Q3. How do you instantiate a covergroup that is declared inside a class? By assigning `covergroup_name = new();` inside the class constructor. The embedded covergroup's name serves as the handle.

Q4. Can a class have more than one covergroup? Yes. Each one is constructed in the class constructor.

Q5. What happens if a covergroup is never constructed? No coverage is collected for it. If its `sample()` method is called, a null-handle error occurs.

Q6. Can a covergroup have arguments? Yes. Arguments are supplied to `new` and can be used in bin definitions, or passed by `ref` so that different instances observe different variables.

1.4 Coverpoint

In plain English

A coverpoint is **one thing you want to watch**: a variable, or an expression built from variables. For that one thing, the coverpoint holds the list of boxes (bins) into which its values are sorted.

Technical definition

A `coverpoint` specifies an integral expression to be covered. Each coverpoint has a set of **bins** associated with the sampled values or value transitions of the expression. The bins are either defined explicitly by the user or

created automatically by the tool. A coverpoint may have an `iff` guard: when the guard expression is false at the moment of sampling, the coverpoint is not sampled.

Syntax

```
[ <label> : ] coverpoint <expression> [ iff ( <guard> ) ]
{
  bins          <name> = { <values> };
  ignore_bins   <name> = { <values> };
  illegal_bins  <name> = { <values> };
  option.<name> = <value>;
}
```

or, with automatic bins, simply:

```
<label> : coverpoint <expression>;
```

What can be covered

Coverpoint	Meaning
`coverpoint mode;`	A variable. With no label, the coverpoint takes the variable's name
`cp_op: coverpoint txn.op;`	A class property reached through a handle
`cp_carry: coverpoint txn.result[8];`	A bit-select or part-select
`cp_sum: coverpoint (a + b);`	An expression. Its width follows the normal expression rules of Module 1
`cp_zero: coverpoint (result == 0);`	A 1-bit condition: two automatic bins, for 0 and 1
`cp_len: coverpoint pkt.payload.size();`	A function call result
`cp_addr: coverpoint addr iff (valid);`	Sampled only when `valid` is true

Only **integral** expressions can be covered: bits, vectors, integers, enums. A `real` or a `string` cannot be a coverpoint (IEEE 1800-2012 added limited support for `real` coverpoints with explicit bins, but tool support is uneven).

> **Key point:** Give every coverpoint a **label** (`cp_op:`). The label is the name in the coverage report, the name used in a `cross`, and the name used to call `get_coverage()` on the coverpoint. A coverpoint on an expression without a label gets a tool-generated name.

The iff guard

```
cp_addr: coverpoint addr iff (valid && !reset);
```

When `sample()` is called (or the sampling event fires), the guard is evaluated first. If it is false, this coverpoint is skipped for this sample: no bin counter changes. Other coverpoints in the same covergroup are not affected.

```
   sample #   valid   addr     cp_addr sampled?
   --------   -----   ----     ----------------
      1         0     8'h10    no   (guard false: the value is on the bus by accident)
      2         1     8'h20    yes
      3         1     8'h20    yes
      4         0     8'hFF    no
```

A guard can also be attached to a single bin: `bins b = {[0:3]} iff (en);`.

Four-state values

If the sampled value of a coverpoint contains X or Z, it does not fall into any automatic bin and does not match any ordinary explicit bin. It is simply not counted.

Coverpoint versus covergroup

Property	Covergroup	Coverpoint
What it is	A container: the whole coverage model for one sampling moment	One observed expression inside the container
Sampling event	Defined at covergroup level	Inherits it; can be restricted with `iff`
Contains	Coverpoints, crosses, options	Bins, options
Instantiated with `new`	Yes	No, it exists inside a covergroup instance
Coverage value	Weighted average of its coverpoints and crosses	Covered bins divided by total bins

> **Common mistake:** Sampling when the data is not meaningful. If `addr` is covered on every clock, the values present on the bus during idle cycles fill the bins, and the coverage number is a lie. Either guard the coverpoint with `iff (valid)`, or (better) call `sample()` only when the monitor has assembled a complete, valid transaction.

> **Common mistake:** Covering a wide variable with automatic bins and believing the result. A coverpoint on a 32-bit address gets 64 bins of about 67 million values each. "100 percent" then means only that 64 values spread over the range were seen. Wide variables need explicit bins chosen from the specification (Part 2).

Interview questions

Q1. What is a coverpoint? A coverpoint specifies one integral variable or expression to be covered inside a covergroup, together with its bins.

Q2. What does `iff` do in a coverpoint? It is a guard. The coverpoint is sampled only if the `iff` expression is true at the sampling moment. It is used to ignore values that are not valid, for example during reset or when `valid` is low.

Q3. Can you cover an expression? Yes: `coverpoint (a + b)`, `coverpoint (x == y)`, a part-select, or a function return value. It should be given a label.

Q4. Can a `real` variable be a coverpoint? Not in general. Coverpoints are for integral types. A real value is normally converted to an integer or classified into an enum first.

Q5. Why should a coverpoint have a label? The label names the coverpoint in reports, lets a `cross` refer to it, and lets code query it (`cg.cp_op.get_coverage()`).

1.5 Sampling

In plain English

A covergroup does not watch its variables continuously. It looks at them only at the instants you choose. Choosing those instants correctly is half of writing good coverage: sample too often and meaningless values are counted; sample at the wrong instant and you record a value the DUT never used.

Technical definition

Sampling is the act of evaluating the coverpoint expressions of a covergroup instance and incrementing the hit counters of the matching bins. A covergroup is sampled in one of two ways:

1. **By a sampling event** written in the covergroup header: `covergroup cg @(posedge clk);`. The covergroup is sampled automatically each time the event occurs.
2. **By calling the built-in `sample()` method** from procedural code. This is used when the covergroup has no sampling event (and may also be used in addition to an event).

The two methods compared

Property	Sampling event `@(...)`	`sample()` method
Who triggers it	The simulator, on every occurrence of the event	Your code, exactly when you call it
Typical location	Covergroup in a module or interface, covering signals	Covergroup in a class, covering transaction fields
Control	Coarse; refine with `iff` guards	Precise: one sample per completed transaction
Cost	Samples on every clock, even when idle	Samples only when something happened
Risk	Counting idle or invalid values; races	Forgetting to call it
Passing values	Reads variables that are in scope	Reads variables in scope, or receives arguments (`with function sample`)

Complete example 1: sampling event with an iff guard

tb_cg_event.sv

```
module tb_cg_event;

  logic     clk = 0;
  bit       valid;
  bit [1:0] mode;

  // stimulus for cycles 0..4 (index 0 is the leftmost bit / first element)
  bit [0:4] valid_v     = 5'b01011;
  bit [1:0] mode_v [5]  = '{2'd0, 2'd1, 2'd2, 2'd3, 2'd1};

  covergroup cg_mode @(posedge clk);
    cp_mode: coverpoint mode iff (valid);
  endgroup

  cg_mode cg = new();

  always #5 clk = ~clk;

  initial begin
    for (int i = 0; i < 5; i++) begin
      valid = valid_v[i];
      mode  = mode_v[i];
      @(negedge clk);
    end
    $display("cp_mode coverage = %0.2f%%", cg.get_coverage());
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
cp_mode coverage = 50.00%
```

Execution flow

Cycle	Posedge at	`valid`	`mode`	Effect on `cp_mode`
0	5	0	0	Guard false: not sampled
1	15	1	1	Bin for value 1 hit
2	25	0	2	Guard false: not sampled
3	35	1	3	Bin for value 3 hit
4	45	1	1	Bin for value 1 hit again

Two of the four bins (values 1 and 3) were hit: 50 percent. Without the guard, the values 0 and 2 would have been counted too and the coverage would be 100 percent, although the design never treated those values as valid.

Line by line

Code	Explanation
`covergroup cg_mode @(posedge clk);`	The covergroup is sampled automatically at every rising clock edge.
`coverpoint mode iff (valid);`	The guard is evaluated at each sample.
`valid_v`, `mode_v`	The stimulus for each cycle, stored in vectors so that the example is deterministic.

Code	Explanation
The `for` loop	Applies the values for cycle `i`, then waits for the falling edge. Values therefore change at time 0, 10, 20... and are stable at every rising edge.

Complete example 2: sample() with arguments

A covergroup can declare its own `sample` method with arguments. The arguments are then the things being covered, and no intermediate variables are needed.

tb_cg_sample_args.sv

```
module tb_cg_sample_args;

  covergroup cg_xfer with function sample(bit [7:0] addr, bit wr);
    cp_addr: coverpoint addr {
      bins lo = {[0:127]};
      bins hi = {[128:255]};
    }
    cp_wr: coverpoint wr;
    x_addr_wr: cross cp_addr, cp_wr;
  endgroup

  cg_xfer cg = new();

  initial begin
    cg.sample(8'h10, 1'b1);      // low address, write
    cg.sample(8'hF0, 1'b0);      // high address, read
    $display("cp_addr   = %0.2f%%", cg.cp_addr.get_coverage());
    $display("cp_wr     = %0.2f%%", cg.cp_wr.get_coverage());
    $display("x_addr_wr = %0.2f%%", cg.x_addr_wr.get_coverage());
    $display("cg_xfer   = %0.2f%%", cg.get_coverage());
  end

endmodule
```

EXPECTED OUTPUT

```
cp_addr   = 100.00%
cp_wr     = 100.00%
x_addr_wr = 50.00%
cg_xfer   = 83.33%
```

Line by line

Code	Explanation
`with function sample(bit [7:0] addr, bit wr);`	Overrides the built-in `sample()` with a version that takes arguments. Inside the covergroup, `addr` and `wr` are used like variables.
`cg.sample(8'h10, 1'b1);`	One call passes the values and samples.
`cp_addr`, `cp_wr`	Both address bins and both `wr` values were seen: 100 percent each.
`x_addr_wr`	The cross has 2 × 2 = 4 combinations. Only (lo, write) and (hi, read) occurred: 50 percent. Cross coverage is covered in Section 3.1.
`cg.get_coverage()`	Average of the three items: (100 + 100 + 50) / 3 = 83.33.

When exactly are the values read?

Sampling method	Values used
`cg.sample()`	The values of the variables at the moment of the call

Sampling method	Values used
`@(posedge clk)`	The values at the moment the event is processed in that time step. This is **not** the "sampled value" used by assertions (Section 4.3). If a variable is changed by a blocking assignment at the same clock edge, the order is a race
Event sampling with `type_option.strobe = 1`	The values at the end of the time step (Postponed region), after all assignments of that time step have settled

```
Race with a sampling event:

   always @(posedge clk) data = data + 1;      // blocking assignment at the clock edge
   covergroup cg @(posedge clk);  coverpoint data;  endgroup

   At each posedge, both the always block and the covergroup wake up.
   Which runs first is not defined, so the covergroup may see the old or the new value.

Three safe alternatives:
   1. sample signals through a clocking block (Module 8): stable sampled values
   2. sample on the opposite clock edge, when nothing changes
   3. do not use a sampling event: call sample() from the monitor after it has captured
      a complete transaction   <-- the normal class-based solution
```

Other controls

Control	Effect
`cg.stop();`	Suspend collection for this instance (for example during reset)
`cg.start();`	Resume collection
`iff` on a coverpoint or bin	Skip the coverpoint or bin when the guard is false
`covergroup cg @(event_name);`	Sample when a named event is triggered: `-> event_name;`

Interview trap: "How many times is a covergroup sampled if it has `@(posedge clk)` and the simulation runs for 1000 clocks?" 1000 times, including every idle cycle, unless guards are used. Coverage quality depends on sampling only meaningful values, not on sampling often.

Common mistake: Sampling the transaction in the **generator** or **driver**. That covers what the testbench intended to send, even if the driver dropped the item or the DUT ignored it during reset. Sample from the **monitor** side, which reports what was actually observed on the interface.

Common mistake: Declaring a sampling event **and** calling `sample()` for the same instance without meaning to. Both take effect, so values are counted twice.

Verification application: The monitor of Module 9 collects one `alu_txn` per `valid_in`/`valid_out` pair and sends it through `mon2scb_mbx`. The scoreboard checks the transaction and then calls `cov.sample(txn)`. Each real ALU operation is therefore sampled exactly once, after it has been observed at the DUT pins.

Interview questions

Q1. What are the ways to sample a covergroup? (1) A sampling event in the covergroup declaration, such as `@(posedge clk)` or a named event; (2) an explicit call to the built-in `sample()` method, optionally overridden with arguments using `with function sample(...)`.

Q2. Which sampling method is preferred in a class-based testbench, and why? Calling `sample()` from the monitor or scoreboard when a complete, valid transaction has been observed. It gives precise control, avoids sampling idle or invalid data, avoids races with the clock, and is cheaper than sampling on every clock.

Q3. What is `with function sample`? A way to give a covergroup its own `sample` method with arguments. The arguments can be used as coverpoint expressions, so the values to be covered are passed directly in the call.

Q4. What is the difference between `iff` on a coverpoint and a sampling event? The sampling event decides when the whole covergroup is sampled. The `iff` guard is a per-coverpoint (or per-bin) condition evaluated at each sample; if it is false, that coverpoint is skipped for that sample.

Q5. How do you stop coverage collection during reset? Use a guard such as `iff (rst_n)` on the coverpoints, call `cg.stop()` when reset is asserted and `cg.start()` when it is released, or simply do not call `sample()` during reset.

Q6. Why can sampling with `@(posedge clk)` cause a race? The covergroup reads the current values when the event is processed. If the covered variable is updated by a blocking assignment in another process triggered by the same clock edge, the order of the two is not defined. Using a clocking block, the `strobe` option, the opposite edge, or procedural `sample()` calls avoids the race.

Part 2: Bins

2.1 Bins and Automatic Bins

In plain English

A coverpoint watches a variable. A **bin** is one box on the tally sheet for that variable: a named counter attached to a value, a range of values, or a sequence of values. Each time the coverpoint is sampled, the counter of every bin that contains the sampled value is increased by one. A bin is **covered** (hit) when its counter has reached the required number, which is 1 by default.

The coverage of a coverpoint is simply:

$$\text{coverage (\%)} = \frac{\text{number of bins that are covered}}{\text{total number of bins of the coverpoint}} \times 100$$

So the bins **are** the coverage model. Choosing them is the real design work in functional coverage: each bin should correspond to something the verification plan says must be tested.

Kinds of bins

Kind	Keyword	Created by	Counts towards coverage
Automatic (implicit) bins	none	The tool, when the coverpoint defines no bins	Yes
Explicit (user-defined) value bins	`bins`	The user	Yes
Wildcard bins	`wildcard bins`	The user	Yes
Transition bins	`bins x = (a => b)`	The user	Yes
Default bin	`bins x = default`	The user	**No**
Ignore bins	`ignore_bins`	The user	No: the values are removed from coverage
Illegal bins	`illegal_bins`	The user	No: a hit is a run-time error

Automatic bins: technical definition

If a coverpoint does not define any bins, the tool creates **automatic bins** (also called implicit bins) for it. The number of automatic bins is decided as follows.

Coverpoint type	Number of automatic bins	Bin names in reports
Enum	One bin per named member of the enum	`auto[NAME]`
Integral, `M` bits wide, with 2^M ≤ `auto_bin_max`	2^M bins, one per value	`auto[0]`, `auto[1]` ...

Coverpoint type	Number of automatic bins	Bin names in reports
Integral, `M` bits wide, with 2^M > `auto_bin_max`	`auto_bin_max` bins; the 2^M values are divided evenly among them	`auto[low:high]`

`auto_bin_max` is a coverage option whose default value is **64**.

When the values do not divide evenly, the **last bin** receives the remaining values. Example from the standard: a 3-bit variable with `auto_bin_max = 3` gets the bins `[0:1]`, `[2:3]` and `[4:7]`.

```
bit [1:0] mode;          4 values  <= 64   ->  4 bins : auto[0] auto[1] auto[2] auto[3]

bit [7:0] data;        256 values  >  64   -> 64 bins of 4 values each :
                                             auto[0:3] auto[4:7] ... auto[252:255]

bit [9:0] len;        1024 values  >  64   -> 64 bins of 16 values each

int       count;      2^32 values  >  64   -> 64 bins of 67,108,864 values each (useless)

alu_op_e  op;         8 names              ->  8 bins : auto[ALU_ADD] ... auto[ALU_SHR]
```

Automatic bins never include values that contain X or Z.

Complete example

tb_bins_auto.sv

```
module tb_bins_auto;

  typedef enum bit [1:0] {CMD_IDLE, CMD_RD, CMD_WR} cmd_e;

  bit [2:0] level;      //    8 values
  bit [9:0] len;        // 1024 values
  cmd_e     cmd;        //    3 named values

  covergroup cg_auto;
    cp_level: coverpoint level;
    cp_len  : coverpoint len { option.auto_bin_max = 4; }
    cp_cmd  : coverpoint cmd;
  endgroup

  cg_auto cg = new();

  initial begin
    level = 7;  len = 300;   cmd = CMD_RD;  cg.sample();
    level = 0;  len = 1000;  cmd = CMD_RD;  cg.sample();

    $display("cp_level = %0.2f%%", cg.cp_level.get_coverage());
    $display("cp_len   = %0.2f%%", cg.cp_len.get_coverage());
    $display("cp_cmd   = %0.2f%%", cg.cp_cmd.get_coverage());
  end

endmodule
```

EXPECTED OUTPUT

```
cp_level = 25.00%
cp_len   = 50.00%
cp_cmd   = 33.33%
```

Line by line

Code	Explanation
`cp_level: coverpoint level;`	3 bits, 8 values, fewer than 64: eight bins, one per value. The values 7 and 0 were sampled: 2 of 8.
`option.auto_bin_max = 4;`	Limits this coverpoint to four automatic bins. The 1024 values are split into `[0:255]`, `[256:511]`, `[512:767]`, `[768:1023]`. 300 is in the second and 1000 in the fourth: 2 of 4.
`cp_cmd: coverpoint cmd;`	An enum: one bin per name, three bins, even though the 2-bit base type has four possible values. The unused encoding 3 gets no bin. Only `CMD_RD` was sampled: 1 of 3.

When automatic bins are appropriate

Coverpoint	Automatic bins?	Reason
Enum (opcode, state, command)	Yes	One bin per name is exactly what is wanted
Small control field (1 to about 6 bits) where every value matters	Yes	One bin per value
1-bit flag or condition	Yes	Two bins
Data or address of 8 bits or more	No	Bins of arbitrary equal ranges do not correspond to anything in the specification; corner values such as 0 and the maximum are hidden inside large bins

Interview trap: "How many bins does `coverpoint a;` create for `bit [7:0] a`?" 64, not 256, because of the default `auto_bin_max` of 64. Each bin covers four values. For a `bit [3:0]` the answer is 16. For an enum with five members it is 5.

Common mistake: Leaving a 32-bit address with automatic bins. The 64 bins are hit quickly by random values, the coverpoint shows 100 percent, and nothing has been learned. Define explicit bins for the boundaries and regions that matter.

Interview questions

Q1. What is a bin? A named counter associated with a value, a set or range of values, or a sequence of values of a coverpoint. The percentage of bins that have been hit is the coverage of the coverpoint.

Q2. What are automatic bins? Bins created by the tool when the coverpoint defines none. An enum gets one bin per member; an integral coverpoint of M bits gets the smaller of 2^M and `auto_bin_max` bins.

Q3. What is `auto_bin_max` and what is its default? The maximum number of automatic bins for a coverpoint. The default is 64.

Q4. How are values distributed when there are more values than automatic bins? Evenly across the bins, in increasing order; if the division is not exact, the last bin receives the extra values.

Q5. A coverpoint is on a 4-bit enum variable with 5 members. How many automatic bins? Five. The number is determined by the enum members, not by the width.

2.2 Explicit Bins

In plain English

With explicit bins you decide what the boxes are. Instead of "64 equal slices of the address range", you write "the first address, the last address, the boundary of each memory region, and everything in between as one box". Each box carries a name that appears in the coverage report, so a hole reads as "bin `max` of `cp_a` was never hit" rather than "bin `auto[252:255]`".

Technical definition

Explicit bins are declared with the `bins` keyword inside the braces of a coverpoint. Each declaration associates a bin name with a set of values of the coverpoint expression. When a coverpoint has at least one explicit bin, **no automatic bins are created**: values that fall in no bin are simply not counted.

Syntax forms

Declaration	Bins created	Meaning
`bins zero = {0};`	1	One bin for one value
`bins low = {[1:15]};`	1	One bin for a range: hit if any value from 1 to 15 is sampled
`bins some = {1, 5, [8:11]};`	1	One bin for a list of values and ranges
`bins val[] = {[0:3]};`	4	`[]` creates one bin **per value**: `val[0]`, `val[1]`, `val[2]`, `val[3]`
`bins pow[] = {1, 2, 4, 8};`	4	One bin per listed value
`bins grp[4] = {[0:15]};`	4	A fixed number of bins; the 16 values are distributed evenly: `[0:3]`, `[4:7]`, `[8:11]`, `[12:15]`
`bins grp[3] = {[0:9]};`	3	10 values in 3 bins: `{0,1,2}`, `{3,4,5}`, `{6,7,8,9}`. The last bin takes the remainder
`bins top = {[200:$]};`	1	`$` means the largest value of the coverpoint type (as a lower bound it means the smallest)
`bins others = default;`	1	Catches every value not in another bin. **Not counted** in the coverage percentage
`bins b = {[0:3]} iff (en);`	1	The bin is incremented only if the guard is true at the sample
`bins mul4[] = {[0:255]} with (item % 4 == 0);`	64	IEEE 1800-2012 `with` filter: keep only values satisfying the expression. Tool support is uneven

```
coverpoint a   (bit [7:0])

   bins zero   = {0};            one bin             |0|
   bins low    = {[1:15]};       one bin             |1 ............ 15|
   bins mid[4] = {[16:239]};     four bins           |16..71|72..127|128..183|184..239|
   bins high   = {[240:254]};    one bin             |240 ........ 254|
   bins max    = {255};          one bin             |255|
                                                     -------------------------------------
                                                     total: 8 bins
```

The difference between `bins b`, `bins b[]` and `bins b[N]`

For the same value set `{[0:7]}`:

Declaration	Number of bins	Fully covered when
`bins b = {[0:7]};`	1	**Any one** of the values 0 to 7 has been sampled
`bins b[] = {[0:7]};`	8	**Every** value 0 to 7 has been sampled
`bins b[2] = {[0:7]};`	2	At least one value from 0 to 3 **and** one from 4 to 7 have been sampled

This is the most frequently asked bin question. A single bin over a range is a weak requirement; an array of bins is a strong one.

Overlapping bins and values with no bin

- A value may belong to more than one bin. All matching bins are incremented.
- A value that belongs to no bin (and there is no `default` bin) is ignored: nothing is incremented and nothing is reported.

Complete example

tb_bins_explicit.sv

```
module tb_bins_explicit;

  bit [7:0] a;

  covergroup cg_a;
    cp_a: coverpoint a {
      bins zero   = {0};
      bins low    = {[1:15]};
      bins mid[4] = {[16:239]};
      bins high   = {[240:254]};
      bins max    = {255};
    }
    cp_pow2: coverpoint a {
      bins pow2[] = {1, 2, 4, 8, 16, 32, 64, 128};
      bins others = default;
    }
  endgroup

  cg_a cg = new();

  initial begin
    a = 0;    cg.sample();
    a = 5;    cg.sample();
    a = 9;    cg.sample();
    a = 100;  cg.sample();
    a = 255;  cg.sample();
    a = 200;  cg.sample();
    a = 64;   cg.sample();

    $display("cp_a    = %0.2f%%", cg.cp_a.get_coverage());
    $display("cp_pow2 = %0.2f%%", cg.cp_pow2.get_coverage());
    $display("cg_a    = %0.2f%%", cg.get_coverage());
  end

endmodule
```

EXPECTED OUTPUT

```
cp_a    = 75.00%
cp_pow2 = 12.50%
cg_a    = 43.75%
```

Which bin does each sample hit?

Sample	`cp_a` bin	`cp_pow2` bin
0	`zero`	`others` (default)
5	`low`	`others`
9	`low` (already hit)	`others`
100	`mid[1]` (72 to 127)	`others`
255	`max`	`others`
200	`mid[3]` (184 to 239)	`others`
64	`mid[0]` (16 to 71)	`pow2[64]`

Explanation

Item	Calculation
`cp_a`	Eight bins: `zero`, `low`, `mid[0]` .. `mid[3]`, `high`, `max`. Hit: `zero`, `low`, `mid[0]`, `mid[1]`, `mid[3]`, `max`. 6 / 8 = 75 percent. The holes are `mid[2]` and `high`.
`mid[4]`	16 to 239 is 224 values; divided among 4 bins that is 56 values each: 16-71, 72-127, 128-183, 184-239.
`cp_pow2`	Eight counted bins, one per power of two. Only `pow2[64]` was hit: 1 / 8 = 12.5 percent. The `default` bin collected six samples but does not count, in either the numerator or the denominator.
`cg_a`	(75 + 12.5) / 2 = 43.75 percent.
Two coverpoints on one variable	Legal and common: the same variable is classified in two different ways.

What is the default bin for?

`bins others = default;` shows, in the report, how many samples fell outside the bins you defined. It is a debugging aid: a large count in `others` may mean the bin definitions miss something. It never contributes to the percentage, and it cannot be used in a cross.

> **Interview trap:** `bins b = {[0:255]};` on an 8-bit variable is covered by the very first sample, whatever its value: one bin, any value hits it. The coverpoint shows 100 percent after one transaction. If the intention is "all 256 values", the declaration must be `bins b[] = {[0:255]};`.

> **Common mistake:** Assuming that values outside the explicit bins are still covered by automatic bins. Once a single explicit bin exists, automatic bins are not created. If `cp_a` above had only `bins zero = {0};`, the coverpoint would have exactly one bin and would reach 100 percent as soon as a zero was sampled.

> **Verification application:** Bins are derived from the specification by asking "which values would a designer treat differently?". Typical choices for a data operand: zero, one, all ones, the sign boundary (`7F` / `80`), and a few ranges. For a length field: minimum, maximum, the values just inside and just outside each legal limit. For an address: the first and last address of each region.

Interview questions

Q1. What is the difference between `bins b = {[0:3]};` and `bins b[] = {[0:3]};`? The first creates one bin that is hit by any of the four values. The second creates four bins, one per value, so all four values must occur for full coverage.

Q2. What does `bins b[2] = {[0:9]};` create? Two bins with the ten values distributed evenly: 0 to 4 in `b[0]` and 5 to 9 in `b[1]`.

Q3. What does `bins b[3] = {[0:9]};` create? Three bins. Ten values do not divide by three, so the first two bins get three values each and the last gets the remaining four: {0,1,2}, {3,4,5}, {6,7,8,9}.

Q4. If a coverpoint has explicit bins, are automatic bins also created for the remaining values? No. Values not covered by any explicit bin are not tracked at all.

Q5. What is a default bin? Does it count towards coverage? `bins name = default;` catches all values not included in any other bin. It is reported but does not count towards the coverage percentage.

Q6. What does `$` mean in a bin range? The maximum value of the coverpoint type when used as the upper bound (`[200:$]`), and the minimum when used as the lower bound (`[$:10]`).

Q7. Can one value belong to two bins? Yes. Overlapping bins are allowed and each matching bin is incremented.

2.3 Wildcard Bins

In plain English

Sometimes only some bits of a value matter. "All opcodes whose top two bits are 01" or "any odd value" are patterns, not ranges. A wildcard bin lets you write a pattern in which certain bit positions mean "any value".

Technical definition

A bin declared with the `wildcard` keyword treats every **X**, **Z** or **?** in its value specification as a don't-care bit position. A sampled value matches the bin if it equals the pattern in all the other bit positions. This is the same matching rule as the wildcard equality operator `==?` (Module 1, Section 4.6).

Syntax

```
wildcard bins odd      = {4'b???1};     // 1, 3, 5, 7, 9, 11, 13, 15       -> one bin
wildcard bins upper    = {4'b1???};     // 8 to 15                          -> one bin
wildcard bins rd       = {4'b00??};     // 0 to 3                           -> one bin
wildcard bins each[]   = {4'b11??};     // 12, 13, 14, 15                   -> four bins
wildcard bins t        = (2'b0? => 2'b1?);   // wildcard transition bin
```

Without the `wildcard` keyword, an X or Z in a bin value is taken literally: the bin would match only a sampled value that really has X or Z in that position.

How it works

```
wildcard bins rd = {4'b00??};

pattern      0  0  ?  ?
sampled      0  0  1  0     -> bits 3 and 2 match, bits 1 and 0 are don't-care  -> HIT
sampled      0  1  1  0     -> bit 2 differs                                    -> no hit
```

A complete example that uses wildcard bins together with ignore and illegal bins follows in Section 2.5.

> **Common mistake:** Forgetting the keyword: `bins odd = {4'b???1};`. The `?` characters are then literal Z values, no two-state sample can ever match, and the bin stays empty forever.

> **Interview trap:** "How many bins does `wildcard bins b[] = {3'b1??};` create?" Four: the pattern matches the values 4, 5, 6 and 7, and `[]` creates one bin for each.

Interview questions

Q1. What is a wildcard bin? A bin whose value contains X, Z or `?` characters that are treated as don't-care bits, so that the bin matches every value fitting the pattern.

Q2. Write a bin that is hit by any even value of a 4-bit coverpoint. `wildcard bins even = {4'b???0};`

Q3. What happens if `wildcard` is omitted but the value contains `?`? The `?` is interpreted as a literal Z bit. The bin only matches a sampled value that has Z in those positions, which normally never happens.

2.4 Ignore Bins

In plain English

Some values of a variable are not interesting, or cannot occur at all. A reserved opcode, an address in an unmapped region, a length that the protocol forbids and the generator never produces. If such values stay in the coverage model, the coverage can never reach 100 percent. `ignore_bins` removes them from the model.

Technical definition

`ignore_bins` specifies a set of values or transitions that are **excluded from coverage**. Ignored values:

- do not increment any counted bin, even if they are also included in another bin;
- are removed from the bins of the coverpoint, so they do not appear in the **denominator** of the coverage calculation;
- produce no error when sampled.

Syntax

```
cp_cmd: coverpoint cmd {
  bins         val[]    = {[0:15]};
  ignore_bins  reserved = {[12:13]};       // val[12] and val[13] are removed
}

cp_op: coverpoint op {
  ignore_bins not_used = {ALU_NOT};        // automatic bins for the other members
}

ignore_bins no_trans = (CMD_WR => CMD_WR); // an ignored transition
wildcard ignore_bins hi = {4'b11??};       // ignore by pattern
```

Effect on the calculation

```
   coverpoint cmd (bit [3:0])  { bins val[] = {[0:15]}; }                  16 bins
   sampled values: 0 to 11                                                 12 / 16 = 75 %

   coverpoint cmd (bit [3:0])  { bins val[] = {[0:15]};
                                 ignore_bins reserved = {[12:15]}; }       12 bins
   sampled values: 0 to 11                                                 12 / 12 = 100 %
```

The second form is how a coverage model is made to describe exactly the reachable, legal space.

> **Common mistake:** Using `ignore_bins` to hide a hole that you could not hit. Every ignored value must be justified by the specification ("reserved", "not supported in this configuration") and reviewed. Ignoring a legal scenario because it is hard to generate removes it from verification without anyone noticing.

Interview questions

Q1. What are ignore bins? Bins that list values or transitions to be excluded from coverage. They are not counted and they reduce the total number of bins used for the percentage.

Q2. If a value is in both a normal bin and an ignore bin, what happens? The ignore bin wins. The value is excluded from the normal bin.

Q3. Does sampling an ignored value cause an error? No. It is silently ignored. That is the difference from an illegal bin.

2.5 Illegal Bins

In plain English

Some values must **never** be seen. If they are seen, something is wrong with the DUT or the testbench. `illegal_bins` marks such values: sampling one produces a run-time error.

Technical definition

`illegal_bins` specifies a set of values or transitions that are illegal. If an illegal value or transition is sampled, the simulator issues a **run-time error**. Illegal bins are excluded from coverage in the same way as ignore bins, and they take precedence over every other bin, including ignore bins.

Syntax

```
cp_cmd: coverpoint cmd {
  bins         val[] = {[0:11]};
  illegal_bins bad   = {[14:15]};
}

illegal_bins bad_trans = (CMD_IDLE => CMD_IDLE [*10]);   // an illegal transition
```

Ignore bins versus illegal bins

Property	`ignore_bins`	`illegal_bins`
Meaning	"Not interesting; do not count it"	"Must never happen"
When the value is sampled	Nothing happens	Run-time error
Counted in the coverage percentage	No	No
Removes the value from other bins	Yes	Yes
Precedence	Lower	Higher: if a value is in both, it is illegal
Typical use	Reserved or unreachable values, combinations not applicable	Encodings that indicate a bug if observed

Complete example: wildcard, ignore and illegal bins together

tb_bins_special.sv

```
module tb_bins_special;

  bit [3:0] cmd;

  covergroup cg_cmd;
    // classification by pattern
    cp_kind: coverpoint cmd {
      wildcard bins read   = {4'b00??};     // 0..3   one bin
      wildcard bins write  = {4'b01??};     // 4..7   one bin
      wildcard bins ctrl[] = {4'b10??};     // 8..11  four bins
    }
    // every legal value separately
    cp_val: coverpoint cmd {
      bins         val[]    = {[0:15]};
      ignore_bins  reserved = {[12:13]};
      illegal_bins bad      = {[14:15]};
    }
  endgroup

  cg_cmd cg = new();

  initial begin
    cmd = 1;   cg.sample();
    cmd = 5;   cg.sample();
    cmd = 9;   cg.sample();
    cmd = 12;  cg.sample();     // reserved: ignored, no error
    cmd = 12;  cg.sample();
    // cmd = 14; cg.sample();   // illegal: this line would cause a run-time error

    $display("cp_kind = %0.2f%%", cg.cp_kind.get_coverage());
    $display("cp_val  = %0.2f%%", cg.cp_val.get_coverage());
    $display("cg_cmd  = %0.2f%%", cg.get_coverage());
  end

endmodule
```

EXPECTED OUTPUT

```
cp_kind = 50.00%
cp_val  = 25.00%
cg_cmd  = 37.50%
```

Line by line

Code	Explanation
`wildcard bins read = {4'b00??};`	One bin for the values 0 to 3. Sample 1 hits it.
`wildcard bins write = {4'b01??};`	One bin for 4 to 7. Sample 5 hits it.
`wildcard bins ctrl[] = {4'b10??};`	Four bins, for 8, 9, 10 and 11. Sample 9 hits one of them.
`cp_kind`	Six bins in total (1 + 1 + 4); three were hit: 50 percent. The value 12 matches no pattern, so it is not counted.
`bins val[] = {[0:15]};`	Sixteen bins, one per value, before exclusions.
`ignore_bins reserved = {[12:13]};`	Removes `val[12]` and `val[13]`. The two samples of 12 increment nothing.

Code	Explanation
`illegal_bins bad = {[14:15]};`	Removes `val[14]` and `val[15]` and makes them errors.
`cp_val`	16 - 2 - 2 = 12 counted bins. Hit: the bins for 1, 5 and 9. 3 / 12 = 25 percent.
Commented line	If `cmd = 14` were sampled, the simulator would report an illegal-bin error. The wording and the severity are tool-specific.

A common shorter idiom is to write only the exclusions and let the tool create automatic bins for the remaining values:

```
cp_op: coverpoint txn.op {
  ignore_bins not_in_this_config = {ALU_SHL, ALU_SHR};   // 6 automatic bins remain
}
```

Interview trap: "Should illegal bins be used to check the DUT?" Not as the only check. Coverage can be switched off for speed, or excluded in some regressions, and then the check silently disappears. A condition that must never occur should be checked by an assertion or by the scoreboard; an illegal bin is at most a second line of defence.

Common mistake: Expecting the simulation to stop at an illegal bin. The standard only requires a run-time error message; whether the simulation stops depends on the tool and its settings.

Interview questions

Q1. What are illegal bins? Bins that declare values or transitions that must never occur. Sampling one produces a run-time error, and the values are excluded from coverage.

Q2. What is the difference between ignore bins and illegal bins? Both exclude values from coverage. An ignored value is skipped silently. An illegal value causes a run-time error. If a value is in both, the illegal bin takes precedence.

Q3. Do ignore and illegal bins change the coverage percentage? Yes. The excluded values are removed from the counted bins, which reduces the denominator, so the same set of hits gives a higher percentage.

Q4. Would you use an illegal bin or an assertion to catch an illegal opcode? An assertion (or a scoreboard check). Checking should not depend on coverage being enabled. An illegal bin can be added as well, to document the coverage model.

2.6 Transition Bins

In plain English

Value bins record **which values** occurred. They cannot tell you in **which order**. For a state machine, the order is the whole point: was the transition from `EXEC` to `DONE` taken? Did the state stay in `EXEC` for three cycles? Did a write follow a write? A transition bin is hit by a **sequence of values** seen on consecutive samples of the coverpoint.

Technical definition

A transition bin specifies one or more ordered sequences of coverpoint values, written with the `=>` operator inside parentheses. The bin is incremented when the values of the coverpoint at **successive sampling points** match the complete sequence. "Successive" refers to samples of the covergroup, not to clock cycles (they are the same only if the covergroup is sampled on every clock).

Syntax forms

Declaration	Sequence(s) that hit the bin
`bins t = (1 => 2);`	1 followed by 2 on the next sample
`bins t = (1 => 2 => 3);`	1, then 2, then 3 on three consecutive samples
`bins t = (1, 2 => 3, 4);`	A set of transitions: 1=>3, 1=>4, 2=>3, 2=>4. **One** bin, hit by any of them
`bins t[] = (1, 2 => 3, 4);`	The same four transitions as **four separate bins**
`bins t = (1 => 2), (3 => 4);`	Two sequences in one bin; either hits it
`bins t = (5 [*3]);`	Consecutive repetition: 5 => 5 => 5
`bins t = (5 [*2:4]);`	5 repeated 2, 3 or 4 times. With `t[]`: three separate bins
`bins t = (1 => 5 [->2] => 7);`	"Goto" repetition: 1, then any values until 5 has occurred twice (not necessarily adjacent), then 7 **immediately** after the second 5
`bins t = (1 => 5 [=2] => 7);`	Non-consecutive repetition: as above, but other values may also occur between the second 5 and the 7
`bins t = default sequence;`	All transitions not covered by other transition bins. Not counted
`ignore_bins t = (2 => 1);` / `illegal_bins t = (2 => 1);`	Exclude a transition / make it an error

```
samples of the coverpoint over time:       1   5   3   5   7

(1 => 5)                hit at sample 2        ^
(5 => 7)                hit at sample 5                    ^
(1 => 5 [->2] => 7)     hit at sample 5   1 ... 5 ..... 5  7   ^   (7 right after 2nd 5)
(1 => 5 => 7)           NOT hit: after the first 5 comes 3, not 7
```

How it works internally

For each transition bin the simulator keeps a small matcher that remembers how much of the sequence has been seen so far. At every sample it compares the new value with the next expected value. Because the matcher needs the **previous** samples, the first sample of a covergroup instance can only start a sequence, never complete a two-value transition.

There are **no automatic transition bins**. Transitions are covered only if you declare them.

Complete example

tb_bins_transition.sv

```
module tb_bins_transition;

  typedef enum bit [1:0] {IDLE, FETCH, EXEC, DONE} state_e;

  state_e state;

  covergroup cg_state;
    cp_state: coverpoint state {
      bins idle_to_fetch = (IDLE => FETCH);
      bins full_cycle    = (IDLE => FETCH => EXEC => DONE);
      bins stay_exec     = (EXEC [*3]);
      bins fetch_to_done = (FETCH => DONE [->1]);
      bins to_idle[]     = (FETCH, EXEC, DONE => IDLE);
    }
  endgroup

  cg_state cg = new();

  initial begin
    state = IDLE;   cg.sample();    // sample 1
    state = FETCH;  cg.sample();    // sample 2
    state = EXEC;   cg.sample();    // sample 3
    state = EXEC;   cg.sample();    // sample 4
    state = EXEC;   cg.sample();    // sample 5
    state = DONE;   cg.sample();    // sample 6
    state = IDLE;   cg.sample();    // sample 7

    $display("cp_state = %0.2f%%", cg.cp_state.get_coverage());
  end

endmodule
```

EXPECTED OUTPUT

```
cp_state = 57.14%
```

Which bins are hit?

The sampled sequence is `IDLE, FETCH, EXEC, EXEC, EXEC, DONE, IDLE`.

Bin	Sequence required	Hit?	Reason
`idle_to_fetch`	IDLE => FETCH	Yes	Samples 1 and 2
`full_cycle`	IDLE => FETCH => EXEC => DONE	**No**	After IDLE, FETCH, EXEC the next sample is EXEC, not DONE. The sequence requires consecutive samples
`stay_exec`	EXEC => EXEC => EXEC	Yes	Samples 3, 4 and 5
`fetch_to_done`	FETCH, then the first later DONE	Yes	FETCH at sample 2; the first DONE is at sample 6
`to_idle[]` (FETCH => IDLE)		No	
`to_idle[]` (EXEC => IDLE)		No	
`to_idle[]` (DONE => IDLE)		Yes	Samples 6 and 7

Seven bins in total (1 + 1 + 1 + 1 + 3), four hit: 4 / 7 = 57.14 percent.

Line by line

Code	Explanation
`bins idle_to_fetch = (IDLE => FETCH);`	A single two-value transition. Enum names can be used directly.
`bins full_cycle = (IDLE => FETCH => EXEC => DONE);`	A four-value sequence. It would be hit only if the machine spent exactly one sample in each state. The hole tells you that this scenario (a one-cycle execute phase) has not been tested.
`bins stay_exec = (EXEC [*3]);`	Three consecutive samples of `EXEC`.
`bins fetch_to_done = (FETCH => DONE [->1]);`	Goto repetition: after `FETCH`, any number of other values, then `DONE`. It tolerates the three `EXEC` samples in between.
`bins to_idle[] = (FETCH, EXEC, DONE => IDLE);`	A set on the left produces three transitions; `[]` makes each a separate bin, so the report shows exactly which return path to `IDLE` is missing.

Value bins versus transition bins

Property	Value bins	Transition bins
What is covered	A value at one sample	A sequence of values over consecutive samples
Syntax	`bins b = {1, 2};` (braces)	`bins b = (1 => 2);` (parentheses)
Automatic bins	Yes	No, they must be written
Needs history	No	Yes: previous samples are remembered
Typical use	Opcodes, data ranges, lengths	FSM transitions, back-to-back operations, command sequences
Alternative		Temporal behaviour on signals can also be covered with `cover property` (Part 5)

Interview trap: "The FSM coverpoint shows that all four states were visited. Is the FSM covered?" No. State (value) coverage does not show that each legal **transition** was taken. An FSM needs transition bins for every arc of the state diagram, and usually `illegal_bins` for the arcs that must not exist.

Interview trap: Braces versus parentheses. `bins b = {1, 2};` is a value bin hit by 1 or by 2. `bins b = (1 => 2);` is a transition bin hit by 1 followed by 2.

Common mistake: Forgetting that transitions are counted between **samples**, not between clock cycles. If the covergroup is sampled once per transaction, `(ALU_ADD => ALU_SUB)` means "an ADD transaction followed by a SUB transaction", however many idle clocks were between them. If the covergroup is sampled on every clock, a state held for several cycles produces `EXEC => EXEC` transitions that you may have to allow for.

> **Verification application:** For the ALU, transition bins on the opcode check back-to-back behaviour: the same operation twice (`ALU_ADD => ALU_ADD`), and an operation that produces a carry followed by one that must not inherit it. For a FIFO testbench, transitions on a "fill level" classification (`EMPTY => PARTIAL => FULL => PARTIAL => EMPTY`) prove that the FIFO was filled and drained completely.

Interview questions

Q1. What is a transition bin? A bin that is hit when the coverpoint takes a specified sequence of values on consecutive samples, written as `(a => b => c)`.

Q2. How many bins does `bins t[] = (1, 2 => 3, 4);` create, and which transitions? Four: 1=>3, 1=>4, 2=>3 and 2=>4. Without `[]` it would be one bin hit by any of the four.

Q3. What does `(3 [*4])` mean? The value 3 on four consecutive samples: 3 => 3 => 3 => 3.

Q4. What is the difference between `[*n]`, `[->n]` and `[=n]` in a transition bin? `[*n]` is consecutive repetition. `[->n]` (goto) means the value occurs n times, not necessarily consecutively, and the next element of the sequence follows immediately after the last occurrence. `[=n]` (non-consecutive) is the same, except that other values may also appear between the last occurrence and the next element.

Q5. Are transition bins created automatically? No. Only value bins are created automatically. Every transition of interest must be declared.

Q6. How would you cover all transitions of a four-state FSM? Declare a transition bin for each legal arc, for example `bins legal[] = (IDLE => FETCH), (FETCH => EXEC), (EXEC => EXEC), (EXEC => DONE), (DONE => IDLE);`, and declare the arcs that must never happen as `illegal_bins`.

Part 3: Cross Coverage, Options and Coverage-Driven Verification

3.1 Cross Coverage

In plain English

Suppose a coverpoint shows that every ALU opcode was used, and another shows that operand `a` was zero, small, large and maximum. Both are at 100 percent. Was a **subtraction** ever done with `a` at its **maximum**? You cannot tell. Each coverpoint is a separate tally sheet.

Cross coverage is a two-dimensional tally sheet: one row per bin of the first coverpoint, one column per bin of the second, and a box for each combination. It records which **combinations** occurred at the same sample.

Technical definition

A `cross` specifies cross coverage between two or more coverpoints (or variables) of the same covergroup. The set of **cross bins** is the Cartesian product of the bins of the crossed coverpoints. A cross bin is hit when, at one sample, each of the crossed coverpoints hits the corresponding bin. The user may regroup, ignore or declare illegal particular cross products with `bins`, `ignore_bins` and `illegal_bins` selected by `binsof` expressions.

Syntax

```
covergroup cg;
  cp_a: coverpoint a;
  cp_b: coverpoint b;

  x_ab : cross cp_a, cp_b;                // automatic cross bins

  x_ab2: cross cp_a, cp_b {               // user-controlled cross bins
    bins         name = binsof(cp_a.low) && binsof(cp_b) intersect {[0:3]};
    ignore_bins  name = binsof(cp_a) intersect {7};
    illegal_bins name = binsof(cp_a.max) && binsof(cp_b.max);
  }
endgroup
```

How it works internally

```
cp_kind : 4 bins (k0 k1 k2 k3)          cp_dir : 2 bins (rd wr)

x_kind_dir : cross cp_kind, cp_dir   ->   4 x 2 = 8 cross bins

                rd        wr
           +---------+---------+
      k0   | <k0,rd> | <k0,wr> |
           +---------+---------+
      k1   | <k1,rd> | <k1,wr> |
           +---------+---------+
      k2   | <k2,rd> | <k2,wr> |
           +---------+---------+
      k3   | <k3,rd> | <k3,wr> |
           +---------+---------+

at each sample: kind hits one row, dir hits one column -> one cross bin is incremented
cross coverage = (cross bins hit) / (total cross bins)
```

Important rules:

Rule	Detail
The cross works on **bins**, not on raw values	Crossing a coverpoint with 4 bins and one with 2 bins gives 8 cross bins, whatever the widths of the variables
Number of automatic cross bins	The product of the numbers of bins of the crossed coverpoints
Bins not included	`default`, `ignore_bins` and `illegal_bins` of the coverpoints do not take part in the cross
Same covergroup	Only coverpoints of the same covergroup can be crossed
Crossing a variable directly	`cross a, b;` is allowed: an implicit coverpoint with automatic bins is created for each variable
More than two	`cross cp_a, cp_b, cp_c;` is allowed; the number of bins is the product of all three
Coverage contribution	The cross is one more item in the covergroup, with its own weight, alongside the coverpoints

Complete example 1: automatic cross bins

tb_cross_basic.sv

```
module tb_cross_basic;

  bit [1:0] kind;
  bit       dir;       // 0 = read, 1 = write

  covergroup cg_xfer;
    cp_kind   : coverpoint kind;              // 4 bins
    cp_dir    : coverpoint dir;               // 2 bins
    x_kind_dir: cross cp_kind, cp_dir;        // 8 cross bins
  endgroup

  cg_xfer cg = new();

  initial begin
    kind = 0;  dir = 0;  cg.sample();
    kind = 0;  dir = 1;  cg.sample();
    kind = 1;  dir = 0;  cg.sample();
    kind = 3;  dir = 1;  cg.sample();
    kind = 0;  dir = 0;  cg.sample();

    $display("cp_kind    = %0.2f%%", cg.cp_kind.get_coverage());
    $display("cp_dir     = %0.2f%%", cg.cp_dir.get_coverage());
    $display("x_kind_dir = %0.2f%%", cg.x_kind_dir.get_coverage());
    $display("cg_xfer    = %0.2f%%", cg.get_coverage());
  end

endmodule
```

EXPECTED OUTPUT

```
cp_kind    = 75.00%
cp_dir     = 100.00%
x_kind_dir = 50.00%
cg_xfer    = 75.00%
```

Explanation

Item	Calculation
`cp_kind`	Values 0, 1 and 3 were sampled: 3 of 4 bins.
`cp_dir`	Both values were sampled: 2 of 2.
`x_kind_dir`	Combinations sampled: (0,rd), (0,wr), (1,rd), (3,wr), and (0,rd) again. Four different cross bins of eight: 50 percent.
`cg_xfer`	(75 + 100 + 50) / 3 = 75 percent.
The lesson	`cp_dir` is at 100 percent and `kind` 1 was sampled, yet a write of kind 1 never happened. Only the cross shows that hole.

Selecting cross bins with binsof and intersect

Expression	Selects
`binsof(cp_a)`	All bins of `cp_a`
`binsof(cp_a.low)`	The bin `low` of `cp_a`
`binsof(cp_a) intersect {[0:3]}`	The bins of `cp_a` that contain at least one of the values 0 to 3
`!binsof(cp_a.low)`	All bins of `cp_a` except `low`

Expression	Selects
`expr1 && expr2`	Cross products that satisfy both selections
`expr1 || expr2`	Cross products that satisfy either selection

Used inside the braces of a cross:

- `bins name = <selection>;` groups all the selected cross products into **one** user-defined cross bin.
- `ignore_bins name = <selection>;` removes the selected cross products from coverage.
- `illegal_bins name = <selection>;` makes the selected cross products errors.

Cross products that are not selected by any user-defined `bins`, `ignore_bins` or `illegal_bins` remain as automatic cross bins.

Complete example 2: user-defined and ignored cross bins

`tb_cross_bins.sv`

```
module tb_cross_bins;

  bit [1:0] kind;
  bit       dir;

  covergroup cg_xfer;
    cp_kind: coverpoint kind;
    cp_dir : coverpoint dir;

    x_kind_dir: cross cp_kind, cp_dir {
      // kind 0 with any direction counts as one scenario
      bins        kind0_any = binsof(cp_kind) intersect {0};
      // kind 3 is read-only: a write of kind 3 cannot happen
      ignore_bins kind3_wr  = binsof(cp_kind) intersect {3} && binsof(cp_dir) intersect {1};
    }
  endgroup

  cg_xfer cg = new();

  initial begin
    kind = 0;  dir = 0;  cg.sample();
    kind = 0;  dir = 1;  cg.sample();
    kind = 1;  dir = 0;  cg.sample();
    kind = 3;  dir = 1;  cg.sample();     // ignored cross product
    kind = 0;  dir = 0;  cg.sample();

    $display("x_kind_dir = %0.2f%%", cg.x_kind_dir.get_coverage());
  end

endmodule
```

EXPECTED OUTPUT

```
x_kind_dir = 33.33%
```

How the cross bins are formed

Cross product (`kind`, `dir`)	Becomes	Hit?
(0, rd), (0, wr)	One user bin `kind0_any`	Yes
(1, rd)	Automatic cross bin	Yes
(1, wr)	Automatic cross bin	No

Cross product (`kind`, `dir`)	Becomes	Hit?
(2, rd)	Automatic cross bin	No
(2, wr)	Automatic cross bin	No
(3, rd)	Automatic cross bin	No
(3, wr)	Removed by `ignore_bins kind3_wr`	Not counted

Six cross bins remain (one user bin and five automatic bins); two were hit: 2 / 6 = 33.33 percent.

Line by line

Code	Explanation
`bins kind0_any = binsof(cp_kind) intersect {0};`	Selects every cross product whose `cp_kind` bin contains the value 0: (0,rd) and (0,wr). They are merged into one bin, so either one covers it.
`ignore_bins kind3_wr = ... && ...;`	Selects the single cross product (3,wr) and removes it. The sample `kind=3, dir=1` therefore changes nothing in the cross (it still counts in `cp_kind` and `cp_dir`).
Remaining products	Not mentioned in the braces, so they stay as automatic cross bins.

Controlling the size of a cross

The number of cross bins grows as a product. Crossing three coverpoints with automatic bins on 8-bit variables gives 64 × 64 × 64 = 262,144 bins, which can never be filled and makes the report useless.

Technique	Effect
Define meaningful explicit bins on each coverpoint first	An 8 × 4 cross (32 bins) instead of 8 × 64 (512 bins)
`ignore_bins` in the cross	Removes combinations that are impossible or irrelevant
Group with `bins x = binsof(...)`	Many products become one scenario
Cross only what the plan asks for	Not every pair of coverpoints needs a cross
Coverpoints used only as cross inputs: `option.weight = 0`	The coverpoint does not count on its own; only the cross counts

Coverpoint versus cross

Property	Coverpoint	Cross
What it covers	Values of one expression	Simultaneous combinations of bins of two or more coverpoints
Number of bins	Defined by its bins	Product of the bin counts of the crossed coverpoints
Bin selection syntax	`bins b = {values};`	`bins b = binsof(...) [intersect {...}];`
Can find	"This value never occurred"	"These two things never occurred together"
Cost	Small	Can explode; must be designed

> **Interview trap:** "Both coverpoints are 100 percent, so the cross must be 100 percent." No. Each coverpoint can be completely covered by different samples. The cross requires the combinations to occur in the same sample. The reverse is true: if a cross of automatic bins is 100 percent, both coverpoints are 100 percent.

Interview trap: "How many bins in `cross a, b` if `a` is `bit [3:0]` and `b` is `bit [7:0]`?" 16 × 64 = 1024, because `b` gets only 64 automatic bins.

Common mistake: Trying to cross coverpoints from two different covergroups, or crossing a coverpoint with a `default` bin and expecting the default bin to appear in the cross. Neither is possible.

Verification application: The ALU verification plan says "every operation must be tested with each operand at its boundary values". That sentence is a cross: `cross cp_op, cp_a` and `cross cp_op, cp_b`, with the single-operand operations (`ALU_NOT`, `ALU_SHL`, `ALU_SHR`) ignored in the cross with `b`.

Interview questions

Q1. What is cross coverage? Coverage of combinations: it records which combinations of bins of two or more coverpoints occurred at the same sampling point.

Q2. How many cross bins does a cross create by default? The product of the number of bins of each crossed coverpoint.

Q3. What are `binsof` and `intersect`? `binsof(cp)` or `binsof(cp.bin)` selects bins of a coverpoint inside a cross. `intersect {values}` restricts the selection to bins that contain at least one of the given values. They are combined with `&&`, `||` and `!` to define, ignore or make illegal particular cross products.

Q4. How do you exclude a combination from a cross? With `ignore_bins` in the cross body, for example `ignore_bins x = binsof(cp_op) intersect {ALU_NOT} && binsof(cp_b);`.

Q5. Can you cross variables directly without declaring coverpoints? Yes. `cross a, b;` creates implicit coverpoints with automatic bins for `a` and `b`. Declaring labelled coverpoints with meaningful bins is preferred.

Q6. Can coverpoints of different covergroups be crossed? No. A cross can only reference coverpoints (or variables) within its own covergroup.

Q7. A cross has 10,000 bins and coverage is stuck at 3 percent. What do you do? The cross is badly designed. Reduce the bins of the crossed coverpoints to the ranges that matter, ignore impossible combinations, and group equivalent ones. A cross should have a number of bins that the regression can realistically fill.

3.2 Coverage Options

In plain English

Options are the settings of a covergroup: how many hits a bin needs before it counts, how important one coverpoint is compared with another, whether each instance is reported separately, how many automatic bins to create.

Technical definition

There are two sets of options.

- `option`: instance-specific options. They apply to a particular covergroup instance (or to a coverpoint or cross of it).

- `type_option`: type-specific options. They apply to the covergroup type as a whole, across all its instances.

Syntax

```
covergroup cg;
  option.per_instance = 1;                // covergroup level
  option.at_least     = 2;                // default for all coverpoints and crosses
  type_option.comment = "bus coverage";

  cp_a: coverpoint a {
    option.auto_bin_max = 8;              // coverpoint level
    option.weight       = 2;
  }
  x: cross cp_a, cp_b {
    option.weight = 0;                    // cross level
  }
endgroup

cg c = new();
c.option.comment     = "instance for port 0";     // may also be set after construction
c.cp_a.option.weight = 3;
cg::type_option.goal = 95;                         // type options: through the type name
```

Instance options (`option.`)

Option	Default	Level	Meaning
`name`	Tool-generated	Covergroup	Name of the instance in the report
`comment`	`""`	Covergroup, coverpoint, cross	Free text stored in the coverage database
`per_instance`	0	Covergroup	If 1, coverage of each instance is also stored and reported separately, not only the type coverage
`at_least`	1	Covergroup, coverpoint, cross	Minimum number of hits for a bin to count as covered
`weight`	1	Covergroup, coverpoint, cross	Weight of the item when computing the coverage of the enclosing level. Must be ≥ 0; a weight of 0 removes the item from the calculation
`goal`	100	Covergroup, coverpoint, cross	Target percentage. Used by reports to mark the item as "goal met"; it does not change the computed value
`auto_bin_max`	64	Covergroup, coverpoint	Maximum number of automatic bins of a coverpoint
`cross_num_print_missing`	0	Covergroup, cross	Number of uncovered cross bins to list in the report
`detect_overlap`	0	Covergroup, coverpoint	If 1, a warning is given when two bins of a coverpoint overlap
`get_inst_coverage`	0	Covergroup	Relevant only with `type_option.merge_instances`: enables per-instance queries when instances are merged

Option	Default	Level	Meaning
`cross_auto_bin_max`	unlimited	Covergroup, cross	Maximum number of automatic cross bins. Defined in IEEE 1800-2005 and 1800-2009, **removed in 1800-2012**. Some tools still accept it; do not rely on it

Type options (`type_option.`)

Option	Default	Level	Meaning
`weight`	1	Covergroup, coverpoint, cross	Weight used in **type** coverage and in the overall `$get_coverage()`
`goal`	100	Covergroup, coverpoint, cross	Target for the type
`comment`	`""`	Covergroup, coverpoint, cross	Comment for the type
`strobe`	0	Covergroup	If 1, sampling takes place at the end of the time step (Postponed region), after all values have settled
`merge_instances`	0	Covergroup	If 1, type coverage is computed from the **union** of the bins hit by all instances. If 0, it is the weighted average of the instance coverages. Tools differ in their default behaviour here

option versus type_option

Property	`option`	`type_option`
Applies to	One instance	The covergroup type (all instances together)
Set where	Inside the covergroup, or through an instance handle: `c.option.x`	Inside the covergroup, or through the type name: `cg::type_option.x`
Can differ between instances	Yes	No
Options available	`name`, `comment`, `per_instance`, `at_least`, `weight`, `goal`, `auto_bin_max`, `cross_num_print_missing`, `detect_overlap`	`weight`, `goal`, `comment`, `strobe`, `merge_instances`
`weight` affects	Instance coverage (`get_inst_coverage()`)	Type coverage (`get_coverage()`) and `$get_coverage()`

Complete example 1: at_least and weight

tb_cg_options.sv

```
module tb_cg_options;

  bit [1:0] sel;
  bit       flag;

  covergroup cg_opt;
    option.per_instance = 1;
    option.comment      = "demonstrates at_least and weight";

    cp_sel: coverpoint sel {
      option.at_least = 2;        // a bin needs two hits
      option.weight   = 3;        // three times as important as cp_flag
    }
    cp_flag: coverpoint flag;     // at_least = 1, weight = 1
  endgroup

  cg_opt cg = new();

  initial begin
    sel = 0;  flag = 0;  cg.sample();
    sel = 0;  flag = 0;  cg.sample();
    sel = 1;  flag = 0;  cg.sample();
    sel = 2;  flag = 1;  cg.sample();
    sel = 2;  flag = 0;  cg.sample();

    $display("cp_sel   = %0.2f%%", cg.cp_sel.get_inst_coverage());
    $display("cp_flag  = %0.2f%%", cg.cp_flag.get_inst_coverage());
    $display("instance = %0.2f%%", cg.get_inst_coverage());
  end

endmodule
```

EXPECTED OUTPUT

```
cp_sel   = 50.00%
cp_flag  = 100.00%
instance = 62.50%
```

Explanation

Item	Calculation
Hits of `cp_sel`	Value 0: two hits. Value 1: one hit. Value 2: two hits. Value 3: none.
`option.at_least = 2`	Only bins with two or more hits count. Bins 0 and 2 are covered; bin 1, with a single hit, is **not**. 2 of 4: 50 percent. With the default `at_least = 1` it would be 75 percent.
`cp_flag`	Both values seen: 100 percent.
Instance coverage	Weighted average: (3 × 50 + 1 × 100) / (3 + 1) = 62.5 percent. Without the weights it would be 75 percent.
`get_inst_coverage()`	The example queries **instance** coverage, because `option.weight` is an instance option. The **type** coverage returned by `get_coverage()` uses `type_option.weight`, which is still 1 here.

Complete example 2: per_instance

tb_cg_per_instance.sv

```
module tb_cg_per_instance;

  covergroup cg_port with function sample(bit [1:0] v);
    option.per_instance = 1;
    cp_v: coverpoint v;
  endgroup

  cg_port cg_port_a = new();
  cg_port cg_port_b = new();

  initial begin
    cg_port_a.set_inst_name("port_a");
    cg_port_b.set_inst_name("port_b");

    cg_port_a.sample(2'd0);
    cg_port_a.sample(2'd1);
    cg_port_b.sample(2'd1);
    cg_port_b.sample(2'd2);

    $display("port_a instance coverage = %0.2f%%", cg_port_a.get_inst_coverage());
    $display("port_b instance coverage = %0.2f%%", cg_port_b.get_inst_coverage());
  end

endmodule
```

EXPECTED OUTPUT

```
port_a instance coverage = 50.00%
port_b instance coverage = 50.00%
```

Explanation

Two instances of one covergroup type observe two ports. Each instance saw two of the four values, so each has 50 percent instance coverage. With `per_instance = 1` the report lists both instances, and you can see that `port_a` never carried the values 2 and 3 and `port_b` never carried 0 and 3.

The **type** coverage (`cg_port::get_coverage()`) combines the instances. Depending on `type_option.merge_instances` and on the tool's default, it is either the union of the hits (values 0, 1 and 2: 75 percent) or the average of the instance coverages (50 percent). Check your simulator's documentation before quoting a type-coverage number for a multi-instance covergroup.

> **Interview trap:** "A covergroup is instantiated in four agents. The report shows 100 percent. Was every agent fully covered?" Not necessarily. Without `option.per_instance = 1` only the combined type coverage may be reported, and one busy agent can hide three idle ones. Set `per_instance` when each instance must be covered on its own.

> **Common mistake:** Setting `option.weight = 0` on a coverpoint to "disable" it and then being surprised that its cross still counts. Weight 0 removes that item from the average; the coverpoint is still sampled and its bins still feed the cross. This is in fact the standard trick for coverpoints that exist only to build a cross.

> **Common mistake:** Raising `auto_bin_max` to a large value to get "one bin per value" on a wide variable. Write `bins b[] = {[lo:hi]};` for the values that matter instead.

Interview questions

Q1. What is the difference between `option` and `type_option`? `option` holds instance-specific settings that can differ for each covergroup instance. `type_option` holds settings for the covergroup type as a whole, shared by all instances.

Q2. What does `option.at_least` do? It sets the minimum number of hits a bin needs before it is counted as covered. The default is 1.

Q3. What does `option.per_instance` do? When set to 1, coverage is tracked and reported for each instance of the covergroup separately, in addition to the type coverage. It is needed when the same covergroup is instantiated for several ports, agents or configurations.

Q4. What is `option.weight` used for? It sets the relative importance of a coverpoint or cross in the covergroup's coverage, or of a covergroup in the overall coverage. A weight of 0 excludes the item from the calculation.

Q5. What does `option.goal` do? It sets the target coverage percentage for the item (default 100). It is used for reporting; it does not alter the calculated coverage.

Q6. What does `option.auto_bin_max` control? The maximum number of automatically created bins for a coverpoint (default 64).

Q7. How do you avoid a race when a covergroup samples on a clock event? Set `type_option.strobe = 1` so that sampling happens at the end of the time step, or sample through a clocking block, or call `sample()` procedurally at a safe point.

3.3 How the Coverage Percentage Is Computed

The three levels

```
Level 1: one coverpoint or one cross

                      number of bins with hits >= at_least
     C_item  =  ------------------------------------------------  x 100
                number of bins (ignore, illegal, default excluded)

Level 2: one covergroup = weighted average of its coverpoints and crosses

                sum of ( weight_i  x  C_item_i )
     C_group =  ---------------------------------
                       sum of weight_i

Level 3: the whole testbench = weighted average of all covergroup types
     $get_coverage()
```

Each coverpoint counts as **one item** and each cross counts as **one item**, whatever its number of bins. A coverpoint with 2 bins and a cross with 64 bins have the same influence on the covergroup's number unless the weights are changed.

Worked example

A covergroup has three items after a test:

Item	Bins	Bins hit	Item coverage	Weight
`cp_op`	8	6	75.0%	1

Item	Bins	Bins hit	Item coverage	Weight
`cp_a`	4	3	75.0%	1
`x_op_a`	32	12	37.5%	1

Covergroup coverage = (75.0 + 75.0 + 37.5) / 3 = **62.5 percent**.

Notice that it is **not** (6 + 3 + 12) / (8 + 4 + 32) = 47.7 percent. Bins are not pooled across items.

If `cp_op` and `cp_a` exist only to feed the cross and are given `option.weight = 0`, the covergroup coverage is that of the cross alone: 37.5 percent.

What is in the denominator

Bin kind	Counted in the total
Automatic bins	Yes
Explicit `bins`, each element of `bins b[]` or `bins b[N]`	Yes
Wildcard bins, transition bins	Yes
`bins x = default;` and `default sequence`	No
`ignore_bins` and values removed by them	No
`illegal_bins` and values removed by them	No
A coverpoint whose `iff` guard was never true	Yes: its bins exist and are simply not hit

Query methods

Call	Returns
`cg_inst.get_inst_coverage()`	Coverage of this covergroup **instance**
`cg_inst.get_coverage()` or `cg_type::get_coverage()`	Coverage of the covergroup **type**, all instances combined
`cg_inst.cp_label.get_inst_coverage()`	Coverage of one coverpoint or cross of this instance
`cg_inst.cp_label.get_coverage()`	Coverage of one coverpoint or cross for the type
`$get_coverage()`	Overall coverage of **all** covergroup types in the simulation

All return a `real` from 0 to 100. `get_coverage()` and `get_inst_coverage()` optionally take two `ref int` arguments that receive the number of covered bins and the total number of bins:

```
int covered, total;
real pct;
pct = cg.cp_op.get_inst_coverage(covered, total);
$display("cp_op: %0d of %0d bins, %0.1f%%", covered, total, pct);
```

get_coverage versus get_inst_coverage

Property	`get_coverage()`	`get_inst_coverage()`
Scope	The covergroup type, over all instances	The one instance it is called on

Property	`get_coverage()`	`get_inst_coverage()`
Can be called without an instance	Yes: `cg_type::get_coverage()`	No
Weights used	`type_option.weight`	`option.weight`
With a single instance and default options	Same value as `get_inst_coverage()`	Same value as `get_coverage()`
With several instances	Combined according to `type_option.merge_instances`	Needs `option.per_instance = 1` to be meaningful

Using the number inside a test

```
// stop when the coverage goal is reached, or after a maximum number of transactions
while (cov.alu_cg.get_inst_coverage() < 100.0 && count < max_txns) begin
  generate_and_send_one_txn();
  count++;
end
```

This is a simple form of coverage-driven test termination. In practice the coverage of a single simulation is rarely the decision criterion; the merged coverage of the whole regression is.

Interview questions

Q1. How is the coverage of a coverpoint calculated? The number of its bins that have reached `at_least` hits, divided by the total number of its bins (excluding default, ignore and illegal bins), times 100.

Q2. How is the coverage of a covergroup calculated? As the weighted average of the coverage of its coverpoints and crosses. Each coverpoint and each cross is one item, with weight 1 by default.

Q3. A covergroup has a coverpoint with 2 bins (both hit) and a coverpoint with 100 bins (10 hit). What is the covergroup coverage? (100 + 10) / 2 = 55 percent. It is not 12 / 102, because items are averaged, not bins.

Q4. What is the difference between `get_coverage()` and `get_inst_coverage()`? `get_coverage()` returns the coverage of the covergroup type across all instances and can be called on the type. `get_inst_coverage()` returns the coverage of the particular instance.

Q5. What does `$get_coverage()` return? The overall functional coverage of all covergroup types in the simulation, as a real number between 0 and 100.

Q6. Do ignore bins increase or decrease the coverage percentage? For the same set of hits they increase it (or leave it unchanged), because the ignored values are removed from the total number of bins.

3.4 Coverage-Driven Verification

In plain English

In directed testing the plan is a list of tests: "write test 1, test 2, ... test 200; when all pass, we are done". In **coverage-driven verification (CDV)** the plan is a list of **things that must be observed**, written as coverage. Tests are generated randomly, and the coverage report, not the test list, says how far along you are and what to do next.

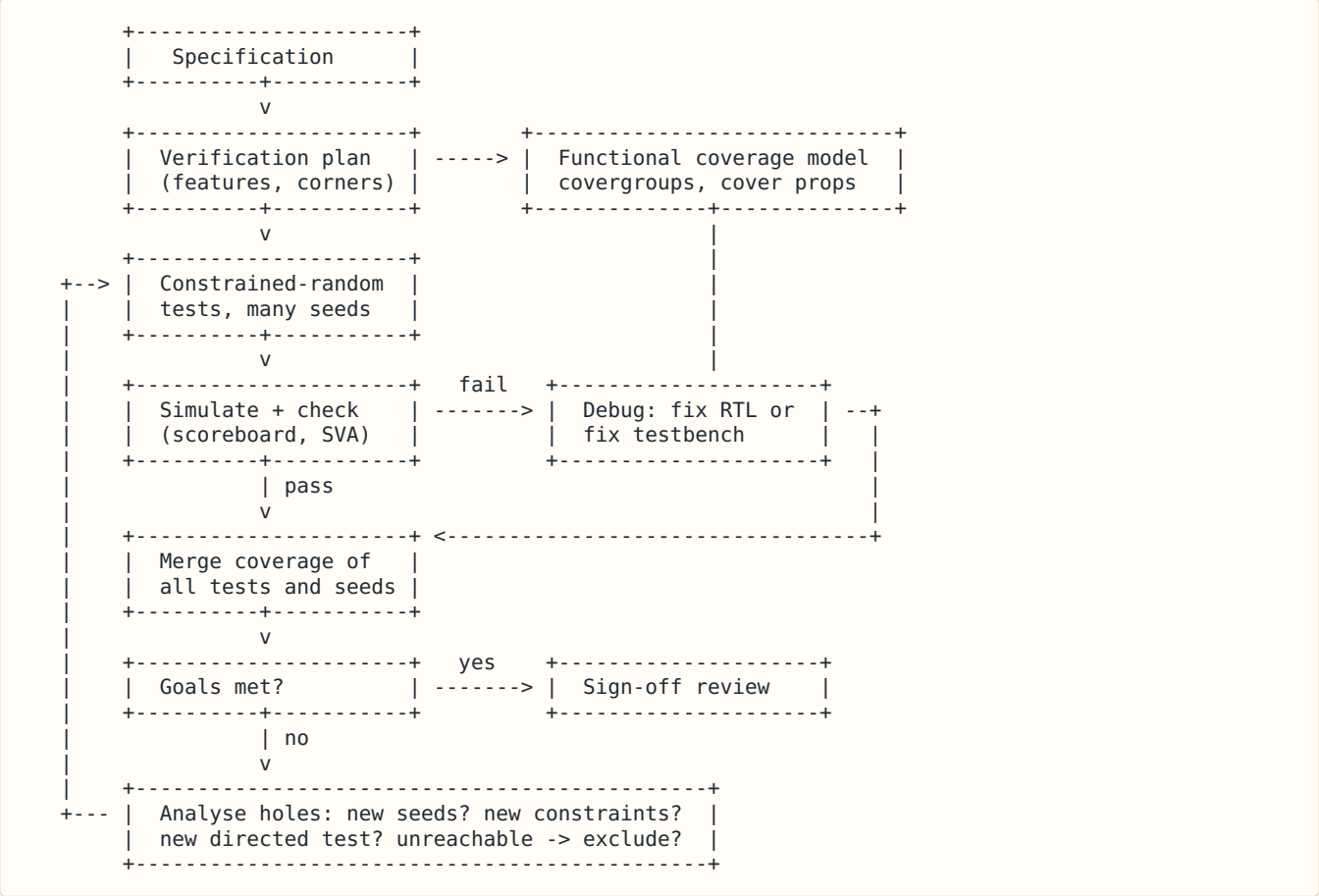
Technical definition

Coverage-driven verification is a methodology in which

1. the features and scenarios of the verification plan are encoded as a functional coverage model,
2. stimulus is generated by constrained randomization,
3. checking is automatic (scoreboard and assertions),
4. coverage results are analysed to find **holes**, and
5. the stimulus (constraints, seeds, tests) is adjusted to fill the holes,

repeated until the coverage goals are met. The process of getting from the first results to the goals is called **coverage closure**.

The CDV loop



What to do with a coverage hole

Cause of the hole	Action
Not enough random runs	Run more seeds
The scenario is legal but improbable (for example FIFO full with simultaneous read and write)	Add a test with tighter constraints or weighted distributions (`dist`) that steer the stimulus towards it
A constraint accidentally forbids it	Fix or relax the constraint
Needs a precise sequence that random stimulus will not produce	Write a directed test or a directed sequence

Cause of the hole	Action
The DUT cannot reach it (unreachable by design)	Confirm with the designer, then exclude it with `ignore_bins` or a documented waiver
The coverage model is wrong (wrong bin range, wrong sampling point)	Fix the coverage model
The feature is not implemented	It is a design bug or a specification question: report it

The typical closure curve



Sign-off criteria in practice

Criterion	Typical target
All tests in the regression pass	100%
Functional coverage	100% of the plan, with every exclusion reviewed
Code coverage (statement, branch, condition, toggle, FSM)	Close to 100%, with waivers for unreachable code
Assertions	None failing; every assertion has been triggered non-vacuously (assertion coverage)
Bug rate	New bugs per week has fallen to near zero

Directed versus coverage-driven verification

Aspect	Directed testing	Coverage-driven verification
Stimulus	Written by hand for each scenario	Constrained-random
Progress metric	Number of tests written and passing	Coverage percentage
Finds unexpected bugs	Rarely: only what the engineer thought of	Often: random combinations nobody listed
Effort over time	Grows linearly with the number of scenarios	High at the start (environment), low per additional scenario
Proof that a scenario occurred	By construction of the test	By the coverage report
Used for	Bring-up, specific corner cases, last holes	The bulk of verification

Key point: Random stimulus without functional coverage is blind: you do not know what the random tests did. Functional coverage without random stimulus is slow: you are back to writing one test per bin. CDV is the combination.

Interview trap: "Coverage is 100 percent. Are we done?" Only if (1) all tests pass with strong checkers, (2) the coverage model has been reviewed against the plan, (3) the exclusions are justified, (4) code coverage has been analysed, and (5) the assertions were actually exercised. 100 percent of a weak model means little.

Common mistake: Writing the coverage model from the RTL instead of from the specification. Coverage copied from the implementation repeats the designer's assumptions and cannot reveal a missing feature.

Interview questions

Q1. What is coverage-driven verification? A methodology in which the verification plan is implemented as functional coverage, stimulus is constrained-random, checking is automatic, and coverage results are used to measure progress and to direct further stimulus until the coverage goals are met.

Q2. What is coverage closure? The process of analysing uncovered items (holes) and eliminating them, by running more seeds, adjusting constraints, adding directed tests, fixing the coverage model, or excluding unreachable items with justification, until the coverage goals are reached.

Q3. What is a coverage hole and how do you handle it? A bin, cross or code item that has not been hit. First determine why: insufficient seeds, over-constrained stimulus, improbable scenario, unreachable code, or an error in the coverage model. Then apply the matching action from the table above.

Q4. How is coverage from many tests combined? Each simulation writes a coverage database. The databases of all tests and seeds in the regression are merged by the tool, and the merged result is analysed.

Q5. When do you exclude a coverage item? Only when it is genuinely unreachable or not applicable (reserved values, configurations not supported), after confirmation with the designer or architect, and with the reason recorded for the sign-off review.

Q6. When is verification complete? When the regression passes, functional and code coverage goals are met with reviewed exclusions, no assertion fails and all assertions have been exercised, and the rate of new bugs has dropped to an agreed level.

3.5 The Complete alu_coverage Class

This section puts Parts 1 to 3 together in the coverage collector of the ALU testbench. The class follows the naming used in the whole course: it receives observed `alu_txn` objects, and its covergroup is `alu_cg`.

From the verification plan to the coverage model

Plan item	Coverage item
Every operation is executed	`cp_op`: automatic bins on the enum, 8 bins

Plan item	Coverage item
Operand `a` takes its boundary values and both halves of its range	`cp_a` : `zero` , `low` , `high` , `max`
The same for operand `b`	`cp_b`
Carry / borrow / shifted-out bit is seen as 0 and as 1	`cp_carry` , guarded by the operations that can produce it
A result of zero is seen	`cp_zero`
Back-to-back operations	`cp_op_seq` : transition bins
Every operation with every class of `a`	`x_op_a` : 8 × 4 = 32 cross bins
Every two-operand operation with every class of `b`	`x_op_b` : the three one-operand operations are ignored, 5 × 4 = 20 cross bins
Carry observed for each operation that can produce one	`x_op_carry` : 3 × 2 = 6 cross bins

The class

alu_coverage.sv

```
// Functional coverage collector for the ALU testbench.
// This file is included inside alu_tb_pkg (or, in the demo below, inside a module),
// after the declaration of alu_txn.
class alu_coverage;

  alu_txn txn;                      // the transaction currently being sampled

  covergroup alu_cg;
    option.per_instance = 1;
    option.comment      = "ALU functional coverage";

    // ---- every operation ----
    cp_op: coverpoint txn.op;

    // ---- operand classes ----
    cp_a: coverpoint txn.a {
      bins zero = {8'h00};
      bins low  = {[8'h01:8'h7F]};
      bins high = {[8'h80:8'hFE]};
      bins max  = {8'hFF};
    }
    cp_b: coverpoint txn.b {
      bins zero = {8'h00};
      bins low  = {[8'h01:8'h7F]};
      bins high = {[8'h80:8'hFE]};
      bins max  = {8'hFF};
    }

    // ---- result properties ----
    cp_carry: coverpoint txn.result[8]
              iff (txn.op inside {ALU_ADD, ALU_SUB, ALU_SHL}) {
      bins no_carry = {1'b0};
      bins carry    = {1'b1};
    }
    cp_zero: coverpoint (txn.result[7:0] == 8'h00) {
      bins non_zero = {1'b0};
      bins zero     = {1'b1};
    }

    // ---- sequences of operations ----
    cp_op_seq: coverpoint txn.op {
      bins same_op[] = (ALU_ADD => ALU_ADD), (ALU_SUB => ALU_SUB);
      bins add_sub   = (ALU_ADD => ALU_SUB);
      bins sub_add   = (ALU_SUB => ALU_ADD);
    }
```

```
    // ---- combinations ----
    x_op_a: cross cp_op, cp_a;

    x_op_b: cross cp_op, cp_b {
      ignore_bins one_operand = binsof(cp_op) intersect {ALU_NOT, ALU_SHL, ALU_SHR};
    }

    x_op_carry: cross cp_op, cp_carry {
      ignore_bins no_carry_ops =
        binsof(cp_op) intersect {ALU_AND, ALU_OR, ALU_XOR, ALU_NOT, ALU_SHR};
    }
  endgroup

  function new();
    alu_cg = new();
  endfunction

  // called by the scoreboard (or a monitor subscriber) for every observed transaction
  function void sample(alu_txn t);
    txn = t;
    alu_cg.sample();
  endfunction

  function void report();
    $display("ALU functional coverage = %0.2f%%", alu_cg.get_inst_coverage());
  endfunction

endclass
```

Line by line

Code	Explanation
`alu_txn txn;`	A handle that the covergroup reads. `sample()` sets it before triggering the covergroup.
`option.per_instance = 1;`	The environment might be instantiated more than once; each collector is reported separately.
`cp_op: coverpoint txn.op;`	Enum coverpoint: eight automatic bins named after the opcodes.
`cp_a`, `cp_b`	Four explicit bins each. The boundaries 0 and 255 have their own bins, and the two ranges split at the MSB, where signed-interpretation bugs appear.
`cp_carry ... iff (...)`	Bit 8 of the result is meaningful only for add, subtract and shift-left. The guard prevents the constant 0 of the logical operations from filling the `no_carry` bin.
`cp_zero: coverpoint (txn.result[7:0] == 8'h00)`	A coverpoint on an expression. Explicit bins give the two outcomes readable names.
`cp_op_seq`	A second coverpoint on `op` with transition bins. `same_op[]` creates two bins (ADD after ADD, SUB after SUB). Four transition bins in total.
`x_op_a`	Every operation with every class of `a`: 32 automatic cross bins.
`x_op_b`	`ALU_NOT`, `ALU_SHL` and `ALU_SHR` do not use `b`. Their 12 cross products are removed; 20 remain.
`x_op_carry`	Only the three operations that can set bit 8 are kept: 6 cross bins.
`alu_cg = new();`	Mandatory construction in the class constructor.
`function void sample(alu_txn t);`	Stores the handle and samples. The transaction must not be modified by another component between this call and the covergroup's evaluation; because `sample()` is a function and executes in zero time, that is guaranteed.

A small test of the collector

The demonstration below uses a cut-down `alu_txn` (only the data fields; the full class with `print`, `copy` and `compare` is in Module 6) so that the file can be run on its own. `alu_pkg.sv` is the package from Module 1 and must be compiled first.

alu_pkg.sv

```
package alu_pkg;

  typedef enum logic [2:0] {
    ALU_ADD = 3'b000,
    ALU_SUB = 3'b001,
    ALU_AND = 3'b010,
    ALU_OR  = 3'b011,
    ALU_XOR = 3'b100,
    ALU_NOT = 3'b101,
    ALU_SHL = 3'b110,
    ALU_SHR = 3'b111
  } alu_op_e;

endpackage
```

tb_alu_coverage.sv

```
// compile order: alu_pkg.sv  tb_alu_coverage.sv   (alu_coverage.sv is included below)
module tb_alu_coverage;
  import alu_pkg::*;

  class alu_txn;                    // cut-down version of the Module 6 transaction
    rand bit [7:0] a, b;
    rand alu_op_e  op;
    logic [8:0]    result;
    int unsigned   id;
  endclass

  `include "alu_coverage.sv"

  alu_coverage cov;
  alu_txn      t;
  alu_op_e     op;

  initial begin
    cov = new();

    // one transaction per opcode, always with a = 0 and b = 255
    op = op.first();
    for (int i = 0; i < op.num(); i++) begin
      t        = new();
      t.id     = i;
      t.op     = op;
      t.a      = 8'h00;
      t.b      = 8'hFF;
      t.result = 9'd0;              // not relevant for the items printed below
      cov.sample(t);
      op = op.next();
    end

    $display("cp_op     = %0.2f%%", cov.alu_cg.cp_op.get_inst_coverage());
    $display("cp_a      = %0.2f%%", cov.alu_cg.cp_a.get_inst_coverage());
    $display("x_op_a    = %0.2f%%", cov.alu_cg.x_op_a.get_inst_coverage());
    $display("x_op_b    = %0.2f%%", cov.alu_cg.x_op_b.get_inst_coverage());
    $display("cp_op_seq = %0.2f%%", cov.alu_cg.cp_op_seq.get_inst_coverage());
  end

endmodule
```

EXPECTED OUTPUT

```
cp_op     = 100.00%
cp_a      = 25.00%
x_op_a    = 25.00%
x_op_b    = 25.00%
cp_op_seq = 25.00%
```

What the numbers say

Item	Calculation	What the hole means
`cp_op`	All 8 opcodes sampled: 8 / 8	Nothing missing
`cp_a`	Only `zero` hit: 1 / 4	`a` was never non-zero
`x_op_a`	8 opcodes × only the `zero` class: 8 / 32	Every operation was tried, but only with `a` = 0
`x_op_b`	5 two-operand opcodes × only the `max` class of `b` : 5 / 20	The same for `b`
`cp_op_seq`	The order was ADD, SUB, AND, ...: only `add_sub` occurred: 1 / 4	No operation was repeated back to back

A test that executes every opcode gives 100 percent on `cp_op` and would also give 100 percent statement coverage on the `case` statement of the ALU. The functional coverage shows how little was really tested. This is the argument for functional coverage in one example.

Connecting the collector in the layered testbench

```
// inside alu_scoreboard (Module 9)
alu_coverage cov;

task run();
  alu_txn txn;
  forever begin
    mon2scb_mbx.get(txn);          // transaction observed by the monitor
    check(txn);                    // compare with the reference model
    cov.sample(txn);               // then record what was exercised
  end
endtask
```

At the end of the test, `alu_test` calls `env.scb.cov.report()`.

> **Verification application:** Coverage is sampled **after** checking and from the **monitor's** transaction. If the DUT produced a wrong result, the test fails anyway; if it produced the right one, the scenario has been both exercised and checked. That pairing is what makes the coverage number trustworthy.

> **Common mistake:** Sampling a transaction handle that the generator keeps re-randomizing. If the collector stores `txn = t` and samples later, the object may already contain the next values. Sample immediately (as above), or sample a copy.

Interview questions

Q1. Where is the coverage collector placed in a layered testbench? On the monitor side. The monitor (through the scoreboard or a dedicated subscriber) passes each observed transaction to a coverage class that owns the covergroup and calls `sample()`.

Q2. Why did `cp_carry` need an `iff` guard? Bit 8 of the result is a carry only for add, subtract and shift-left. For the other operations it is always 0; without the guard, those operations would fill the `no_carry` bin and hide the fact that an addition without carry was never tested.

Q3. Why does the cross of `op` and `b` ignore some operations? `ALU_NOT`, `ALU_SHL` and `ALU_SHR` do not use operand `b`, so combinations of those operations with classes of `b` say nothing about the design. Leaving them in would add twelve bins that are meaningless to fill.

Q4. A test runs all 8 opcodes once. `cp_op` is 100 percent. Is the ALU verified? No. The crosses with the operand classes, the carry coverage and the sequence coverage are almost empty. One execution per opcode exercises a tiny part of the behaviour.

Q5. How would you write coverage for "all operations with both operands at maximum"? The simplest form is a dedicated coverpoint on the opcode with a guard: `cp_op_max: coverpoint txn.op iff (txn.a == 8'hFF && txn.b == 8'hFF);`, which has one bin per operation. The alternative is a three-way cross `cross cp_op, cp_a, cp_b` in which every product that does not involve `cp_a.max` and `cp_b.max` is removed: `ignore_bins not_max = !binsof(cp_a.max) || !binsof(cp_b.max);`.

Part 4: Assertion Fundamentals

4.1 What Is an Assertion?

In plain English

An assertion is a statement in the code that says **"this must always be true"**, and that complains immediately when it is not.

A specification is full of such rules: "a grant is never given without a request", "the FIFO is never full and empty at the same time", "`valid` stays high until `ready` is seen". A scoreboard does not check these directly; it looks at finished transactions. An assertion watches the signals themselves, at every clock, for the whole simulation, and reports the exact time and place where a rule was broken.

Technical definition

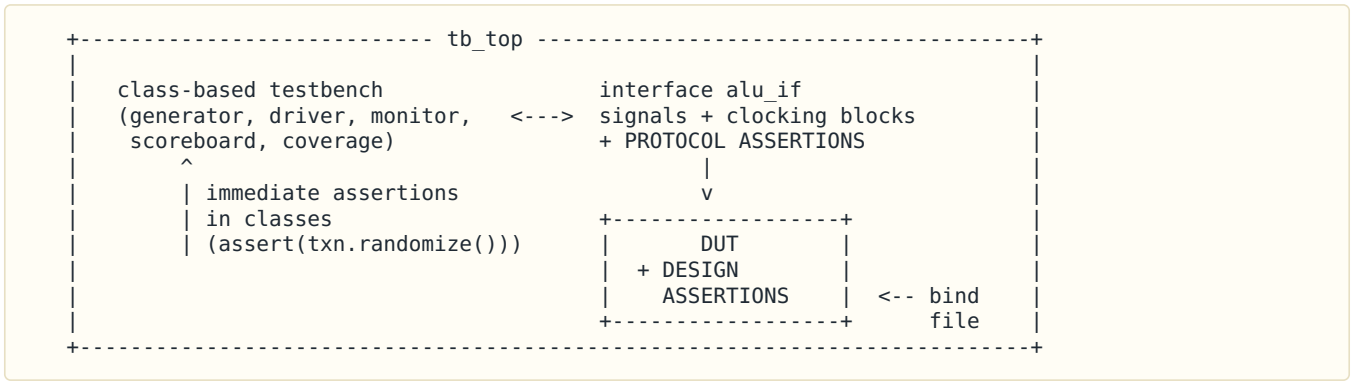
An **assertion** is a check embedded in design or verification code that states an intended property of the design. The simulator (or a formal tool) evaluates it and reports a failure when the property does not hold. SystemVerilog Assertions (SVA) come in two kinds:

- **Immediate assertions**: procedural statements that check a condition at the moment they are executed, like an `if`.
- **Concurrent assertions**: statements that check behaviour **over time**, evaluated on a clock, using sequences and properties.

Why assertions are needed

Without assertions	With assertions
A protocol violation is noticed only if it corrupts data that the scoreboard later compares, possibly thousands of cycles after the cause	The violation is reported at the clock edge where it happens, with the name of the rule
Debug starts at the scoreboard mismatch and works backwards through waveforms	Debug starts at the failing assertion, next to the root cause
Internal design assumptions exist only in the designer's head	The assumptions are written in the code and checked in every simulation
Checks are hand-written `always` blocks with flags and counters	One line per rule, in a language made for the purpose
Checkers must be rewritten for formal verification	The same assertions are used by formal tools

Where assertions are in the verification environment



Written by	Typical content	Location
Designer	Internal assumptions: FSM state is one-hot, counter never overflows, FIFO never written when full	Inside the RTL, or in a separate module attached with `bind`
Verification engineer	Interface protocol rules: handshakes, signal stability, timing between request and response, no X on control signals	In the interface, or in a checker module attached with `bind`
Verification engineer	Testbench sanity checks: randomization succeeded, handle not null	Immediate assertions in classes

Benefits and limits

Benefits	Limits
Errors are caught at the source and at the exact time	Poorly suited to checking data transformations (a scoreboard is better)
Compact, readable description of temporal behaviour	Complex properties are hard to write correctly and can pass for the wrong reason (vacuity)
Active in every test, without effort from the test writer	Many assertions slow the simulation down
Reusable from block level to system level and in formal verification	Concurrent assertions cannot be placed inside classes
`cover property` gives temporal functional coverage	

> **Key point:** Assertions, the scoreboard and functional coverage are complementary. Assertions check **rules on signals over time**. The scoreboard checks **the data result of each transaction**. Coverage records **what was exercised**. A serious testbench has all three.

Interview questions

Q1. What is an assertion? A statement that specifies a property the design must satisfy. The tool checks it during simulation (or proves it formally) and reports a failure when the property is violated.

Q2. What are the types of assertions in SystemVerilog? Immediate assertions and concurrent assertions. Immediate assertions are further divided into simple and deferred; concurrent assertions use the directives `assert property`, `assume property`, `cover property` and `restrict property`.

Q3. What are the advantages of assertions? Failures are detected close to the cause, in time and in location; temporal protocol rules are expressed concisely; they document design intent; they work in every test and can be reused in formal verification; `cover property` provides temporal coverage.

Q4. Who writes assertions, the designer or the verification engineer? Both. Designers write assertions about internal assumptions of the RTL. Verification engineers write assertions about interface protocols and end-to-end rules.

4.2 Immediate Assertions

In plain English

An immediate assertion is an `if` statement with a built-in complaint. When the simulator reaches it, the condition is evaluated on the spot, using the current values of the variables. If the condition is true, the assertion passes. If it is false (or X), it fails.

Technical definition

An **immediate assertion** is a procedural statement that tests an expression when the statement is executed. It is not temporal: there is no clock and no notion of "next cycle". It can be used wherever a procedural statement is allowed: in `initial` and `always` blocks, tasks, functions, and **inside class methods**.

Syntax

```
[ label : ] assert ( expression )
              [ pass_statement ]
            [ else fail_statement ]
```

Part	Meaning
`label :`	Optional name, shown in the failure message and in assertion reports. Always give one
`expression`	Evaluated like the condition of an `if` . True (non-zero and known) passes; 0, X or Z fails
Pass statement	Optional. Executed when the assertion passes
`else` fail statement	Optional. Executed when the assertion fails. If it is omitted, the tool calls `$error` with a default message

Severity system tasks

The fail (or pass) statement normally calls one of four system tasks:

Task	Severity	Effect
`$fatal(...)`	Fatal	Reports the message and **terminates** the simulation
`$error(...)`	Error	Reports a run-time error; the simulation continues. This is the default when there is no `else`
`$warning(...)`	Warning	Reports a warning; the simulation continues
`$info(...)`	Information	Reports a message with no severity

Each of them prints the message together with the file name, line number, scope and simulation time. The exact layout is tool-specific. `$fatal` takes an optional first argument (0, 1 or 2) that controls how much diagnostic information is printed: `$fatal(1, "message")`.

Complete example

To keep the output identical on every simulator, the example uses `$display` in the action blocks.

tb_immediate.sv

```
module tb_immediate;

  int         a, b;
  logic [3:0] grant;
  logic       unknown;

  initial begin
    a = 5;
    b = 5;
    a_equal: assert (a == b)
      $display("[%0t] a_equal passed: a=%0d b=%0d", $time, a, b);
    else
      $display("[%0t] a_equal FAILED: a=%0d b=%0d", $time, a, b);

    #10 b = 6;
    a_equal_again: assert (a == b)
      $display("[%0t] a_equal_again passed", $time);
    else
      $display("[%0t] a_equal_again FAILED: a=%0d b=%0d", $time, a, b);

    #10 grant = 4'b0110;
    a_onehot: assert ($onehot(grant))
      else $display("[%0t] a_onehot FAILED: grant=%b", $time, grant);

    grant = 4'b0100;
    a_onehot_again: assert ($onehot(grant))
      $display("[%0t] a_onehot_again passed: grant=%b", $time, grant);

    a_known: assert (unknown)
      else $display("[%0t] a_known FAILED: an X condition counts as false", $time);
  end

endmodule
```

EXPECTED OUTPUT

```
[0] a_equal passed: a=5 b=5
[10] a_equal_again FAILED: a=5 b=6
[20] a_onehot FAILED: grant=0110
[20] a_onehot_again passed: grant=0100
[20] a_known FAILED: an X condition counts as false
```

Line by line

Code	Explanation
`a_equal: assert (a == b)`	Evaluated when execution reaches it, at time 0. 5 equals 5: pass.
The statement after the condition	The pass statement. Note there is **no semicolon** between it and `else` other than the one ending the pass statement itself.
`else $display(...)`	The fail statement. It replaces the default `$error`.
`a_equal_again` at time 10	`b` is now 6: the fail statement runs.

Code	Explanation
`a_onehot`	`$onehot` is true when exactly one bit is 1. `0110` has two: fail. Only an `else` branch is given, so nothing is printed on a pass.
`a_onehot_again`	`0100` has one bit set: pass. Only a pass statement is given; had it failed, the default `$error` would have been issued.
`a_known`	`unknown` was never assigned and is X. An X condition is treated as false, so the assertion fails.

Immediate assertion versus if statement

Property	`if (!cond) $error(...)`	`assert (cond) else $error(...)`
Checks the condition	Yes	Yes
States intent	No: it is ordinary code	Yes: it is recognisably a check
Appears in assertion reports and assertion coverage	No	Yes
Can be switched off globally (`$assertoff` , tool options)	No	Yes
Usable by formal tools	No	Yes (with restrictions)
Default action on failure	None	`$error`

Deferred immediate assertions

A simple immediate assertion in combinational code can fail on a **glitch**: when two inputs of the checked expression change in the same time step, the checking code may run once after the first change and again after the second, and the intermediate, never-really-existing combination can violate the condition.

```
always_comb begin
  a_simple  : assert (!(grant_a && grant_b));          // may report a zero-time glitch
  a_deferred: assert #0 (!(grant_a && grant_b));       // reports only the settled value
  a_final   : assert final (!(grant_a && grant_b));    // reported at the end of the time step
end
```

Form	Syntax	When the failure is reported
Simple immediate	`assert (expr)`	Immediately, each time the statement executes
Observed deferred	`assert #0 (expr)`	The result is held and reported in the Observed region of the time step; if the block re-executes in the same time step, earlier pending reports are discarded
Final deferred (IEEE 1800-2012)	`assert final (expr)`	Reported in the Postponed region, when nothing can change any more

Deferred assertions may only have a subroutine call (such as `$error`) as their action. They are the right choice for combinational checks in `always_comb` blocks and in continuous code. Deferred assertions can also be written outside procedural blocks, directly in a module.

Real-world uses of immediate assertions

```
// 1. Check that randomization succeeded (see the trap below)
if (!txn.randomize()) $fatal(1, "alu_txn randomization failed");

// 2. Check a handle before using it
a_vif_set: assert (vif != null) else $fatal(1, "virtual interface not set");

// 3. Check an argument at the start of a task
a_len_ok: assert (len inside {[1:16]}) else $error("illegal length %0d", len);

// 4. Combinational check in RTL
always_comb a_onehot_state: assert #0 ($onehot(state));
```

Interview trap: `assert (txn.randomize());` is widely written and has a hidden danger. If assertions are disabled (with `$assertoff` or a tool option, often done to speed up a regression), the simulator may not evaluate the expression at all, so **`randomize()` is never called** and the transaction keeps its old values. Call the function unconditionally and check the result: `if (!txn.randomize()) $fatal(...)`, or `ok = txn.randomize(); assert (ok);`.

Common mistake: Putting a semicolon after the condition when an `else` follows: `assert (a == b); else $error(...)`. The semicolon ends the assertion as a null pass statement, and the `else` becomes a syntax error.

Common mistake: Using an immediate assertion in a clocked block to check something "one cycle later". An immediate assertion sees only the present values. Anything involving time needs a concurrent assertion.

Interview questions

Q1. What is an immediate assertion? A procedural statement that evaluates an expression when it is executed and passes if the expression is true. It is not clocked and checks only the current values.

Q2. Where can immediate assertions be used? Anywhere a procedural statement is allowed: `initial` and `always` blocks, tasks, functions and class methods.

Q3. What happens when an immediate assertion fails and there is no `else` clause? The tool issues an error by calling `$error` with a default message; the simulation continues.

Q4. What are the severity levels? `$fatal`, `$error`, `$warning` and `$info`. `$fatal` ends the simulation.

Q5. What is a deferred immediate assertion? `assert #0 (expr)` or `assert final (expr)`. The reporting of the result is postponed to a later region of the same time step, so that transient glitches in combinational logic do not produce false failures.

Q6. Can assertions be used inside a class? Immediate assertions can. Concurrent assertions cannot: they are only allowed in modules, interfaces, programs and checkers.

Q7. What does an immediate assertion do if the expression evaluates to X? It fails. X and Z are treated as false, as in an `if` statement.

4.3 Concurrent Assertions

In plain English

A concurrent assertion describes behaviour that spreads over several clock cycles: "if a request is seen, a grant must follow within three cycles". It is not executed once like a statement. It is switched on for the whole simulation, and at **every clock edge** it starts a new check. Many checks can therefore be in progress at the same time, which is where the name "concurrent" comes from.

Technical definition

A **concurrent assertion** is a statement that specifies a **property** to be evaluated at each occurrence of a **clocking event**. Its expressions use the **sampled values** of the signals: the values the signals had just before the clock edge. A new evaluation **attempt** (also called a thread) starts at every clock tick. Each attempt ends in success, failure, or (if a `disable iff` condition occurs) is discarded.

Syntax

```
[ label : ] assert property ( [ @(clock_event) ] [ disable iff (rst_expr) ] property_expr )
              [ pass_statement ]
            [ else fail_statement ] ;
```

The simplest example:

```
a_no_overlap: assert property (@(posedge clk) !(grant_a && grant_b))
  else $error("both grants active");
```

Where a concurrent assertion can be written

Location	Allowed
Module, interface, program, checker, generate block (outside any procedure)	Yes: the normal case
Inside an `always` or `initial` block	Yes: the clock can be inferred from the block
Inside a `class`, a task or a function	**No**

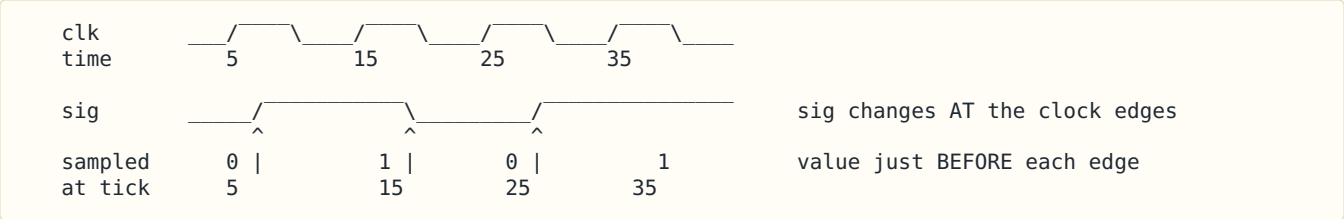
Sampled values: the most important rule

The simulator divides each time step into regions (Module 7 used the same idea for race avoidance):

```
one time step that contains a rising clock edge
+-----------+---------+----------+-------+----------+----------+-----------+
| Preponed  | Active  | Inactive |  NBA  | Observed | Reactive | Postponed |
+-----------+---------+----------+-------+----------+----------+-----------+
     |           |                  |         |          |
     |           |                  |         |          +-- assertion action blocks run
     |           |                  |         |              (pass / fail statements);
     |           |                  |         |              program-block code runs
     |           |                  |         +-- concurrent assertions are EVALUATED,
     |           |                  |             using the sampled values
     |           |                  +-- nonblocking assignments update the flip-flops
     |           +-- the clock edge occurs; blocking assignments and always blocks run
     +-- signal values are SAMPLED here, before anything in this time step has changed
```

So a concurrent assertion uses the values of its signals as they were **immediately before** the clock edge. This is exactly what a flip-flop clocked by the same edge would capture. Two consequences:

- A signal that changes **at** the clock edge (driven by a flip-flop on that same edge) is seen by the assertion with its **old** value at this tick and its new value at the next tick.
- Assertions are free of races with the design and the testbench: the result does not depend on the order in which processes execute.



Complete example: what value does the assertion see?

tb_sva_sampling.sv

```
module tb_sva_sampling;

  logic clk = 0;
  logic sig = 0;

  always #5 clk = ~clk;

  // "sig must be 1 at every rising clock edge"
  a_sig: assert property (@(posedge clk) sig)
    else $display("[%0t] a_sig FAILED: sampled sig=%b, current sig=%b",
                  $time, $sampled(sig), sig);

  initial begin
    @(posedge clk);  sig = 1;      // time 5 : changes at the clock edge
    @(posedge clk);  sig = 0;      // time 15
    @(posedge clk);  sig = 1;      // time 25
    @(posedge clk);                // time 35
    @(negedge clk);  $finish;      // time 40
  end

endmodule
```

EXPECTED OUTPUT

```
[5] a_sig FAILED: sampled sig=0, current sig=1
[25] a_sig FAILED: sampled sig=0, current sig=1
```

Execution flow

Time	Sampled value of `sig` (Preponed)	What the `initial` block does at this edge	Result of the assertion
5	0	Sets `sig = 1`	**Fails**: the value before the edge was 0
15	1	Sets `sig = 0`	Passes: the value before the edge was 1, although `sig` is 0 by the time the result is reported
25	0	Sets `sig = 1`	**Fails**
35	1	Nothing	Passes

Line by line

Code	Explanation
`assert property (@(posedge clk) sig)`	A concurrent assertion with the clock given inside the property. The property is the simplest possible: a Boolean that must be true at every tick.
`$sampled(sig)`	Returns the sampled value, the one the assertion used.
`sig` in the fail statement	The action block runs in the Reactive region, after the `initial` block has already changed `sig`, so it prints the **new** value. This is why failure messages sometimes seem to contradict the waveform; use `$sampled` in action blocks.
Failure at time 5	In the waveform, `sig` rises exactly at time 5 and looks high "at" the edge. The assertion correctly treats it as low, because a flip-flop clocked at time 5 would have captured 0.

Evaluation attempts

```
property: @(posedge clk) req |-> ##2 ack        (if req, then ack two ticks later)

tick        1      2      3      4      5      6
req         1      1      0      0      1      0
ack         0      0      1      1      0      0

attempt 1   start ............. end (ack=1 at tick 3)   -> pass
attempt 2          start ............. end (ack=1 at 4) -> pass
attempt 3                 start/end (req=0)             -> vacuous pass
attempt 4                        start/end (req=0)      -> vacuous pass
attempt 5                               start ......... -> needs tick 7

A new attempt starts at every tick. Attempts 1 and 2 overlap in time: they are concurrent.
```

Immediate versus concurrent assertions

Property	Immediate assertion	Concurrent assertion
Keyword	`assert (expr)`	`assert property (prop)`
Nature	Procedural statement	Declarative statement, always active
Clock	None	Required (explicit, inferred or default)
When evaluated	When execution reaches the statement	At every clock tick
Values used	Current values	Sampled values (from the Preponed region)
Evaluation region	Where the procedure is executing (Active or Reactive)	Observed
Can describe behaviour over several cycles	No	Yes: sequences, delays, repetition, implication
Allowed in classes, tasks, functions	Yes	No
Allowed directly in a module or interface	Deferred forms only	Yes
Sensitive to glitches	Simple form yes; deferred forms no	No
Formal verification	Limited	Yes: the main input of formal tools
Typical use	Testbench sanity checks, combinational checks	Protocol and timing rules

Interview trap: "A signal goes high exactly at the rising edge of the clock. Does a concurrent assertion at that edge see it as high or low?" Low. The assertion uses the value sampled in the Preponed region, before the edge. It sees the high value at the next tick.

Common mistake: Printing signals in an assertion's action block without `$sampled`. The action block runs after the design has updated, so the printed values are those **after** the edge and may not match what the assertion evaluated.

Common mistake: Trying to write `assert property` inside a class (for example in a driver or monitor). It does not compile. Put the assertion in the interface and, if the class needs to control it, expose a signal or use `$assertoff` / `$asserton`.

Interview questions

Q1. What is a concurrent assertion? An assertion that checks a property over time. It is evaluated at every tick of its clocking event using sampled values, and each tick starts a new evaluation attempt.

Q2. What is the difference between immediate and concurrent assertions? An immediate assertion is a procedural statement checked once when executed, with current values and no clock. A concurrent assertion is clock-based, uses sampled values, is evaluated at every clock tick, can span many cycles, and cannot be placed in a class.

Q3. In which regions are concurrent assertions sampled and evaluated? The signal values are sampled in the Preponed region, the property is evaluated in the Observed region, and the pass/fail action blocks execute in the Reactive region.

Q4. Why are sampled values used? To model what clocked hardware sees (the value before the edge) and to make the assertion independent of the execution order of processes in the time step, which removes races.

Q5. Why can a concurrent assertion not be written inside a class? Concurrent assertions are static, declarative constructs tied to the design hierarchy and a clock; they exist for the whole simulation. Class objects are dynamic and are created and destroyed at run time, so the language does not allow concurrent assertions in them.

Q6. What is an assertion attempt or thread? One evaluation of the property that begins at a particular clock tick. Since a new attempt begins at every tick and a property may take several cycles, several attempts can be active at once.

4.4 Sequences and Properties

In plain English

Concurrent assertions are built in layers, like sentences from words:

- A **Boolean expression** is a word: "`req` is high".
- A **sequence** is a phrase describing an order of events over clock ticks: "`req` is high, and one tick later `gnt` is high".

- A **property** is a complete rule, usually of the form "if this, then that": "whenever `req` is high, `gnt` is high within two ticks".
- A **directive** says what to do with the rule: check it (`assert`), assume it (`assume`), or record that it happened (`cover`).

The four layers

```
   Layer 4   DIRECTIVE     assert property / assume property / cover property / restrict
                ^             "what to do with the property"
   Layer 3   PROPERTY      implication |->  |=>,  not,  disable iff,  clock
                ^             "a rule that is true or false"
   Layer 2   SEQUENCE      ##delay, [*n] repetition, and, or, intersect, throughout, within
                ^             "a pattern of Boolean values over consecutive clock ticks"
   Layer 1   BOOLEAN       req && !gnt,   count == 0,   $rose(valid),   $onehot(state)
                             "an expression evaluated at one clock tick with sampled values"
```

Sequences

Definition. A sequence is a list of Boolean expressions in a linear order of increasing time, separated by cycle delays. A sequence **matches** if each expression is true at its tick.

```
sequence s_req_gnt;
  req ##1 gnt;                  // req true at this tick, gnt true at the next tick
endsequence

sequence s_handshake (logic r, logic g, int max_wait);   // a sequence with arguments
  r ##[1:max_wait] g;
endsequence
```

Element	Meaning
`a ##1 b`	`a` at tick T, `b` at tick T+1
`a ##0 b`	`a` and `b` at the same tick
`a ##3 b`	`b` three ticks after `a`
`a ##[1:3] b`	`b` one, two or three ticks after `a`
`a [*3]`	`a` true on three consecutive ticks

Part 5 covers delays and repetition in full.

A sequence by itself is not a rule. It is a pattern that can be found (matched) starting at some tick, or not.

Properties

Definition. A property specifies behaviour that must hold. It is built from sequences and Boolean expressions with property operators, most importantly **implication**, and it evaluates to true or false for each attempt.

```
property p_req_gnt;
  @(posedge clk) disable iff (!rst_n)
    req |-> ##[1:2] gnt;       // if req, then gnt within 1 to 2 ticks
endproperty
```

Part	Meaning
`@(posedge clk)`	The clocking event

Part	Meaning
`disable iff (!rst_n)`	The check is abandoned while reset is active
`req`	The **antecedent**: the condition that triggers the check
`|->`	Implication: "if the antecedent matches, the consequent must match"
`##[1:2] gnt`	The **consequent**: what must then happen

Properties, like sequences, can take arguments, which makes them reusable:

```
property p_stable_until_ack (logic vld, logic ack, logic [7:0] dat);
  @(posedge clk) disable iff (!rst_n)
    vld && !ack |=> $stable(dat);
endproperty

a_wdata_stable: assert property (p_stable_until_ack(wvalid, wready, wdata));
a_rdata_stable: assert property (p_stable_until_ack(rvalid, rready, rdata));
```

Directives

Directive	Meaning in simulation	Meaning in formal verification
`assert property (p)`	Check that `p` holds; report a failure if not	A proof obligation: prove that `p` always holds
`assume property (p)`	Checked exactly like `assert`	A constraint on the inputs: the tool only considers input behaviour satisfying `p`
`cover property (p)`	Record that `p` (or a sequence) was observed; counted in coverage	Find a trace that reaches it: proves the scenario is reachable
`restrict property (p)`	Ignored	A constraint, like `assume`, that is never checked

`cover property` is the temporal form of functional coverage. It answers "did this sequence of events ever occur?".

Sequence versus property

Aspect	Sequence	Property
Describes	A pattern of events over ticks	A rule about behaviour
Result	Matches or does not match	Holds (true) or fails (false)
Keywords	`sequence` ... `endsequence`	`property` ... `endproperty`
Operators	`##`, `[*]`, `[->]`, `[=]`, `and`, `or`, `intersect`, `throughout`, `within`, `first_match`	`|->`, `|=>`, `not`, `and`, `or`, `if` ... `else`, `disable iff`
Can contain implication	No	Yes
Can be asserted directly	Yes, but then it must match at **every** tick, which is rarely intended	Yes
Can be covered directly	Yes: `cover property (seq)`	Yes
Used inside	Other sequences and properties	Other properties and directives

Complete example

tb_sva_seq_prop.sv

```
module tb_sva_seq_prop;

  logic clk = 0;
  logic req, gnt;

  // stimulus for cycles 0..7 (cycle k is the posedge at time 10k+5)
  bit [0:7] req_v = 8'b0100_1000;
  bit [0:7] gnt_v = 8'b0010_0001;

  always #5 clk = ~clk;

  // Layer 2: sequences
  sequence s_gnt_within_2;
    ##[1:2] gnt;
  endsequence

  sequence s_req_then_gnt;
    req ##1 gnt;
  endsequence

  // Layer 3: property
  property p_req_gnt;
    @(posedge clk) req |-> s_gnt_within_2;
  endproperty

  // Layer 4: directives
  a_req_gnt: assert property (p_req_gnt)
    else $display("[%0t] a_req_gnt FAILED: no gnt within 2 cycles of req", $time);

  c_req_gnt: cover property (@(posedge clk) s_req_then_gnt)
    $display("[%0t] c_req_gnt covered: gnt came one cycle after req", $time);

  initial begin
    for (int i = 0; i < 8; i++) begin
      req = req_v[i];
      gnt = gnt_v[i];
      @(negedge clk);
    end
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[25] c_req_gnt covered: gnt came one cycle after req
[65] a_req_gnt FAILED: no gnt within 2 cycles of req
```

Stimulus and evaluation

Cycle	Time	`req`	`gnt`	`a_req_gnt`	`c_req_gnt`
0	5	0	0	No request: nothing to check	
1	15	1	0	Attempt A starts: `gnt` required at cycle 2 or 3	Possible match starts
2	25	0	1	Attempt A **passes**	**Covered** (`req` at 1, `gnt` at 2)
3	35	0	0		
4	45	1	0	Attempt B starts: `gnt` required at cycle 5 or 6	Possible match starts
5	55	0	0	Still waiting	No `gnt` : no match
6	65	0	0	Attempt B **fails**: window exhausted	

Cycle	Time	`req`	`gnt`	`a_req_gnt`	`c_req_gnt`
7	75	0	1	Too late	

Line by line

Code	Explanation
`bit [0:7] req_v = 8'b0100_1000;`	The value of `req` for each cycle, written left to right: cycles 1 and 4 are 1.
`sequence s_gnt_within_2; ##[1:2] gnt; endsequence`	A sequence that starts with a delay: `gnt` true one or two ticks after the starting point.
`property p_req_gnt; @(posedge clk) req |-> s_gnt_within_2; endproperty`	Uses the named sequence as the consequent.
`a_req_gnt: assert property (p_req_gnt) else ...`	The directive. Only a fail statement is given.
`c_req_gnt: cover property (...) $display(...)`	A cover directive may have one statement, executed each time the property is covered.
The `for` loop	Applies the stimulus for cycle `i`, then waits for the falling edge, so signals change at times 0, 10, 20 ...

Key point: The failure is reported at time 65, the last tick at which `gnt` could still have arrived, not at time 45 when the request was made. A failing attempt ends when it becomes certain that the property cannot hold. When debugging, look backwards from the failure time to find the start of the attempt (most simulators report the start time as well).

Common mistake: Asserting a sequence that was meant as a scenario: `assert property (@(posedge clk) req ##1 gnt);`. This demands that `req` is true at **every** clock tick and `gnt` at every following one. It fails at the first tick where `req` is low. A rule needs an implication (`req |-> ##1 gnt`); a scenario you only want to observe needs `cover property`.

Verification application: Verification IP for a bus protocol is largely a library of named properties, one per rule in the protocol specification, each asserted in the interface and each paired with `cover property` statements that prove the interesting scenarios occurred. In formal verification the same properties on DUT inputs are switched from `assert` to `assume`.

Interview questions

Q1. What is a sequence? A specification of Boolean expressions that must be true at successive clock ticks, in a given order, with given delays. A sequence matches or does not match.

Q2. What is a property? A specification of design behaviour built from sequences and Boolean expressions with property operators such as implication. A property is true or false for each evaluation attempt and is what an assertion checks.

Q3. What is the difference between a sequence and a property? A sequence is a temporal pattern; a property is a rule. Implication, `not` and `disable iff` belong to properties. Sequences are building blocks for properties.

Q4. What are the layers of SVA? Boolean layer, sequence layer, property layer and directive (verification) layer.

Q5. What is the difference between `assert`, `assume` and `cover`? `assert` checks that the property holds. `assume` is checked like `assert` in simulation but in formal verification constrains the environment. `cover` does not check anything; it records that the property or sequence was observed.

Q6. What is `cover property` used for? Temporal functional coverage: confirming that a sequence of events (for example a back-to-back transfer, or a FIFO going from full to empty) actually occurred during simulation. It also shows that an assertion's triggering condition occurred, so the assertion did not pass only vacuously.

Q7. Why is `assert property (@(posedge clk) a ##1 b);` usually wrong? Without an implication, the sequence must match starting at every clock tick, so `a` must be true on every tick. The intended rule is normally `a |-> ##1 b` (or `a |=> b`).

4.5 Clocking Events

In plain English

A concurrent assertion needs to know **when to look**. The clocking event is that instant, typically the rising edge of a clock. All delays in the assertion (`##1`, `##[1:3]`) are counted in ticks of this event.

Technical definition

Every concurrent assertion is associated with a **clocking event**. The sampled values of the assertion's expressions are taken at each occurrence of the event, and one tick of delay (`##1`) means one occurrence of the event. The clock can be specified in four ways.

The four ways to specify the clock

```
// 1. Explicitly, in the assertion or in the property
a1: assert property (@(posedge clk) req |=> gnt);

property p_req_gnt;
  @(posedge clk) req |=> gnt;
endproperty
a2: assert property (p_req_gnt);

// 2. Explicitly, in a sequence
sequence s_req_gnt;
  @(posedge clk) req ##1 gnt;
endsequence

// 3. Inferred from an always block
always @(posedge clk) begin
  a3: assert property (req |=> gnt);      // clock inferred from the always block
end

// 4. Default clocking for the whole module or interface
default clocking cb_default @(posedge clk);
endclocking

a4: assert property (req |=> gnt);        // uses the default clocking
a5: assert property (valid |-> !$isunknown(data));
```

Method	When to use
Explicit `@(posedge clk)` in each property	A few assertions, or assertions on different clocks

Method	When to use
`default clocking`	Many assertions on the same clock: the normal style in an interface or checker module
Inferred from `always`	Assertions embedded in RTL next to the logic they check
A clocking block as the clock: `@(vif.mon_cb)` or `@(mon_cb)`	Inside an interface that already has clocking blocks (Module 8)

If none of these applies, the assertion has no clock and the compiler reports an error.

Variations

Clocking event	Meaning
`@(posedge clk)`	Rising edges
`@(negedge clk)`	Falling edges
`@(edge clk)`	Both edges (IEEE 1800-2009 and later)
`@(posedge clk iff en)`	Only the rising edges at which `en` is true. Ticks where `en` is false do not exist for the assertion: they are neither sampled nor counted by `##`

default clocking and default disable iff

```
interface alu_if (input logic clk);
  logic rst_n, valid_in, valid_out;
  // ...

  default clocking cb_sva @(posedge clk); endclocking
  default disable iff (!rst_n);

  a_valid_out: assert property (valid_in |=> valid_out);      // short and readable
endinterface
```

`default disable iff` (IEEE 1800-2009) does for the reset condition what `default clocking` does for the clock. An assertion that states its own clock or its own `disable iff` overrides the default.

Multi-clock properties (awareness level)

A property may refer to more than one clock:

```
// a request in the clk_a domain must be followed by an acknowledge in the clk_b domain
a_cdc: assert property (@(posedge clk_a) req |=> @(posedge clk_b) ack);
```

After the antecedent matches on `clk_a`, the consequent is evaluated at the **next tick of `clk_b`** that is strictly later. Multi-clock assertions are used at clock-domain crossings; the details are beyond what a fresher interview normally expects, but you should know that they exist.

Clock and sampling together: a timing diagram

```
property: @(posedge clk) a |=> b

clk    ___/‾‾‾\___/‾‾‾\___/‾‾‾\___/‾‾‾\___
tick      T1      T2      T3      T4

a      _________\_____________________      a is 1 before T1, 0 before T2
b      __________/‾‾‾‾‾‾‾‾‾‾‾‾‾\__________      b is 0 before T1, 1 before T2 and T3

T1: sampled a = 1  -> antecedent true, b must be 1 at T2
T2: sampled b = 1  -> attempt from T1 passes ; sampled a = 0 -> no new obligation
T3, T4: a = 0      -> nothing to check
```

Interview trap: "What is the clock of `assert property (req |=> gnt);` written directly in a module with no `default clocking`?" There is none, and it is a compile error. A concurrent assertion always needs a clocking event from one of the four sources.

Common mistake: Using a signal that glitches, or a gated clock that the design does not actually use, as the assertion clock. The assertion then samples at instants when the design is not sampling. Use the same clock (and edge) as the logic being checked.

Common mistake: Writing `@(posedge clk)` inside both the sequence and the property that uses it, on different clocks, by accident. This silently creates a multi-clock property with different semantics.

Interview questions

Q1. How is the clock of a concurrent assertion specified? Explicitly with `@(posedge clk)` in the assertion, property or sequence; by inference from an enclosing `always` block; through a `default clocking` declaration; or by naming a clocking block.

Q2. What is `default clocking`? A declaration that makes one clocking block the default clock for all concurrent assertions (and cycle delays) in a module, interface or program that do not specify their own clock.

Q3. What does `##1` mean? A delay of one tick of the assertion's clocking event, not one time unit.

Q4. What does `@(posedge clk iff en)` do? It uses only the rising clock edges where `en` is true as ticks. Edges where `en` is false are skipped completely.

Q5. Can a property use two different clocks? Yes. Multi-clock properties are allowed; the clock changes at an implication or at a `##1` between sub-sequences. They are used for clock-domain-crossing checks.

Q6. What is `default disable iff`? A declaration that supplies a default `disable iff` condition, usually the reset, for all concurrent assertions in the scope that do not have their own.

Part 5: Assertion Operators and System Functions

5.1 Implication Operators: |-> and |=>

In plain English

Most rules have the form **"if this happens, then that must happen"**. "If there is a request, there must be an acknowledge." The rule says nothing about cycles in which there is no request.

The implication operator expresses exactly this. Its left side is the trigger (the **antecedent**). Its right side is the obligation (the **consequent**). When the antecedent does not occur, there is nothing to check and the property is considered satisfied.

There are two implication operators. They differ only in **when** the consequent starts:

- `|->` **overlapped** implication: the consequent starts in the **same** clock tick in which the antecedent ends.
- `|=>` **non-overlapped** implication: the consequent starts in the **next** clock tick.

Technical definition

```
antecedent_sequence  |->  consequent_property      overlapped
antecedent_sequence  |=>  consequent_property      non-overlapped
```

For each evaluation attempt:

1. The antecedent sequence is evaluated starting at the tick of the attempt.
2. If the antecedent **does not match**, the property is true for this attempt. This is called **vacuous success**.
3. If the antecedent **matches**, the consequent is evaluated starting
   - at the tick where the antecedent match **ended**, for `|->`;
   - at the tick **after** the antecedent match ended, for `|=>`.
4. The property is true if the consequent holds, false if it does not.

The two operators are related by an exact identity:

```
a |=> b      is the same as      a |-> ##1 b
```

Implication can only be used in a **property**, never inside a sequence.

Timing diagrams

```
Overlapped:       a |-> b             "when a is true, b must be true in the SAME tick"

   tick        T1     T2     T3
   a            1      0      0
   b            1      -      -           checked at T1
                ^
                antecedent and consequent in the same tick

Non-overlapped:   a |=> b             "when a is true, b must be true in the NEXT tick"

   tick        T1     T2     T3
   a            1      0      0
   b            -      1      -           checked at T2
                ^      ^
                |      consequent
                antecedent
```

When the antecedent is a multi-cycle sequence, the consequent is anchored to the **end** of the antecedent:

```
(a ##1 b) |-> c          a at T1, b at T2, c must be true at T2  (same tick as b)
(a ##1 b) |=> c          a at T1, b at T2, c must be true at T3
```

Complete example

tb_sva_implication.sv

```systemverilog
module tb_sva_implication;

  logic clk = 0;
  logic req, ack;

  // stimulus for cycles 0..6 (cycle k is the posedge at time 10k+5)
  bit [0:6] req_v = 7'b0101010;
  bit [0:6] ack_v = 7'b0110001;

  always #5 clk = ~clk;

  // overlapped: ack in the SAME cycle as req
  a_overlap: assert property (@(posedge clk) req |-> ack)
    else $display("[%0t] a_overlap    FAILED: req without ack in the same cycle", $time);

  // non-overlapped: ack in the cycle AFTER req
  a_nonoverlap: assert property (@(posedge clk) req |=> ack)
    else $display("[%0t] a_nonoverlap FAILED: req not followed by ack", $time);

  initial begin
    for (int i = 0; i < 7; i++) begin
      req = req_v[i];
      ack = ack_v[i];
      @(negedge clk);
    end
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[35] a_overlap    FAILED: req without ack in the same cycle
[45] a_nonoverlap FAILED: req not followed by ack
[55] a_overlap    FAILED: req without ack in the same cycle
```

Evaluation, cycle by cycle

Cycle	Time	`req`	`ack`	`a_overlap` (`req |-> ack`)	`a_nonoverlap` (`req |=> ack`)
0	5	0	0	Vacuous pass	Vacuous pass
1	15	1	1	**Pass**: `ack` is 1 now	Attempt starts; `ack` needed at cycle 2
2	25	0	1	Vacuous pass	**Pass** (attempt of cycle 1); new attempt vacuous
3	35	1	0	**FAIL**: `ack` is 0 now	Attempt starts; `ack` needed at cycle 4
4	45	0	0	Vacuous pass	**FAIL** (attempt of cycle 3)
5	55	1	0	**FAIL**	Attempt starts; `ack` needed at cycle 6
6	65	0	1	Vacuous pass	**Pass** (attempt of cycle 5)

Points to notice

Observation	Explanation
The same stimulus gives different results	At cycle 5, `req` is high with no `ack`: the overlapped assertion fails immediately. The acknowledge arrives one cycle later, which satisfies the non-overlapped assertion. Choosing the wrong operator makes a correct design fail or a wrong design pass.
Failure time of `a_nonoverlap`	The request was at time 35; the failure is reported at time 45, when the consequent was evaluated.
Four of the seven attempts of each assertion were vacuous	Cycles 0, 2, 4 and 6 have `req` = 0. Nothing was checked in them.

Vacuous success

An implication whose antecedent never matches passes at every tick without ever checking anything. This is the main weakness of assertions: **an assertion that never triggers looks exactly like an assertion that always holds**.

```
a_req_ack: assert property (@(posedge clk) req |=> ack);

If the test never drives req = 1:
     every attempt is a vacuous pass   ->   zero failures   ->   "assertion passed"
     but the rule was never tested
```

How vacuity is detected and handled:

Technique	How
Cover the antecedent	`c_req: cover property (@(posedge clk) req);` If this cover is never hit, the assertion was never exercised
Assertion coverage reports	Simulators report, per assertion, the number of attempts, real (non-vacuous) passes, vacuous passes and failures
Do not count vacuous passes as passes	`$assertvacuousoff;` stops pass action blocks from executing on vacuous success
Review	An assertion with zero real passes in the whole regression is a hole, the same as an uncovered bin

Which operator to choose

Situation	Operator	Example
The response is combinational, or must be true in the same cycle as the condition	`|->`	`wr_en |-> !full`
The response comes from a flip-flop clocked by the same edge, so it is visible one cycle later	`|=>`	`valid_in |=> valid_out`
The response comes after an explicit number of cycles	`|->` with a delay	`req |-> ##[1:3] ack`
Checking the state **after** a clocked update, in terms of the state before	`|=>` with `$past`	`en |=> count == $past(count) + 4'd1`

Overlapped versus non-overlapped

Property	`|->` overlapped	`|=>` non-overlapped
Consequent starts	In the tick where the antecedent ends	In the tick after the antecedent ends
Equivalent form	`a |-> b`	`a |-> ##1 b`
Read as	"if a, then b at the same time"	"if a, then b on the next clock"
Typical use	Combinational relationships, conditions that must hold simultaneously, consequents that begin with an explicit `##` delay	Registered outputs, next-state checks
Vacuous when	The antecedent does not match	The antecedent does not match

Interview trap: "What is the difference between `a |=> b` and `a |-> ##1 b`?" None. They are equivalent. And "between `a |-> b` and `a |=> b`?": one clock tick.

Interview trap: "The assertion `req |=> ack` passed in every test. Is the handshake verified?" Not necessarily. If `req` was never asserted, all passes were vacuous. Check the assertion coverage or add a `cover property` on the antecedent.

Common mistake: Writing `|=> ##1 b` when one cycle of delay was intended. `a |=> ##1 b` means `b` **two** ticks after `a`.

Common mistake: Using a level instead of an edge in the antecedent. `req |-> ##[1:3] ack` starts a new attempt at every tick while `req` is high. If `req` stays high for five cycles, five overlapping attempts are created and one `ack` may satisfy several of them, or late attempts may fail unexpectedly. `$rose(req) |-> ##[1:3] ack` starts one attempt per request.

Common mistake: Trying to use implication inside a sequence, for example `sequence s; a |-> b; endsequence`. Implication is a property operator; the code does not compile.

Interview questions

Q1. What is the implication operator? A property operator of the form antecedent implies consequent. If the antecedent sequence matches, the consequent must hold; if the antecedent does not match, the property succeeds vacuously.

Q2. What is the difference between `|->` and `|=>`? With `|->` (overlapped) the consequent is evaluated starting in the same clock tick in which the antecedent matched. With `|=>` (non-overlapped) it is evaluated starting one clock tick later. `a |=> b` is equivalent to `a |-> ##1 b`.

Q3. What is vacuous success? The success of an implication because its antecedent did not match, so that the consequent was never checked.

Q4. Why is vacuous success a problem and how do you detect it? An assertion whose antecedent never occurs reports no failures although it never checked anything. It is detected with assertion coverage (real passes versus vacuous passes) or with a `cover property` on the antecedent.

Q5. In `(a ##2 b) |=> c`, at which tick is `c` checked if the attempt starts at tick 1? `a` at tick 1, `b` at tick 3 (the antecedent ends there), `c` at tick 4.

Q6. Can the antecedent be a property? No. The antecedent must be a sequence (a Boolean is the simplest sequence). The consequent may be a property.

Q7. Write an assertion: whenever `start` is high, `busy` must be high in the same cycle. `assert property (@(posedge clk) start |-> busy);`

5.2 Cycle Delays and Repetition

In plain English

Protocol rules talk about time: "two cycles later", "within one to four cycles", "for three cycles in a row", "after the third occurrence". The `##` operator counts clock ticks between events. The repetition operators say how many times something must happen.

Cycle delay:

Syntax	Meaning
`a ##1 b`	`b` is true exactly one tick after `a`
`a ##N b`	`b` is true exactly N ticks after `a` (N is a constant ≥ 0)
`a ##0 b`	`a` and `b` are true in the same tick (used to join two sequences at a common tick)
`a ##[m:n] b`	`b` is true at some tick between m and n ticks after `a`
`a ##[1:$] b`	`b` is true at some later tick; `$` means "no upper limit"
`##2 b`	A sequence beginning with a delay: `b` is true two ticks after the starting point
`a ##[*] b`, `a ##[+] b`	Short forms of `##[0:$]` and `##[1:$]` (IEEE 1800-2009)

```
a ##2 b                    tick:   T    T+1   T+2
                           a       1
                           b                  1

a ##[1:3] b                tick:   T    T+1   T+2   T+3
                           a       1
                           b            ?     ?     ?       b must be 1 at one of these

As a consequent, ##[1:3] b succeeds at the FIRST tick where b is 1,
and fails at T+3 if b was 0 at T+1, T+2 and T+3.
```

Repetition operators

Operator	Name	Meaning
`a [*n]`	Consecutive repetition	`a` is true on n consecutive ticks
`a [*m:n]`	Consecutive repetition range	`a` is true on m to n consecutive ticks
`a [*0:$]`, `a [*]`	Zero or more	`a` true for any number of consecutive ticks, including none
`a [*1:$]`, `a [+]`	One or more	`a` true for at least one tick
`a [->n]`	Goto repetition	`a` is true on n ticks, **not necessarily consecutive**; the match **ends at the tick of the n-th occurrence**
`a [=n]`	Non-consecutive repetition	`a` is true on n ticks, not necessarily consecutive; the match may **extend beyond** the n-th occurrence for as long as `a` stays false

Exact definitions in terms of simpler operators:

```
a [*3]     =   a ##1 a ##1 a
a [->1]    =   (!a) [*0:$] ##1 a                  "wait for a"
a [->2]    =   (!a)[*0:$] ##1 a ##1 (!a)[*0:$] ##1 a
a [=2]     =   a [->2] ##1 (!a) [*0:$]
```

A repetition can also be applied to a parenthesised sequence: `(a ##1 b) [*2]` means `a ##1 b ##1 a ##1 b`.

The three repetitions on one timeline

```
tick     1    2    3    4    5    6    7    8
a        0    1    0    1    1    0    0    0
b        0    0    0    0    0    1    0    1

Sequence started at tick 1:

a [*2]                 needs a at ticks 1 and 2        -> no match (a = 0 at tick 1)
started at tick 4:     a at 4 and 5                    -> match, ends at tick 5

a [->2]                the 2nd occurrence of a is at tick 4
                                                       -> match, ends at tick 4

a [->3] ##1 b          the 3rd a is at tick 5, b must be 1 at tick 6
                                                       -> match, ends at tick 6

a [=3]  ##1 b          the 3rd a is at tick 5; then a may stay 0 for a while;
                       b may be at tick 6, 7, 8 ... as long as a has not been 1 again
                                                       -> matches at tick 6 and at tick 8

a [->3] ##1 b  if b were 0 at tick 6                   -> no match (b must come at once)
a [=3]  ##1 b  if b were 0 at tick 6                   -> still matches at tick 8
```

Consecutive versus goto versus non-consecutive

Property	`[*n]`	`[->n]`	`[=n]`
Occurrences must be consecutive	Yes	No	No
Where the match ends	At the n-th consecutive tick	Exactly at the n-th occurrence	At the n-th occurrence or any later tick before the next occurrence
What may follow	The next element comes right after the last repetition	The next element comes right after the n-th occurrence	The next element may come after a gap
Typical use	"held for n cycles"	"after the n-th event, immediately ..."	"n events happened before ..."
Example	`busy [*3]`	`beat [->4] ##1 last`	`beat [=4] ##1 done`

Complete example

tb_sva_delay_rep.sv

```
module tb_sva_delay_rep;

  logic clk = 0;
  logic start, busy, done;

  // stimulus for cycles 0..11 (cycle k is the posedge at time 10k+5)
  bit [0:11] start_v = 12'b0100_0010_0000;
  bit [0:11] busy_v  = 12'b0011_0001_0000;
  bit [0:11] done_v  = 12'b0001_0000_0000;

  always #5 clk = ~clk;

  // done must arrive 1 to 3 cycles after start rises
  a_range: assert property (@(posedge clk) $rose(start) |-> ##[1:3] done)
    else $display("[%0t] a_range FAILED: no done within 3 cycles of start", $time);

  // after start rises, busy must be high for exactly 2 cycles and then low
  a_busy: assert property (@(posedge clk) $rose(start) |=> busy [*2] ##1 !busy)
    else $display("[%0t] a_busy  FAILED: busy was not high for 2 cycles", $time);

  initial begin
    for (int i = 0; i < 12; i++) begin
      start = start_v[i];
      busy  = busy_v[i];
      done  = done_v[i];
      @(negedge clk);
    end
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[85] a_busy  FAILED: busy was not high for 2 cycles
[95] a_range FAILED: no done within 3 cycles of start
```

Stimulus

cycle	0	1	2	3	4	5	6	7	8	9	10	11
time	5	15	25	35	45	55	65	75	85	95	105	115
start	0	1	0	0	0	0	1	0	0	0	0	0
busy	0	0	1	1	0	0	0	1	0	0	0	0
done	0	0	0	1	0	0	0	0	0	0	0	0

Evaluation

Attempt	`a_range`: `$rose(start) |-> ##[1:3] done`	`a_busy`: `$rose(start) |=> busy [*2] ##1 !busy`
Starts at cycle 1 (`start` rises)	`done` allowed at cycles 2, 3, 4. It is 0 at cycle 2 and 1 at cycle 3: **passes at cycle 3**	Cycle 2: `busy` =1. Cycle 3: `busy` =1. Cycle 4: `busy` =0. **Passes at cycle 4**
Starts at cycle 6 (`start` rises again)	`done` allowed at cycles 7, 8, 9. It is 0 at all three: **fails at cycle 9 (time 95)**	Cycle 7: `busy` =1. Cycle 8: `busy` =0, but a second high cycle was required: **fails at cycle 8 (time 85)**
All other cycles	`$rose(start)` is false: vacuous	Vacuous

Line by line

Code	Explanation
`$rose(start)`	True in the tick where the sampled `start` is 1 and was 0 in the previous tick. It gives one attempt per start pulse. Section 5.3 covers it.
`|-> ##[1:3] done`	The overlapped implication followed by a delay range. The consequent begins at the antecedent's tick and looks 1 to 3 ticks ahead.
`|=> busy [*2] ##1 !busy`	Starting one tick after `start` rose: two consecutive ticks of `busy`, then one tick of `!busy`.
Failure times	`a_busy` fails as soon as the pattern is broken (cycle 8). `a_range` can only fail when the whole window has passed (cycle 9).

Unbounded delays and "eventually"

```
a_eventually: assert property (@(posedge clk) req |-> ##[1:$] ack);
```

This says that `ack` must follow `req` at some point, with no deadline. In simulation such an assertion **can never fail**: if `ack` never comes, the attempt is simply still waiting when the simulation ends. Some tools report unfinished attempts at the end of the run; many do not by default.

For simulation, always give a bound (`##[1:16] ack`). To make an open-ended requirement fail at the end of simulation, IEEE 1800-2009 provides strong operators: `req |-> strong(##[1:$] ack)` or `req |-> s_eventually ack`. These are mainly used in formal verification.

Interview trap: "What is the difference between `a ##1 b [*3]` and `(a ##1 b) [*3]`?" Repetition binds more tightly than `##`. The first is `a ##1 b ##1 b ##1 b` (four ticks). The second is `a ##1 b ##1 a ##1 b ##1 a ##1 b` (six ticks).

Interview trap: "What is the difference between `[->n]` and `[=n]`?" Both count n non-consecutive occurrences. With goto `[->n]` the sequence ends at the n-th occurrence, so whatever follows must come immediately after it. With `[=n]` the sequence may continue past the n-th occurrence while the expression is false, so whatever follows may come later.

Common mistake: Reading `##[1:3]` as "for three cycles". It is a window in which the next expression must be true **once**. "True for three cycles" is `[*3]`.

Common mistake: Using a variable as a delay or repetition count: `##n` where `n` is a signal. The values in `##` and `[*]` must be compile-time constants (parameters are fine). A variable number of cycles needs a counter with a local variable, or a different formulation.

Verification application: Typical uses in a bus checker: `$rose(req) |-> ##[1:8] gnt` (response time limit), `$fell(cs_n) |-> !cs_n [*4]` (minimum chip-select length), `start |=> (data_valid [->8]) ##1 done` (exactly eight data beats, then done).

Interview questions

Q1. What does `##3` mean? A delay of three ticks of the assertion's clock.

Q2. What does `a ##[2:4] b` mean? After `a` is true, `b` must be true 2, 3 or 4 ticks later.

Q3. Explain `[*n]`, `[->n]` and `[=n]`. `[*n]`: the expression is true on n consecutive ticks. `[->n]`: goto repetition, true on n not necessarily consecutive ticks, ending exactly at the n-th. `[=n]`: non-consecutive repetition, true on n ticks, and the match may extend past the n-th occurrence while the expression remains false.

Q4. Write a sequence for "`a` high for 3 cycles, then `b` high for 2 cycles". `a [*3] ##1 b [*2]`

Q5. Write an assertion: after `req` rises, `gnt` must come within 1 to 5 cycles. `assert property (@(posedge clk) $rose(req) |-> ##[1:5] gnt);`

Q6. Can `req |-> ##[1:$] ack` fail in simulation? No. With an unbounded upper limit the attempt just remains pending if `ack` never arrives. A finite bound, or a strong operator such as `s_eventually`, is needed for a failure to be reported.

Q7. What does `a [*0]` mean? An empty match: `a` repeated zero times, taking no clock ticks. It appears inside ranges such as `a [*0:3]`, where the sequence may skip `a` entirely.

5.3 Sampled Value Functions: $rose, $fell, $stable, $changed, $past

In plain English

A Boolean such as `valid` tells you the **level** of a signal at one tick. Many rules are about **changes**: "when `valid` goes high", "while `valid` is waiting, `data` must not change", "the counter is one more than it was". These five functions compare the value at the current tick with the value at earlier ticks.

Technical definition

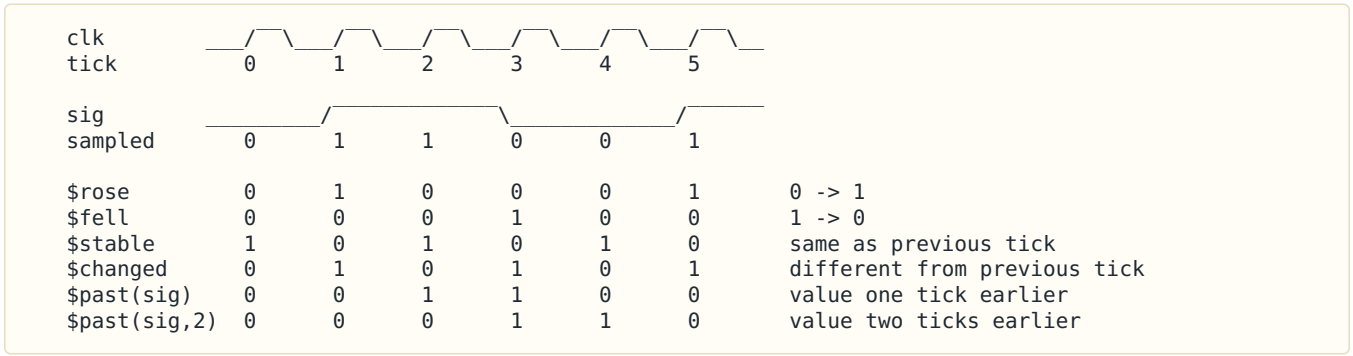
The sampled value functions use the **sampled value** of their argument at the current tick of the clocking event and at previous ticks.

Function	Returns true (1) when	Applies to
`$rose(expr)`	The least significant bit of `expr` changed **to 1** since the previous tick (from 0, X or Z)	LSB only
`$fell(expr)`	The least significant bit of `expr` changed **to 0** since the previous tick (from 1, X or Z)	LSB only
`$stable(expr)`	The value of `expr` is the **same** as at the previous tick	Whole expression
`$changed(expr)`	The value of `expr` is **different** from the previous tick (IEEE 1800-2009)	Whole expression
`$past(expr)`	Not a Boolean: returns the **value** `expr` had at the previous tick	Whole expression
`$past(expr, n)`	The value `expr` had n ticks ago	Whole expression
`$past(expr, n, gate)`	The value `expr` had n ticks ago, counting only ticks at which `gate` was true	Whole expression

Function	Returns true (1) when	Applies to
`$sampled(expr)`	The sampled value of `expr` at the current tick	Whole expression

Each function takes an optional last argument giving the clocking event; inside a clocked assertion it is taken from the assertion.

How it works internally



The simulator keeps a small history of the sampled values of each expression used in such a function, like a shift register clocked by the assertion clock.

The first tick. At the first tick there is no earlier sample. The "previous" value is then the initial value of the variable: its declaration initialiser if it has one, otherwise the default of its type (0 for `bit`, **X for** `logic`). The diagram above assumes a two-state signal. For a `logic` signal that is 0 at the first tick without having been initialised in its declaration, the previous value is X, so `$fell` is **true** and `$stable` is false at that first tick. This is one reason almost every assertion has a `disable iff` for reset.

Complete example

Sampled value functions can also be called in procedural code inside a clocked `always` block, where they use the clock of the block. This lets us print them tick by tick.

tb_sva_sampled_funcs.sv

```
module tb_sva_sampled_funcs;

  logic clk = 0;
  bit   sig;

  // stimulus for cycles 0..5 (cycle k is the posedge at time 10k+5)
  bit [0:5] sig_v = 6'b011001;

  always #5 clk = ~clk;

  always @(posedge clk)
    $display("[%0t] sig=%b  rose=%b fell=%b stable=%b changed=%b  past=%b past2=%b",
             $time, sig, $rose(sig), $fell(sig), $stable(sig), $changed(sig),
             $past(sig), $past(sig, 2));

  initial begin
    for (int i = 0; i < 6; i++) begin
      sig = sig_v[i];
      @(negedge clk);
    end
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[5] sig=0  rose=0 fell=0 stable=1 changed=0  past=0 past2=0
[15] sig=1  rose=1 fell=0 stable=0 changed=1  past=0 past2=0
[25] sig=1  rose=0 fell=0 stable=1 changed=0  past=1 past2=0
[35] sig=0  rose=0 fell=1 stable=0 changed=1  past=1 past2=1
[45] sig=0  rose=0 fell=0 stable=1 changed=0  past=0 past2=1
[55] sig=1  rose=1 fell=0 stable=0 changed=1  past=0 past2=0
```

Line by line

Time	Explanation
5	First tick. `sig` is a `bit`, so the "previous" value is the default 0: nothing rose or fell, the value is stable, `$past` returns 0.
15	`sig` went 0 → 1: `$rose` and `$changed` are true. `$past(sig)` is the value at time 5.
25	`sig` stayed 1: `$stable` is true. `$past(sig, 2)` is still the value from time 5.
35	`sig` went 1 → 0: `$fell` and `$changed` are true. `$past(sig, 2)` is the value from time 15.
45	Stable at 0.
55	Rose again.

Using the functions in assertions

```
// A request pulse starts exactly one transaction
a_gnt: assert property (@(posedge clk) $rose(req) |-> ##[1:4] gnt);

// When chip select is released, the bus must be released in the same cycle
a_release: assert property (@(posedge clk) $fell(cs) |-> !oe);

// While valid waits for ready, data must not change
a_stable: assert property (@(posedge clk) valid && !ready |=> $stable(data));

// The address may only change when the bus is idle
a_addr: assert property (@(posedge clk) $changed(addr) |-> !busy);

// A register captures its input: q is the previous value of d
a_reg: assert property (@(posedge clk) en |=> q == $past(d));

// A counter increments by one when enabled
a_cnt: assert property (@(posedge clk) en |=> count == $past(count) + 4'd1);
```

$past in detail

Call	Meaning
`$past(d)`	Value of `d` one tick ago
`$past(d, 3)`	Value of `d` three ticks ago
`$past(d, 1, en)`	Value of `d` at the most recent earlier tick at which `en` was true
`$past(d, 2, en)`	Value of `d` at the second most recent earlier tick at which `en` was true

The gating form is used with clock enables: if a register only loads when `en` is 1, then "the previous value" in the sense of the design is the value at the last enabled tick, not at the last clock.

`$past(d, n)` and the delay `##n` are two ways of spanning time. The following two assertions state the same rule:

```
// looking forward from the cause
a_fwd : assert property (@(posedge clk) valid_in |=> valid_out);

// looking backward from the effect (also catches valid_out without valid_in)
a_back: assert property (@(posedge clk) valid_out |-> $past(valid_in));
```

In practice both directions are asserted: the first says "every input produces an output", the second says "there is no output without an input".

$rose versus a level, and $rose versus posedge

Expression	Meaning
`sig`	The sampled value is 1 at this tick (level)
`$rose(sig)`	The sampled value is 1 at this tick and was not 1 at the previous tick (a synchronous edge, seen through the clock)
`@(posedge sig)`	An event: the signal itself rises, at any time. This is an event control, not an expression, and cannot be used as a term inside a sequence

```
clk     ___/‾‾\___/‾‾\___/‾‾\___/‾‾\___/‾‾\___
tick       1       2       3       4       5
req     _____/‾‾‾‾‾‾‾‾‾‾‾‾‾‾‾‾‾‾‾‾‾‾‾‾\_________
sampled    0       1       1       1       0

req               -> true at ticks 2, 3, 4      (three attempts are started)
$rose(req)        -> true at tick 2 only        (one attempt is started)
```

Interview trap: "`logic [3:0] vec` changes from `4'b0010` to `4'b0100`. Is `$rose(vec)` true?" No. `$rose` and `$fell` look at the **least significant bit only**, and bit 0 stayed 0. To detect "the vector became non-zero" use `$rose(|vec)`; to detect any change use `$changed(vec)`.

Interview trap: "What does `$past(sig)` return at the first clock tick?" The initial value of `sig`: its declaration initialiser, or the default of its type, which is X for four-state types.

Common mistake: `count == $past(count) + 1` for a counter that wraps. The literal `1` is 32 bits wide, so the right side is evaluated in 32 bits: when `$past(count)` is 15, the sum is 16, which never equals the 4-bit `count` value 0. Write `$past(count) + 4'd1`, so that the addition wraps like the hardware. Section 6.3 demonstrates this bug.

Common mistake: Using `$stable(sig)` in the same tick as the condition when the stability is required from the next tick on: `valid && !ready |-> $stable(data)` compares the data with the tick **before** `valid` rose. The correct form is `valid && !ready |=> $stable(data)`.

Common mistake: `@(posedge clk) $rose(clk)`. The clock sampled at its own rising edge is always the pre-edge value 0, so `$rose(clk)` is never true. Do not apply sampled value functions to the assertion's own clock.

Interview questions

Q1. What do `$rose`, `$fell` and `$stable` do? They compare the sampled value at the current clock tick with that at the previous tick. `$rose` is true if the LSB changed to 1, `$fell` if the LSB changed to 0, and `$stable` if the whole value is unchanged.

Q2. What is `$past`? A function that returns the sampled value an expression had a given number of clock ticks earlier (one by default), optionally counting only ticks at which a gating expression was true.

Q3. What is the difference between `$rose(a)` and `@(posedge a)`? `$rose(a)` is a Boolean evaluated at ticks of the assertion clock; it is true if the sampled `a` is 1 now and was not 1 at the previous tick. `@(posedge a)` is an event control that triggers at the actual instant `a` rises, independent of any clock.

Q4. What is `$changed`? The complement of `$stable`: true if the value differs from the previous tick.

Q5. `$rose` is applied to a multi-bit signal. What is checked? Only bit 0.

Q6. Write an assertion: `data` must not change while `valid` is high and `ready` is low. `assert property (@(posedge clk) valid && !ready |=> $stable(data));`

Q7. Write an assertion that `q` is `d` delayed by two clocks.
`assert property (@(posedge clk) q == $past(d, 2));` (with a suitable `disable iff` so that the first two ticks after reset are not checked).

Q8. Why would you use `$rose(req)` instead of `req` as an antecedent? To start exactly one evaluation attempt per request. With a level, a new attempt starts at every tick while the signal is high.

5.4 Vector Functions: $onehot, $onehot0, $isunknown, $countones

In plain English

These four functions look at the bits of **one** vector at one moment and answer a question about them: is exactly one bit set, is at most one bit set, is any bit unknown, how many bits are set. They are ordinary system functions: they work in assertions, in procedural code and in coverage.

Technical definition

Function	Returns	Definition
`$onehot(expr)`	1 if **exactly one** bit of `expr` is 1	`$countones(expr) == 1`
`$onehot0(expr)`	1 if **at most one** bit of `expr` is 1 (zero or one)	`$countones(expr) <= 1`
`$isunknown(expr)`	1 if **any** bit of `expr` is X or Z	`^expr === 1'bx`
`$countones(expr)`	The number of bits of `expr` that are 1 (an integer)	X and Z bits are not counted
`$countbits(expr, v1, v2, ...)`	The number of bits having any of the listed values (IEEE 1800-2012)	`$countbits(e, '0)` counts zeros; `$countbits(e, 'x, 'z)` counts unknown bits

Values at a glance

`expr`	`$onehot`	`$onehot0`	`$isunknown`	`$countones`
`4'b0000`	0	1	0	0
`4'b0100`	1	1	0	1
`4'b0110`	0	0	0	2
`4'b1111`	0	0	0	4
`4'b01x0`	1	1	1	1
`4'bzzzz`	0	1	1	0

The last two rows show that `$onehot` and `$onehot0` count only the bits that are definitely 1. A vector with one 1 and one X passes `$onehot`. If X must be rejected as well, combine the checks: `$onehot(v) && !$isunknown(v)`.

Typical assertions

```
// A one-hot encoded state register always has exactly one bit set
a_state_onehot: assert property (@(posedge clk) disable iff (!rst_n) $onehot(state));

// An arbiter grants to at most one requester
a_one_grant: assert property (@(posedge clk) disable iff (!rst_n) $onehot0(grant));

// Chip selects: never two active at once (active low, so invert first)
a_cs: assert property (@(posedge clk) $onehot0(~cs_n));

// Control signals are never unknown after reset
a_ctrl_known: assert property (@(posedge clk) disable iff (!rst_n)
                               !$isunknown({valid_in, valid_out}));

// Data is known whenever it is qualified by valid
a_data_known: assert property (@(posedge clk) disable iff (!rst_n)
                               valid_out |-> !$isunknown(result));

// At most two of the eight interrupt lines may be active
a_irq: assert property (@(posedge clk) $countones(irq) <= 2);
```

$onehot versus $onehot0

Property	`$onehot`	`$onehot0`
All bits zero	Fails (returns 0)	Passes (returns 1)
Exactly one bit set	Passes	Passes
Two or more bits set	Fails	Fails
Use when	One must always be active: one-hot FSM state, one-hot select of a multiplexer	Idle is allowed: grants, chip selects, interrupt acknowledge

Interview trap: "What is the difference between `$onehot` and `$onehot0`?" `$onehot0` also accepts the all-zero value. An arbiter with no request has `grant` = 0; `$onehot(grant)` would fail in every idle cycle.

Interview trap: "Why write `valid |-> !$isunknown(data)` instead of just `!$isunknown(data)`?" Data buses are legitimately X or Z when they carry nothing (an idle or tri-stated bus). The X check is meaningful only when the data is qualified by `valid`. Control signals such as `valid` itself must never be X, so they are checked unconditionally.

Verification application: "No X on the interface after reset" is the first assertion written in almost every interface. X on a control signal usually means a missing reset or an unconnected port, and since `if` statements treat X as false (Module 1, Section 4.3), the testbench might otherwise never notice.

Interview questions

Q1. What does `$onehot` return? 1 if exactly one bit of the expression is 1, otherwise 0.

Q2. What is the difference between `$onehot` and `$onehot0`? `$onehot` requires exactly one bit set. `$onehot0` requires at most one bit set, so it also returns 1 when all bits are zero.

Q3. What does `$isunknown` return? 1 if any bit of the expression is X or Z.

Q4. What does `$countones` do? It returns the number of bits in the expression that are 1.

Q5. Write an assertion that a 4-bit `grant` is one-hot or zero and never unknown. `assert property (@(posedge clk) disable iff (!rst_n) $onehot0(grant) && !$isunknown(grant));`

Q6. How would you check that a signal has even parity with these functions? `$countones(data) % 2 == 0`, or equivalently `^data == 1'b0`.

5.5 disable iff

In plain English

During reset the design is deliberately not following its normal rules: outputs are forced to their reset values, handshakes are cut off in the middle. An assertion that keeps checking during reset reports failures that are not bugs. `disable iff` tells an assertion: **"while this condition is true, do not check, and throw away any check that is in progress"**.

Technical definition

`disable iff (expression)` specifies a **disable condition** (normally the reset) for a property.

- If the disable condition is true when an evaluation attempt would start, the attempt is disabled.
- If the disable condition becomes true **at any time while an attempt is in progress**, even between two clock ticks, that attempt is discarded.
- A disabled attempt is neither a pass nor a fail. Its action blocks do not execute.
- The disable expression is **not sampled**: its current value is used. This makes `disable iff` behave like an asynchronous reset.

Syntax

```
property p_req_ack;
  @(posedge clk) disable iff (!rst_n)       // after the clock, before the property body
    req |=> ack;
endproperty

a1: assert property (@(posedge clk) disable iff (!rst_n) req |=> ack);

default disable iff (!rst_n);               // for every assertion in this scope
```

Rules:

Rule	Detail
Position	At the top level of the property, after the clocking event. It cannot be nested inside a sub-property or a sequence
Number	At most one `disable iff` per assertion
Polarity	The expression is the condition that **disables**. For an active-low reset `rst_n` it is `!rst_n`; for an active-high `rst` it is `rst`
Default	`default disable iff (expr);` applies to all assertions in the scope that have no `disable iff` of their own
Override	An assertion with its own `disable iff` ignores the default. `disable iff (1'b0)` switches the default off for one assertion

How it works

```
property: @(posedge clk) disable iff (!rst_n)  req |=> ack

clk     ___/‾‾\___/‾‾\___/‾‾\___/‾‾\___/‾‾\___
tick       1      2      3      4      5
rst_n   ‾‾‾‾‾‾‾‾‾‾‾‾‾‾‾‾‾‾‾‾‾‾\______/‾‾‾‾‾‾        reset pulse between tick 3 and tick 4
req        0      0      1      0      0
ack        0      0      0      0      0

attempt at tick 3: req = 1, so ack is required at tick 4
                   reset becomes active before tick 4  ->  attempt DISCARDED, no failure

without disable iff: the same attempt FAILS at tick 4 (ack = 0)
```

Complete example

tb_sva_disable_iff.sv

```systemverilog
module tb_sva_disable_iff;

  logic clk = 0;
  logic rst_n, req, ack;

  // stimulus for cycles 0..5 (cycle k is the posedge at time 10k+5)
  bit [0:5] rst_v = 6'b011101;      // reset active in cycle 0 and again in cycle 4
  bit [0:5] req_v = 6'b101100;
  bit [0:5] ack_v = 6'b000100;

  always #5 clk = ~clk;

  a_with_disable: assert property (@(posedge clk) disable iff (!rst_n) req |=> ack)
    else $display("[%0t] a_with_disable FAILED", $time);

  a_no_disable: assert property (@(posedge clk) req |=> ack)
    else $display("[%0t] a_no_disable   FAILED", $time);

  initial begin
    for (int i = 0; i < 6; i++) begin
      rst_n = rst_v[i];
      req   = req_v[i];
      ack   = ack_v[i];
      @(negedge clk);
    end
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[15] a_no_disable   FAILED
[45] a_no_disable   FAILED
```

Evaluation

Cycle	Time	`rst_n`	`req`	`ack`	`a_with_disable`	`a_no_disable`
0	5	0	1	0	Disabled: reset is active	Attempt starts
1	15	1	0	0	Vacuous	**FAIL**: `ack` is 0 (attempt of cycle 0)
2	25	1	1	0	Attempt starts	Attempt starts
3	35	1	1	1	Pass (attempt of cycle 2); new attempt starts	Pass; new attempt starts

Cycle	Time	`rst_n`	`req`	`ack`	`a_with_disable`	`a_no_disable`
4	45	0	0	0	Attempt of cycle 3 **discarded**: reset went low at time 40	**FAIL**: `ack` is 0 (attempt of cycle 3)
5	55	1	0	0	Vacuous	Vacuous

Explanation

- The request seen at cycle 0 is sampled while reset is active. A design in reset will not answer it. `a_no_disable` reports a failure that is not a design bug.
- The request at cycle 3 is cut off by a reset at time 40, in the middle of the attempt. `a_with_disable` discards the attempt at the moment reset is asserted; `a_no_disable` fails.
- `a_with_disable` produces no output: it never fails, and it has no pass statement.

disable iff versus putting reset in the antecedent

Form	Behaviour
`disable iff (!rst_n) req |=> ack`	Reset prevents attempts from starting **and** cancels attempts already running. Reset is evaluated asynchronously
`(rst_n && req) |=> ack`	Reset only prevents attempts from **starting** (sampled at the clock). An attempt that started before reset continues and may fail during reset
`req |=> (ack || !rst_n)`	Each term must be handled by hand; unreadable for multi-cycle properties

Interview trap: "`disable iff (rst_n)` with an active-low reset: what happens?" The assertion is disabled whenever the design is **out of** reset, which is the whole useful part of the simulation. It never fails and never checks anything. Wrong reset polarity in `disable iff` is one of the most common silent assertion bugs; assertion coverage (zero non-vacuous passes) reveals it.

Interview trap: "Is the `disable iff` condition sampled like the rest of the property?" No. It is evaluated with current values, continuously. A reset pulse shorter than a clock period still disables the attempts that are active at that time.

Common mistake: Disabling the assertions that check the reset behaviour itself. `!rst_n |-> count == 0` under `default disable iff (!rst_n)` is always disabled. Give such an assertion `disable iff (1'b0)`, or place it in a scope without the default.

Interview questions

Q1. What is `disable iff`? A clause of a property that specifies a condition, usually reset, under which the property is not checked. Attempts that would start, and attempts already in progress, are disabled while the condition is true.

Q2. What is the result of an attempt that is disabled? Neither pass nor fail. It is discarded and no action block runs.

Q3. Is `disable iff` synchronous or asynchronous? Asynchronous. The expression is not sampled at the clock; if it becomes true at any time during an attempt, the attempt is disabled.

Q4. What is the difference between `disable iff (!rst_n) a |=> b` and `(rst_n && a) |=> b`? The first also cancels an attempt that is already running when reset is asserted. The second only stops new attempts from starting; a running attempt continues and can fail during reset.

Q5. How do you apply the same disable condition to all assertions in an interface? With `default disable iff (!rst_n);`.

5.6 Assertion Pass and Failure Behaviour

The possible outcomes of an attempt

Outcome	When	Action block executed
Success (non-vacuous pass)	The antecedent matched and the consequent held	Pass statement
Vacuous success	The antecedent did not match	Pass statement, unless vacuous passes are switched off (see below)
Failure	The antecedent matched and the consequent did not hold; or a property without implication was false	Fail statement; if there is none, the default `$error`
Disabled	The `disable iff` condition was true	None
Incomplete (pending)	The simulation ended while the attempt was still waiting	None; some tools list unfinished attempts in the report

Action blocks

```
a_req_ack: assert property (@(posedge clk) disable iff (!rst_n) req |=> ack)
  pass_count++;                                          // pass statement (optional)
else begin                                               // fail statement (optional)
  fail_count++;
  $error("[%0t] ack missing after req (req=%b ack=%b)",
         $time, $sampled(req), $sampled(ack));
end
```

Fact	Detail
When action blocks run	In the **Reactive** region of the time step in which the attempt finished
Values seen in action blocks	Current values, after the design has updated. Use `$sampled(sig)` for the values the assertion used
No fail statement	The simulator reports an error (`$error`) with a tool-specific message naming the assertion, the file and line, and the time
`else $fatal(...)`	The simulation stops at the first failure
After a failure	The assertion remains active: new attempts continue at every tick, so one design bug can produce many failure messages
Pass statement	Rarely used. Printing in it floods the log, because most attempts are (vacuous) passes
`cover property`	Has one optional statement, executed when the property is covered. It has no `else`

What a simulator reports for a failure

With no `else` clause, a failure produces a message of this general form (the wording differs between tools):

```
Error: "<file>", line <n>: <hierarchical assertion name>: started at <t1> failed at <t2>
       Offending '<expression>'
```

The two times are important: the **start** time is the tick at which the antecedent began to match, and the **failure** time is the tick at which the consequent was found false. Debugging begins at the start time.

At the end of simulation the tools also provide an assertion summary and assertion coverage: for each assertion, the number of attempts, real passes, vacuous passes, failures and incomplete attempts.

Controlling assertions

System task	Effect
`$assertoff;`	Stops new attempts of all assertions; attempts already running continue
`$asserton;`	Re-enables assertions
`$assertkill;`	Stops new attempts and also kills attempts in progress
`$assertoff(0, tb_top.dut);`	The same, limited to a hierarchy (first argument: number of levels, 0 = all levels below)
`$assertpassoff;` / `$assertpasson;`	Disable / enable execution of pass action blocks
`$assertfailoff;` / `$assertfailon;`	Disable / enable execution of fail action blocks
`$assertvacuousoff;`	Do not execute pass action blocks for vacuous successes
`$assertcontrol(...)`	General form that covers all of the above (IEEE 1800-2012)

A typical use is to switch assertions off while the DUT is deliberately driven with illegal stimulus (error-injection tests), or during the reset phase of a test: `$assertoff; apply_reset(); $asserton;`.

Counting failures for the test verdict

```systemverilog
// in the interface
int unsigned sva_errors;

a_valid_out: assert property (valid_in |=> valid_out)
  else begin
    sva_errors++;
    $error("valid_out did not follow valid_in");
  end
```

At the end of the test, the test class reads `vif.sva_errors` together with the scoreboard's error count and prints the final PASS or FAIL. In practice `$error` is enough: every simulator counts error-severity messages, and the regression script fails the test if the count is not zero.

> **Interview trap:** "Does the simulation stop when an assertion fails?" No. By default a failing assertion issues an error and the simulation continues. It stops only if the fail statement calls `$fatal` or `$finish`, or if the simulator is configured to stop on errors.

> **Interview trap:** "Is the pass statement executed on a vacuous success?" By the standard, yes: a vacuous success is a success, unless `$assertvacuousoff` has been called. Tools differ in their default settings, so do not rely on a counter incremented in a pass statement to prove that an assertion was really exercised. Use `cover property` on the antecedent, or the tool's assertion coverage.

> **Common mistake:** Writing an action block that prints signal values without `$sampled`, then spending an hour on a message that "contradicts the waveform".

Interview questions

Q1. What happens by default when a concurrent assertion fails? The simulator issues a run-time error (as if `else $error(...)` had been written), reporting the assertion name, location and time. The simulation continues.

Q2. What are action blocks? The optional statements executed when an assertion passes (pass statement) or fails (the statement after `else`).

Q3. In which region do action blocks execute? The Reactive region.

Q4. How do you make the simulation stop at the first assertion failure? Call `$fatal` in the fail statement, or use a simulator option that stops on assertion failure.

Q5. How do you disable assertions during reset or error injection? With `disable iff` in the property, or procedurally with `$assertoff` and `$asserton` (optionally for a specific hierarchy).

Q6. What is the difference between `$assertoff` and `$assertkill`? `$assertoff` prevents new attempts but lets attempts in progress finish. `$assertkill` also terminates attempts in progress.

Q7. What information does assertion coverage give? For each assertion: how many attempts were made, how many passed non-vacuously, how many passed vacuously, how many failed, and how many were incomplete. An assertion with no non-vacuous pass was never really tested.

5.7 Sequence Operators

The implication, delay and repetition operators cover most assertions. The operators in this section combine whole sequences. For a fresher interview you should know what each one means and recognise it in code.

Overview

Operator	Form	Meaning
`and`	`s1 and s2`	Both sequences match, starting at the same tick. They may end at different ticks; the result ends when the **later** one ends
`intersect`	`s1 intersect s2`	Both sequences match, starting at the same tick **and ending at the same tick** (same length)
`or`	`s1 or s2`	At least one of the sequences matches
`throughout`	`expr throughout s`	The Boolean `expr` is true at **every** tick from the start to the end of `s`
`within`	`s1 within s2`	`s1` matches somewhere inside the span of a match of `s2` (starts no earlier, ends no later)

Operator	Form	Meaning
`first_match`	`first_match(s)`	Of all the possible matches of `s`, only the earliest-ending one is used
`.triggered`	`s.triggered`	A Boolean that is true in the time step in which sequence `s` reached its end point. Used to test, in another sequence or in procedural code, that a sequence has just completed
`.matched`	`s.matched`	Like `.triggered`, for a sequence on a **different clock**: true at the first tick of the destination clock after `s` ended

Timelines

```
tick              1    2    3    4    5
a                 1    0    0    0    0
b                 0    0    1    0    0
c                 0    0    0    0    1
en                1    1    1    1    0

s1 = a ##2 b              matches ticks 1..3
s2 = a ##4 c              matches ticks 1..5

s1 and s2                 both match from tick 1            -> match, ends at tick 5
s1 intersect s2           lengths differ (3 and 5 ticks)    -> NO match
s1 or s2                  at least one matches              -> match (ends at 3, and at 5)
en throughout s1          en = 1 at ticks 1, 2, 3           -> match
en throughout s2          en = 0 at tick 5                  -> NO match
(b) within s2             b occurs at tick 3, inside 1..5   -> match
```

Typical uses

```
// chip select must stay low for the whole transfer
a_cs_low: assert property (@(posedge clk)
  $fell(cs_n) |-> (!cs_n throughout (##[1:8] done)));

// a burst of exactly 4 beats: 'last' on the 4th valid beat, whole burst within 10 cycles
sequence s_four_beats;  valid [->4] ##0 last;  endsequence
a_burst: assert property (@(posedge clk)
  $rose(start) |=> (s_four_beats intersect (1'b1 [*1:10])));

// the response is either an acknowledge or an error
a_resp: assert property (@(posedge clk)
  req |=> (ack ##1 !ack) or (err ##1 !err));

// an interrupt pulse occurs somewhere inside the busy window
c_irq_in_busy: cover property (@(posedge clk)
  ($rose(irq) ##1 $fell(irq)) within ($rose(busy) ##[1:$] $fell(busy)));

// only the first grant after a request matters
a_first: assert property (@(posedge clk)
  first_match(req ##[1:5] gnt) |-> ##1 !gnt);
```

`intersect` with `1'b1 [*m:n]` is the standard idiom for "this sequence must take between m and n ticks": the constant-true sequence matches any m to n ticks, and `intersect` forces the two to have the same length.

and versus intersect

Property	`and`	`intersect`
Both sequences must match	Yes	Yes
Same start tick	Yes	Yes

Property	`and`	`intersect`
Same end tick	Not required	**Required**
End of the combined match	The later of the two ends	The common end
Typical use	Two independent requirements after a trigger	Limiting the length of a sequence; two views of the same window

Why first_match matters

A sequence with a range has several possible matches. When such a sequence is the **antecedent** of an implication, the consequent must hold for **every** match:

```
(req ##[1:3] gnt) |-> ##1 done

If gnt is high at 1, 2 and 3 ticks after req, the antecedent matches three times,
and done is required after each of them.

first_match(req ##[1:3] gnt) |-> ##1 done        only the first gnt counts
```

Local variables (awareness level)

A sequence or property can declare a variable, capture a value at one tick and compare it at a later tick. This is how assertions check **data** across cycles.

```
// The ALU registers its result: the sum computed from the inputs at the tick of valid_in
// must appear on result at the next tick.
property p_add_result;
  logic [8:0] exp;
  @(posedge clk) disable iff (!rst_n)
    (valid_in && op == ALU_ADD, exp = a + b) |=> (result == exp);
endproperty
a_add_result: assert property (p_add_result);
```

`(expression, variable = value)` means: when the expression is true, also perform the assignment. Each evaluation attempt has its **own copy** of the local variable, so overlapping attempts do not disturb each other. That is something a module-level variable cannot provide.

Other property operators (awareness level)

Operator	Meaning
`not p`	The property `p` must **not** hold. `not (a ##1 b)` means "`a` followed by `b` never happens"
`p1 and p2`, `p1 or p2`	Both / at least one property hold
`if (c) p1 else p2`	Select the property to check by a condition
`p1 until p2`, `s_until`, `until_with`	`p1` holds at every tick until `p2` holds (IEEE 1800-2009)
`nexttime p`, `always p`, `s_eventually p`	Temporal-logic operators (IEEE 1800-2009), mainly used in formal verification
`strong(s)`, `weak(s)`	Whether an unfinished sequence at the end of simulation counts as a failure (strong) or not (weak)

Tool support for the 2009 operators in simulation is good in the major simulators but not universal; `##`, repetition and implication are supported everywhere.

> **Interview trap:** "What is the difference between `and` and `intersect`?" Both need both sequences to match from the same starting tick. `intersect` additionally requires them to end at the same tick.

> **Common mistake:** Using `throughout` with a sequence on the left. The left operand of `throughout` must be a Boolean expression; the right operand is the sequence.

> **Common mistake:** Checking data over several cycles with a module-level register written by the assertion's action block. Overlapping transactions overwrite it. Use a local variable in the property.

Interview questions

Q1. What does `throughout` do? `expr throughout seq` requires the Boolean expression to be true at every clock tick during the match of the sequence.

Q2. What is the difference between `and` and `intersect` for sequences? `and` requires both sequences to match starting at the same tick, with any end ticks. `intersect` requires the same start tick and the same end tick.

Q3. What does `within` do? `s1 within s2` matches when `s1` matches entirely inside the time span of a match of `s2`.

Q4. What is `first_match` and when is it needed? It keeps only the first (earliest-ending) match of a sequence that could match several times, typically because of a delay range. It is needed when such a sequence is used as an antecedent and the consequent should be checked only once.

Q5. What are local variables in assertions used for? To capture a data value at one point of a sequence and compare it at a later point, with a separate copy for each evaluation attempt. They make data-integrity checks across cycles possible, for example "the data written is the data read later".

Q6. What is `.triggered`? A method of a named sequence that returns true in the time step at which the sequence reached its end. It lets another property, or procedural code such as `wait (s.triggered)`, react to the completion of a sequence.

Q7. How do you write "sequence `s` must complete within 3 to 6 cycles"? `s intersect (1'b1 [*3:6])`.

5.8 Where Assertions Live, bind, and Assertions vs Scoreboard

The possible locations

Location	Who uses it	Advantages	Disadvantages
Inline in the RTL module	Designers	Next to the logic it checks; can see every internal signal	Mixes verification code with design code; must be excluded from synthesis (most tools ignore assertions, or the code is wrapped in `` `ifdef ``); RTL files change when assertions change

Location	Who uses it	Advantages	Disadvantages
In the **interface**	Verification engineers	The natural home for protocol rules; reused wherever the interface is used; visible to the testbench through the virtual interface	Sees only the interface signals, not DUT internals
In a separate **checker module** connected with `bind`	Both	The RTL file is not touched; assertions are kept, reviewed and versioned separately; can still see internal signals	One more file and a port list to maintain
In a `checker` ... `endchecker` construct	Advanced users, formal	A construct designed for packaging assertions, with free variables for formal	Uneven simulator support
In a class	Not possible for concurrent assertions		Immediate assertions only

bind

In plain English. `bind` attaches a module full of assertions to a design module **from the outside**. The simulator behaves as if the assertion module had been instantiated inside the design module, but the design source file is not edited.

Technical definition. The `bind` directive instantiates a module, interface, program or checker inside a target module (or inside specific instances of it) without modifying the target's source. The port connections of the bound instance are resolved **in the scope of the target**, so they can name the target's ports and its internal signals.

Syntax

```
// bind to EVERY instance of a module
bind <target_module>  <checker_module>  <instance_name> ( <port connections> );

// bind to ONE specific instance
bind <hierarchical.path.to.instance>  <checker_module>  <instance_name> ( <connections> );
```

```
without bind                              with bind
------------                              ---------
module alu (...);                         module alu (...);       <- unchanged RTL file
  ...                                       ...
  alu_sva u_sva (.*);  <- RTL edited      endmodule
endmodule
                                          // in a separate file (or in tb_top):
                                          bind alu alu_sva u_alu_sva (.*);

Resulting hierarchy in both cases:   tb_top.dut.u_alu_sva
```

Example

```
// checker module: all ports are inputs
module alu_sva
  import alu_pkg::*;
(
  input logic       clk, rst_n, valid_in, valid_out,
  input logic [7:0] a, b,
  input alu_op_e    op,
  input logic [8:0] result
);
  default clocking cb @(posedge clk); endclocking
  default disable iff (!rst_n);

  a_valid_follow: assert property (valid_in |=> valid_out);
endmodule

// bind statement: connect by name to the signals of module alu
bind alu alu_sva u_alu_sva (.*);
```

Point	Explanation
All ports of the checker are inputs	A checker only observes
`bind alu ...`	Every instance of `alu` in the design receives its own copy of the checker
`(.*)`	Each checker port is connected to the signal of the same name inside `alu`. Named connections (`.count(count)`) can reach internal signals that are not ports
Where the `bind` statement goes	In a separate file compiled with the testbench, or inside `tb_top`
Parameters	`bind sync_fifo sync_fifo_sva #(.DEPTH(DEPTH)) u_sva (...);` passes the target's parameter to the checker

Section 6 uses `bind` in all four practical examples.

Assertions versus the scoreboard

Aspect	Assertions (SVA)	Scoreboard
Looks at	Signals, cycle by cycle	Transactions, one per operation
Best at	Temporal and protocol rules: handshakes, stability, timing windows, illegal signal combinations, X checks	Data correctness: is the result what the reference model predicts? Ordering, loss or duplication of data
Reference	The rule written in the property	A reference model or expected-data queue
Time of detection	At the clock edge where the rule is broken	When the transaction completes, possibly much later
Where it lives	Interface, DUT (via `bind`), checker module	A class in the testbench environment
Language	Declarative: sequences and properties	Procedural, object-oriented
Reuse in formal verification	Yes	No
Weak at	Complex data transformations, long-lived state	Cycle-accurate protocol timing

Aspect	Assertions (SVA)	Scoreboard
Example for the ALU	"`valid_out` is high exactly one cycle after `valid_in`"; "`result` is never X when `valid_out` is high"	"For `a`=200, `b`=100, `op`=ADD the result must be 300"

Covergroup versus cover property

Aspect	`covergroup`	`cover property`
Covers	Values, ranges, combinations (cross), value transitions of sampled data	Sequences of events on signals over clock cycles
Sampling	Event or `sample()` call	Every clock tick
Where	Classes, modules, interfaces	Modules, interfaces, checkers (not classes)
Granularity of result	Bins with hit counts, percentages	One hit count per cover statement
Typical use	Transaction-level coverage in the monitor path	Protocol scenarios: back-to-back transfers, stalls, full-to-empty

Key point: Use the scoreboard for **what** the DUT computed, assertions for **how and when** the signals behaved, covergroups for **which data** was exercised and cover properties for **which signal scenarios** occurred.

Interview trap: "Can assertions replace the scoreboard?" No. Simple data checks can be written with `$past` or local variables, as in the ALU example, but a general check of data against a reference model, with queues, out-of-order responses or complex algorithms, is procedural work that belongs in a scoreboard. The opposite is also true: a scoreboard does not notice a protocol violation that happens not to corrupt the data.

Common mistake: Putting the `bind` statement inside the target module's own file. The point of `bind` is that the design file stays untouched.

Interview questions

Q1. Where can concurrent assertions be placed? In modules, interfaces, programs and checkers: inline in the RTL, in the testbench interface, or in a separate checker module attached to the DUT with `bind`. They cannot be placed in classes.

Q2. What is `bind`? A directive that instantiates a module (typically one containing assertions and coverage) inside a target module or instance without modifying the target's source code. The bound instance can connect to the target's ports and internal signals.

Q3. Why use `bind` instead of writing assertions in the RTL? The design files stay unchanged and free of verification code; assertions can be developed, reviewed and switched on or off independently; the same checker can be bound to RTL from another team or to a gate-level netlist (for port-level checks).

Q4. How do you bind to only one instance of a module? Use the hierarchical instance path as the target: `bind tb_top.dut.u_fifo0 sync_fifo_sva u_sva (...);`.

Q5. What is the difference between an assertion and a scoreboard check? An assertion checks signal-level, cycle-accurate rules continuously and reports at the moment of violation. A scoreboard compares completed transactions against expected data from a reference model. They find different kinds of bugs.

Q6. What is the difference between `cover property` and a covergroup? A covergroup records values, combinations and transitions of data at sampling points. `cover property` records that a temporal sequence of signal behaviour occurred. Covergroups can be used in classes; cover properties cannot.

Part 6: Practical Assertion Sets

This part applies Parts 4 and 5 to four small designs: a synchronous FIFO, a valid/ready handshake, a counter and the course ALU. Each example follows the same method, which is also the method to describe in an interview when asked "how would you write assertions for this block?".

The method

1. Read the specification and write each rule as one English sentence.
2. Classify each rule: reset behaviour, combinational (same-cycle) relationship, next-cycle behaviour, stability, timing window, "never" condition, X check.
3. Translate each sentence into one property. One rule, one assertion, one clear label.
4. Decide which rules are about the **DUT** (assert) and which are about how the **environment** must drive it (assume, in formal terms).
5. Add `cover property` statements for the scenarios that prove the assertions were exercised.

English pattern	Assertion pattern
"X and Y are never both true"	`!(x && y)`
"When X, Y must be true in the same cycle"	`x |-> y`
"After X, Y must be true in the next cycle"	`x |=> y`
"After X, Y within N cycles"	`x |-> ##[1:N] y`
"While X, signal S must not change"	`x |=> $stable(s)`
"S changes only when X"	`$changed(s) |-> x`
"After reset, S has value V"	`!rst_n |-> s == v` (not disabled by reset)
"S is never unknown"	`!$isunknown(s)`
"The new value is the old value plus one"	`x |=> s == $past(s) + 1'b1` (mind the width)

All four checkers use the same header:

```
default clocking cb @(posedge clk); endclocking
default disable iff (!rst_n);
```

and all testbenches use the timing scheme of this module: the clock rises at 5, 15, 25 ..., and stimulus changes at falling edges.

6.1 Assertions for a Synchronous FIFO

The DUT

`sync_fifo` is an 8-bit wide, 16-entry synchronous FIFO. A write is accepted on a rising clock edge when `wr_en` is 1 and the FIFO is not full. A read is accepted when `rd_en` is 1 and the FIFO is not empty; the data appears on `dout` after that edge. Writes when full and reads when empty are ignored.

sync_fifo.sv

```
module sync_fifo #(
  parameter int WIDTH = 8,
  parameter int DEPTH = 16
) (
  input  logic             clk,
  input  logic             rst_n,
  input  logic             wr_en,
  input  logic             rd_en,
  input  logic [WIDTH-1:0] din,
  output logic [WIDTH-1:0] dout,
  output logic             full,
  output logic             empty
);

  localparam int AW = $clog2(DEPTH);

  logic [WIDTH-1:0] mem [DEPTH];
  logic [AW-1:0]    wr_ptr, rd_ptr;
  logic [AW:0]      count;              // 0 .. DEPTH
  logic             do_wr, do_rd;

  assign full  = (count == DEPTH);
  assign empty = (count == 0);
  assign do_wr = wr_en && !full;
  assign do_rd = rd_en && !empty;

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      wr_ptr <= '0;
      rd_ptr <= '0;
      count  <= '0;
      dout   <= '0;
    end
    else begin
      if (do_wr) begin
        mem[wr_ptr] <= din;
        wr_ptr      <= wr_ptr + 1'b1;
      end
      if (do_rd) begin
        dout   <= mem[rd_ptr];
        rd_ptr <= rd_ptr + 1'b1;
      end
      if (do_wr && !do_rd)      count <= count + 1'b1;
      else if (do_rd && !do_wr) count <= count - 1'b1;
    end
  end

endmodule
```

The rules

#	Rule in English	Kind	Label
1	During reset the FIFO is empty, not full, and holds zero entries	Reset	`a_reset_state`
2	The FIFO is never full and empty at the same time	Never	`a_not_full_and_empty`
3	The entry count never exceeds the depth	Never	`a_count_range`
4	`full` is high exactly when the count equals the depth; `empty` exactly when it is zero	Same cycle	`a_full_flag`, `a_empty_flag`
5	Control and status signals are never unknown; `din` is known when written	X check	`a_ctrl_known`, `a_din_known`
6	A write alone increases the count by one	Next cycle	`a_wr_incr`

#	Rule in English	Kind	Label
7	A read alone decreases the count by one	Next cycle	`a_rd_decr`
8	A simultaneous accepted write and read leaves the count unchanged	Next cycle	`a_rw_same`
9	With no write and no read, the count does not change	Stability	`a_idle_stable`
10	A write to a full FIFO (with no read) is ignored: it stays full	Next cycle	`a_full_hold`
11	A read from an empty FIFO (with no write) is ignored: it stays empty	Next cycle	`a_empty_hold`
12	`full` can only fall because of a read; `empty` can only fall because of a write	Cause	`a_full_fell`, `a_empty_fell`
13	The environment should not write when full or read when empty	Environment	`a_wr_when_full`, `a_rd_when_empty`

The checker

sync_fifo_sva.sv

```
module sync_fifo_sva #(
  parameter int DEPTH = 16
) (
  input logic                 clk,
  input logic                 rst_n,
  input logic                 wr_en,
  input logic                 rd_en,
  input logic [7:0]           din,
  input logic                 full,
  input logic                 empty,
  input logic [$clog2(DEPTH):0] count       // internal signal of the FIFO
);

  default clocking cb @(posedge clk); endclocking
  default disable iff (!rst_n);

  // ---------------- reset (must NOT be disabled by reset) ----------------
  a_reset_state: assert property (disable iff (1'b0)
    !rst_n |-> (empty && !full && count == 0));

  // ---------------- invariants ----------------
  a_not_full_and_empty: assert property (!(full && empty));
  a_count_range       : assert property (count <= DEPTH);
  a_full_flag         : assert property (full  == (count == DEPTH));
  a_empty_flag        : assert property (empty == (count == 0));

  // ---------------- no unknown values ----------------
  a_ctrl_known: assert property (!$isunknown({wr_en, rd_en, full, empty}));
  a_din_known : assert property (wr_en |-> !$isunknown(din));

  // ---------------- counting ----------------
  a_wr_incr: assert property (
    (wr_en && !full && !(rd_en && !empty)) |=> count == $past(count) + 1);

  a_rd_decr: assert property (
    (rd_en && !empty && !(wr_en && !full)) |=> count == $past(count) - 1);

  a_rw_same: assert property (
    (wr_en && !full && rd_en && !empty) |=> $stable(count));

  a_idle_stable: assert property ((!wr_en && !rd_en) |=> $stable(count));

  // ---------------- overflow / underflow protection ----------------
  a_full_hold : assert property ((full  && wr_en && !rd_en) |=> full);
  a_empty_hold: assert property ((empty && rd_en && !wr_en) |=> empty);
```

```
  // ---------------- cause of flag changes ----------------
  a_full_fell : assert property ($fell(full)  |-> $past(rd_en));
  a_empty_fell: assert property ($fell(empty) |-> $past(wr_en));

  // ---------------- rules for the environment ----------------
  a_wr_when_full: assert property (wr_en |-> !full)
    else $display("[%0t] a_wr_when_full FAILED: write attempted while the FIFO is full",
                  $time);

  a_rd_when_empty: assert property (rd_en |-> !empty)
    else $display("[%0t] a_rd_when_empty FAILED: read attempted while the FIFO is empty",
                  $time);

  // ---------------- coverage of interesting scenarios ----------------
  c_became_full  : cover property ($rose(full));
  c_full_to_empty: cover property (full ##[1:$] empty);
  c_simul_rw     : cover property (wr_en && rd_en && !full && !empty);

endmodule
```

Line by line

Assertion	Explanation
`a_reset_state`	Checks the reset values. It overrides the default with `disable iff (1'b0)`, otherwise it would be disabled exactly when it should check.
`a_not_full_and_empty`	A pure Boolean property: checked at every clock tick.
`a_full_flag`	`full == (count == DEPTH)` compares two 1-bit values: the flag must agree with the count in both directions.
`a_ctrl_known`	Concatenates four signals and checks them together.
`a_din_known`	Data is checked only when qualified by `wr_en`.
`a_wr_incr`	Antecedent: a write is accepted and no read is accepted. Consequent, one tick later: the count is the previous count plus one. `count` is 5 bits and never exceeds 16, so the 32-bit addition `$past(count) + 1` cannot wrap and the comparison is safe here. (Section 6.3 shows a case where it is not.)
`a_rd_decr`	The mirror image. The antecedent contains `!empty`, so `$past(count)` is at least 1 and the subtraction cannot go negative.
`a_rw_same`	Simultaneous accepted write and read.
`a_full_hold`	A write when full, with no read, must leave the FIFO full: the overflow is ignored.
`a_full_fell`	Looks backwards: if `full` has just fallen, there must have been a read request in the previous cycle.
`a_wr_when_full`, `a_rd_when_empty`	Rules about the stimulus, not about the FIFO. In simulation they flag a testbench or upstream-block error. In formal verification they would be `assume property`. They are the only assertions here with a custom fail statement, so that the example has exact output.
`c_became_full`	Proves that the FIFO was filled at least once, so the full-related assertions were not vacuous.
`c_full_to_empty`	A cover with an unbounded delay is fine: it is hit when `empty` eventually follows `full`.

The testbench

tb_fifo_sva.sv

```
// compile order: sync_fifo.sv  sync_fifo_sva.sv  tb_fifo_sva.sv
module tb_fifo_sva;

  logic       clk   = 0;
  logic       rst_n = 1;
  logic       wr_en = 0;
  logic       rd_en = 0;
  logic [7:0] din   = '0;
  logic [7:0] dout;
  logic       full, empty;

  sync_fifo dut (.*);

  // attach the checker to every instance of sync_fifo; 'count' is internal to the FIFO
  bind sync_fifo sync_fifo_sva #(.DEPTH(DEPTH)) u_sync_fifo_sva (
    .clk(clk), .rst_n(rst_n), .wr_en(wr_en), .rd_en(rd_en), .din(din),
    .full(full), .empty(empty), .count(count)
  );

  always #5 clk = ~clk;

  initial begin
    #1 rst_n = 0;                       // assert reset at time 1
    @(negedge clk);  rst_n = 1;         // release at time 10

    // 17 writes into a 16-entry FIFO: the last one is attempted while full
    for (int i = 0; i < 17; i++) begin
      wr_en = 1;
      din   = 8'hA0 + i;
      @(negedge clk);
    end
    wr_en = 0;

    rd_en = 1;                          // one read
    @(negedge clk);
    rd_en = 0;
    @(negedge clk);

    $display("[%0t] full=%b empty=%b dout=%h", $time, full, empty, dout);
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[175] a_wr_when_full FAILED: write attempted while the FIFO is full
[200] full=0 empty=0 dout=a0
```

Execution flow

Time	Event
1	Reset asserted. The FIFO's count becomes 0 asynchronously.
5	First clock tick, in reset: `a_reset_state` checks `empty`, `!full`, `count == 0`. All other assertions are disabled.
10	Reset released. The first write (`A0`) is driven.
15, 25 ... 165	Sixteen writes are accepted. After the edge at 165 the count is 16 and `full` rises. At each of these ticks `a_wr_incr` starts an attempt that passes at the next tick. `a_empty_fell` passes at time 25.
170	The seventeenth write is driven although `full` is 1.

Time	Event
175	The tick samples `wr_en` = 1 and `full` = 1. **`a_wr_when_full` fails.** The FIFO ignores the write; `a_full_hold` starts an attempt.
185	`rd_en` = 1 is sampled: the first entry (`A0`) is read, the count drops to 15. `a_full_hold` passes (the FIFO was still full before this edge).
195	`a_rd_decr` passes (15 = 16 - 1). `a_full_fell` passes (`rd_en` was 1 at the previous tick).
200	The testbench prints the state: not full, not empty, `dout` = `a0` .

The design is correct, so none of the DUT assertions fails. The one failure comes from the environment rule, and it points at the exact cycle in which the testbench misbehaved.

Points to notice

Code	Explanation
`#1 rst_n = 0;`	Reset is asserted a little after time 0, so that the falling edge of `rst_n` is certain to be seen by the `always_ff` block. Driving it at time 0 would race with the start of the block.
`bind sync_fifo sync_fifo_sva #(.DEPTH(DEPTH)) ...`	The port connections and the parameter are resolved **inside** `sync_fifo` . That is how the checker reaches the internal signal `count` and the FIFO's own `DEPTH` .
The `bind` statement is inside `tb_fifo_sva`	It could equally be in a file of its own, compiled with the testbench.
No data check	The assertions verify flags and counting. That each word comes out unchanged and in order is checked by a scoreboard with a queue as reference model (Module 9). It can be written as an assertion with local variables, but the scoreboard is simpler and clearer.

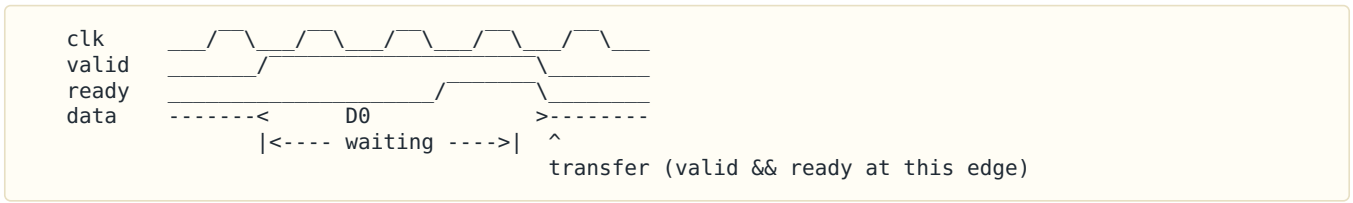
Interview trap: "Write an assertion that a FIFO never overflows." Be precise about what is meant. If the **design** must be safe, the property is "a write when full does not change the state": `full && wr_en && ! rd_en |=> full` (and the count is stable). If the **environment** must never try, the property is `wr_en |-> ! full` . Interviewers like candidates who ask which one is wanted.

Verification application: FIFO checkers like this one are bound to every FIFO instance in a chip. A system-level test that overflows an internal FIFO then fails immediately at the FIFO, instead of thousands of cycles later at a scoreboard far downstream.

6.2 Assertions for a Valid/Ready Handshake

The protocol

A sender presents `data` and raises `valid` . The receiver raises `ready` when it can accept. A **transfer** happens on a rising clock edge at which `valid` and `ready` are both 1. This is the handshake used by AXI and many other protocols.



The rules

#	Rule in English	Label
1	During reset, `valid` is low	`a_reset_valid`
2	Once `valid` is raised it must stay high until the transfer happens (the sender may not withdraw a request)	`a_valid_hold`
3	While `valid` is high and `ready` is low, `data` must not change	`a_data_stable`
4	`valid` and `ready` are never unknown	`a_ctrl_known`
5	`data` is not unknown while `valid` is high	`a_data_known`
6	After `valid` rises, `ready` must come within a limited number of cycles (here: the same cycle or up to 3 cycles later)	`a_ready_timeout`

The checker

handshake_sva.sv

```
module handshake_sva #(
  parameter int MAX_WAIT = 3
) (
  input logic       clk,
  input logic       rst_n,
  input logic       valid,
  input logic       ready,
  input logic [7:0] data
);

  default clocking cb @(posedge clk); endclocking
  default disable iff (!rst_n);

  a_reset_valid: assert property (disable iff (1'b0) !rst_n |-> !valid);

  a_valid_hold: assert property (valid && !ready |=> valid)
    else $display("[%0t] a_valid_hold    FAILED: valid dropped before ready", $time);

  a_data_stable: assert property (valid && !ready |=> $stable(data))
    else $display("[%0t] a_data_stable   FAILED: data changed while waiting for ready",
                  $time);

  a_ready_timeout: assert property ($rose(valid) |-> ##[0:MAX_WAIT] ready)
    else $display("[%0t] a_ready_timeout FAILED: no ready within %0d cycles",
                  $time, MAX_WAIT);

  a_ctrl_known: assert property (!$isunknown({valid, ready}));
  a_data_known: assert property (valid |-> !$isunknown(data));

  // scenarios worth seeing
  c_transfer    : cover property (valid && ready);
  c_stall       : cover property (valid && !ready);
  c_back_to_back: cover property ((valid && ready) [*2]);

endmodule
```

The testbench

The stimulus deliberately breaks three rules, at three different times.

tb_handshake_sva.sv

```
// compile order: handshake_sva.sv  tb_handshake_sva.sv
module tb_handshake_sva;

  logic       clk = 0;
  logic       rst_n, valid, ready;
  logic [7:0] data;

  // stimulus for cycles 0..11 (cycle k is the posedge at time 10k+5)
  bit [0:11]  rst_v   = 12'b0111_1111_1111;
  bit [0:11]  valid_v = 12'b0111_0100_1110;
  bit [0:11]  ready_v = 12'b0001_0000_0110;
  logic [7:0] data_v [12] = '{8'h00, 8'hA1, 8'hA1, 8'hA1, 8'h00, 8'hB2,
                              8'hB2, 8'h00, 8'hC3, 8'hC4, 8'hC4, 8'h00};

  handshake_sva u_sva (.*);

  always #5 clk = ~clk;

  initial begin
    for (int i = 0; i < 12; i++) begin
      rst_n = rst_v[i];
      valid = valid_v[i];
      ready = ready_v[i];
      data  = data_v[i];
      @(negedge clk);
    end
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[65] a_valid_hold    FAILED: valid dropped before ready
[85] a_ready_timeout FAILED: no ready within 3 cycles
[95] a_data_stable   FAILED: data changed while waiting for ready
```

Stimulus

cycle	0	1	2	3	4	5	6	7	8	9	10	11
time	5	15	25	35	45	55	65	75	85	95	105	115
rst_n	0	1	1	1	1	1	1	1	1	1	1	1
valid	0	1	1	1	0	1	0	0	1	1	1	0
ready	0	0	0	1	0	0	0	0	0	1	1	0
data	00	A1	A1	A1	00	B2	B2	00	C3	C4	C4	00

Evaluation

Cycles	What happens	Result
1 to 3	`valid` rises at cycle 1 and waits; `data` holds `A1`; `ready` arrives at cycle 3	A legal transfer. `a_valid_hold` and `a_data_stable` pass at cycles 2 and 3; `a_ready_timeout` passes at cycle 3
5 to 6	`valid` rises at cycle 5 with `ready` low, then **drops at cycle 6** without a transfer	`a_valid_hold` fails at cycle 6 (time 65). `data` is still `B2` at cycle 6, so `a_data_stable` passes
5 to 8	After the rise of `valid` at cycle 5, `ready` stays low in cycles 5, 6, 7 and 8	`a_ready_timeout` fails at cycle 8 (time 85), the end of its window
8 to 9	`valid` rises at cycle 8 with `data` = `C3`; at cycle 9 `data` has **changed to** `C4` while still waiting	`a_data_stable` fails at cycle 9 (time 95). `ready` is high at cycle 9, so the timeout check for this request passes

Cycles	What happens	Result
9 to 10	Two transfers in consecutive cycles	`c_back_to_back` is covered

Points to notice

Code	Explanation
`valid && !ready |=> valid`	The antecedent is "waiting". The consequent only requires `valid` to still be high in the next cycle. Applied at every waiting cycle, this forces `valid` to stay high for the whole wait.
`valid && !ready |=> $stable(data)`	The non-overlapped operator is essential: the stability is required in the cycle **after** a waiting cycle.
`$rose(valid) |-> ##[0:MAX_WAIT] ready`	`##[0:...]` includes the same cycle, because `ready` may already be high when `valid` rises. `$rose` gives one attempt per request. The bound uses a parameter, which is legal because parameters are constants.
`handshake_sva u_sva (.*);`	Here the checker is instantiated directly in the testbench, because there is no DUT module to bind to. In a real environment it would be placed in, or bound to, the interface.

Interview trap: "Can `ready` depend on `valid`?" In AXI, a receiver **may** wait for `valid` before raising `ready`, but a sender must **not** wait for `ready` before raising `valid`. So "`valid` must not depend on `ready`" is a rule, and it cannot be fully checked by a simple simulation assertion; it is a design-review and formal-verification item. Knowing which rules assertions can and cannot check is a mark of experience.

Common mistake: Writing rule 2 as `$rose(valid) |-> valid until ready` without checking tool support, or as `valid |=> valid`, which forbids `valid` from ever falling. The waiting condition `valid && !ready` must be in the antecedent.

Verification application: These six assertions, parameterised by data width and timeout, are instantiated on every valid/ready channel of a design. For the five channels of an AXI interface, the same properties are asserted five times with different signal arguments.

6.3 Assertions for a Simple Counter

The DUT

`counter.sv`

```
module counter (
  input  logic       clk,
  input  logic       rst_n,
  input  logic       en,
  output logic [3:0] count
);

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)  count <= '0;
    else if (en) count <= count + 1'b1;
  end

endmodule
```

The rules

#	Rule in English	Label
1	During reset the count is 0	`a_reset`
2	When `en` is 1, the count increases by one (modulo 16)	`a_incr`
3	When `en` is 0, the count does not change	`a_hold`
4	When enabled at 15, the count wraps to 0	`a_wrap`
5	The count is never unknown	`a_known`

The checker, including one deliberately wrong assertion

counter_sva.sv

```
module counter_sva (
  input logic       clk,
  input logic       rst_n,
  input logic       en,
  input logic [3:0] count
);

  default clocking cb @(posedge clk); endclocking
  default disable iff (!rst_n);

  a_reset: assert property (disable iff (1'b0) !rst_n |-> count == 4'd0);

  a_incr : assert property (en |=> count == $past(count) + 4'd1);

  a_hold : assert property (!en |=> $stable(count));

  a_wrap : assert property (en && count == 4'hF |=> count == 4'h0);

  a_known: assert property (!$isunknown(count));

  // WRONG on purpose: the literal 1 is 32 bits wide, so the sum does not wrap at 15
  a_incr_bad: assert property (en |=> count == $past(count) + 1)
    else $display("[%0t] a_incr_bad FAILED: the assertion is wrong, not the counter",
                  $time);

  c_wrap: cover property (en && count == 4'hF ##1 count == 4'h0);

endmodule
```

The testbench

tb_counter_sva.sv

```
// compile order: counter.sv  counter_sva.sv  tb_counter_sva.sv
module tb_counter_sva;

  logic       clk   = 0;
  logic       rst_n = 1;
  logic       en    = 0;
  logic [3:0] count;

  counter dut (.*);

  bind counter counter_sva u_counter_sva (.*);

  always #5 clk = ~clk;

  initial begin
    #1 rst_n = 0;
    @(negedge clk);                    // time 10
    rst_n = 1;
    en    = 1;
    repeat (17) @(negedge clk);        // 17 enabled clock edges: 15, 25, ... 175
    en = 0;                            // time 180
    repeat (2) @(negedge clk);         // time 200
    $display("[%0t] count=%0d", $time, count);
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[175] a_incr_bad FAILED: the assertion is wrong, not the counter
[200] count=1
```

Execution flow

Time	Sampled `count`	Event
5	0	Tick during reset: `a_reset` passes
15	0	First enabled tick. The count becomes 1 after the edge
25 ... 155	1 ... 14	`a_incr` passes at every tick
165	15	Enabled at 15: `a_incr`, `a_incr_bad` and `a_wrap` start attempts. The count wraps to 0 after the edge
175	0	`a_incr`: 0 == 15 + 4'd1 in 4 bits → 0 == 0: **pass**. `a_wrap`: pass. `a_incr_bad`: 0 == 15 + 1 in 32 bits → 0 == 16: **FAIL**
185	1	`en` is now 0 (changed at 180): `a_hold` starts an attempt
195	1	`a_hold` passes
200		Seventeen increments from 0, modulo 16: the count is 1

Why `a_incr_bad` fails

```
   count == $past(count) + 1            count == $past(count) + 4'd1

   operand widths:  4      4       32    operand widths:  4      4        4
   expression width = 32 bits            expression width = 4 bits

   $past(count) = 15:                    $past(count) = 15:
      15 + 1 = 16   (no wrap in 32 bits)    15 + 1 = 0    (wraps in 4 bits, as in hardware)
      count = 0  -> 0 == 16 is FALSE        count = 0  -> 0 == 0 is TRUE
```

This is the expression-width rule of Module 1 (Section 4.4) appearing in an assertion. The counter is correct; the assertion is wrong. A failing assertion does not always mean a design bug: **the assertion is code too and can itself be wrong**.

Points to notice

Code	Explanation
`bind counter counter_sva u_counter_sva (.*);`	The checker's port names equal the counter's port names, so `.*` connects everything.
`a_reset` with `disable iff (1'b0)`	The reset check must be active during reset.
`a_wrap`	Technically implied by `a_incr`, but stating the corner case explicitly documents it and gives it its own pass/fail and coverage entry.
`c_wrap`	A cover on a two-tick sequence: enabled at 15, then 0. It proves that the wrap-around was reached, without which `a_wrap` would be vacuous.

Interview trap: "Write an assertion that a counter increments by one." The expected follow-up is "what happens at the maximum value?". Mention the wrap-around and the width of the `+ 1` before you are asked.

Common mistake: Checking the increment with `|->` instead of `|=>`: `en |-> count == $past(count) + 4'd1`. In the cycle where `en` is sampled high, the counter has not yet incremented; the new value is visible one tick later.

6.4 Assertions for the ALU

The rules

The ALU of Module 1 registers its result: when `valid_in` is 1 at a rising edge, `result` is updated and `valid_out` is 1 during the next cycle.

#	Rule in English	Label
1	During reset, `valid_out` is 0 and `result` is 0	`a_reset`
2	`valid_out` is high one cycle after every `valid_in`	`a_valid_follow`
3	`valid_out` is low one cycle after every cycle without `valid_in`	`a_no_spurious_valid`
4	`result` changes only as a consequence of `valid_in`	`a_result_hold`
5	`valid_out` is never unknown; `result` is not unknown when `valid_out` is high	`a_valid_known`, `a_result_known`

#	Rule in English	Label
6	Inputs are not unknown when `valid_in` is high (rule for the environment)	`a_inputs_known`
7	An addition gives the 9-bit sum of the previous inputs	`a_add`
8	A NOT operation gives the inverted `a` with bit 8 clear	`a_not`

Rules 7 and 8 show that simple data checks can be written as assertions. Checking **all eight** operations is the scoreboard's job; an assertion per opcode would duplicate the reference model.

The DUT

`alu_pkg.sv` was listed in Section 3.5. The ALU is repeated here so that this module's examples are complete.

alu.sv

```
module alu
  import alu_pkg::*;
(
  input  logic       clk,
  input  logic       rst_n,
  input  logic       valid_in,
  input  logic [7:0] a,
  input  logic [7:0] b,
  input  alu_op_e    op,
  output logic       valid_out,
  output logic [8:0] result
);

  logic [8:0] result_nxt;

  always_comb begin
    unique case (op)
      ALU_ADD: result_nxt = a + b;
      ALU_SUB: result_nxt = a - b;
      ALU_AND: result_nxt = {1'b0, a & b};
      ALU_OR : result_nxt = {1'b0, a | b};
      ALU_XOR: result_nxt = {1'b0, a ^ b};
      ALU_NOT: result_nxt = {1'b0, ~a};
      ALU_SHL: result_nxt = {a, 1'b0};
      ALU_SHR: result_nxt = {2'b00, a[7:1]};
    endcase
  end

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      valid_out <= 1'b0;
      result    <= '0;
    end
    else begin
      valid_out <= valid_in;
      if (valid_in) result <= result_nxt;
    end
  end

endmodule
```

The checker

alu_sva.sv

```
module alu_sva
  import alu_pkg::*;
(
  input logic       clk,
  input logic       rst_n,
  input logic       valid_in,
  input logic [7:0] a,
  input logic [7:0] b,
  input alu_op_e    op,
  input logic       valid_out,
  input logic [8:0] result
);

  default clocking cb @(posedge clk); endclocking
  default disable iff (!rst_n);

  // ---------------- reset ----------------
  a_reset: assert property (disable iff (1'b0)
    !rst_n |-> (!valid_out && result == 9'd0));

  // ---------------- valid_out follows valid_in by exactly one cycle ----------------
  a_valid_follow     : assert property ( valid_in |=>  valid_out);
  a_no_spurious_valid: assert property (!valid_in |=> !valid_out);

  // ---------------- result only changes after valid_in ----------------
  a_result_hold: assert property (!valid_in |=> $stable(result));

  // ---------------- no X ----------------
  a_valid_known : assert property (!$isunknown(valid_out));
  a_result_known: assert property (valid_out |-> !$isunknown(result));
  a_inputs_known: assert property (valid_in  |-> !$isunknown({a, b, op}));

  // ---------------- two functional checks ----------------
  a_add: assert property (
    (valid_in && op == ALU_ADD) |=> result == $past(a) + $past(b));

  a_not: assert property (
    (valid_in && op == ALU_NOT) |=> result == {1'b0, ~$past(a)});

  // ---------------- scenarios ----------------
  c_carry       : cover property (valid_out && result[8]);
  c_back_to_back: cover property (valid_in ##1 valid_in);
  c_idle_gap    : cover property (valid_in ##1 !valid_in ##1 valid_in);

endmodule
```

Line by line

Assertion	Explanation
`a_valid_follow` and `a_no_spurious_valid`	Together they state that `valid_out` is exactly `valid_in` delayed by one cycle. The first catches a lost output; the second catches an output that appears from nowhere.
`a_result_hold`	If nothing was accepted, the result register must keep its value. This catches a missing enable on the register.
`a_result_known`	The X check on data is qualified by `valid_out`. After reset the result is 0, so here it would also hold unconditionally, but the qualified form is the general pattern.
`a_inputs_known`	A rule for the driver: when it presents an operation, all inputs must be driven.
`a_add`	At the tick after an accepted addition, `result` must equal the sum of the **previous** inputs. The comparison is made in 9 bits (the width of `result`), so the two 8-bit `$past` values are extended before the addition and the carry is included.

Assertion	Explanation
`a_not`	The concatenation makes the expected value 9 bits wide with bit 8 clear.
`c_carry`	Was a result with bit 8 set ever produced?
`c_back_to_back`, `c_idle_gap`	Were operations sent in consecutive cycles, and with a one-cycle gap? Both are needed to trust the two valid assertions.

The testbench

tb_alu_sva.sv

```
// compile order: alu_pkg.sv  alu.sv  alu_sva.sv  tb_alu_sva.sv
module tb_alu_sva;
  import alu_pkg::*;

  logic       clk      = 0;
  logic       rst_n    = 1;
  logic       valid_in = 0;
  logic [7:0] a        = '0;
  logic [7:0] b        = '0;
  alu_op_e    op       = ALU_ADD;
  logic       valid_out;
  logic [8:0] result;

  alu dut (.*);

  bind alu alu_sva u_alu_sva (.*);

  always #5 clk = ~clk;

  initial begin
    #1 rst_n = 0;
    @(negedge clk);  rst_n = 1;                                      // time 10

    @(negedge clk);  valid_in = 1;  a = 8'd200;  b = 8'd100;         // time 20
                     op = ALU_ADD;

    @(negedge clk);  $display("[%0t] valid_out=%b result=%0d", $time, valid_out, result);
                     a = 8'h0F;  op = ALU_NOT;                       // time 30

    @(negedge clk);  $display("[%0t] valid_out=%b result=%0d", $time, valid_out, result);
                     valid_in = 0;                                   // time 40

    @(negedge clk);  $display("[%0t] valid_out=%b result=%0d", $time, valid_out, result);
                     $finish;                                        // time 50
  end

endmodule
```

EXPECTED OUTPUT

```
[30] valid_out=1 result=300
[40] valid_out=1 result=240
[50] valid_out=0 result=240
```

No assertion fails, so the only output is that of the testbench.

Execution flow

Time	Sampled at this tick	Assertion activity
5	Reset active	`a_reset` passes. Everything else is disabled
15	`valid_in` = 0	`a_no_spurious_valid` and `a_result_hold` start attempts

Time	Sampled at this tick	Assertion activity
25	`valid_in` = 1, ADD, 200, 100	The attempts from 15 pass (`valid_out` = 0, `result` still 0). `a_valid_follow` and `a_add` start attempts. The DUT stores 300
35	`valid_in` = 1, NOT, `a` = `0F`; `valid_out` = 1, `result` = 300	`a_valid_follow` passes. `a_add` passes: 300 == 200 + 100. `a_not` starts an attempt. The DUT stores `{0, ~0F}` = `0F0` = 240
45	`valid_in` = 0; `valid_out` = 1, `result` = 240	`a_valid_follow` passes. `a_not` passes: 240 == `{1'b0, 8'hF0}`. `c_back_to_back` was covered at time 35

The same rules inside the interface

In the layered testbench the port-level rules are normally placed in `alu_if`, so that they travel with the interface:

```
interface alu_if (input logic clk);
  import alu_pkg::*;

  logic       rst_n, valid_in, valid_out;
  logic [7:0] a, b;
  alu_op_e    op;
  logic [8:0] result;

  // clocking blocks drv_cb and mon_cb, modports DUT, DRV, MON  (Module 8)

  default clocking cb_sva @(posedge clk); endclocking
  default disable iff (!rst_n);

  a_valid_follow     : assert property ( valid_in |=>  valid_out);
  a_no_spurious_valid: assert property (!valid_in |=> !valid_out);
  a_result_known     : assert property (valid_out |-> !$isunknown(result));
  a_inputs_known     : assert property (valid_in  |-> !$isunknown({a, b, op}));
endinterface
```

Placement	Used for
In `alu_if`	Rules that involve only the interface signals. Active in every testbench that uses the interface
In `alu_sva`, bound to `alu`	Rules that need internal signals, or that the design team owns

Interview trap: "The scoreboard already compares every result. Why add `a_valid_follow`?" Because the scoreboard only sees what the monitor delivers. If `valid_out` never rises, the monitor never produces a transaction, the scoreboard compares nothing, and a naive testbench reports "0 errors". The assertion catches the missing response directly. (A good scoreboard also checks, at the end of the test, that the number of received transactions equals the number sent.)

Verification application: The complete verification of the ALU therefore has four parts: the scoreboard with its reference model (all eight operations, every transaction), `alu_sva` and the interface assertions (timing and X rules at every clock), `alu_coverage` (which operations and operand classes were exercised), and the cover properties (which signal scenarios occurred). Together they answer: is it right, is it always well-behaved, and have we tried enough.

6.5 Interview Questions on Practical Assertions

Q1. Write assertions for a FIFO. State the rules first, then code them: never full and empty together (`!(full && empty)`); after reset, empty and not full; a write when full does not change the state (`full && wr_en && !rd_en |=> full`); a read when empty does not change the state; a write alone increments the count and a read alone decrements it (using `$past`); flags agree with the count; no X on the flags. Add covers for full, empty and simultaneous read/write.

Q2. Write an assertion: `valid` must remain high until `ready` is high. `assert property (@(posedge clk) disable iff (!rst_n) valid && !ready |=> valid);`

Q3. Write an assertion: data must be stable while `valid` is high and `ready` is low. `assert property (@(posedge clk) disable iff (!rst_n) valid && !ready |=> $stable(data));`

Q4. Write an assertion: after `req`, `ack` must arrive within 2 to 5 cycles. `assert property (@(posedge clk) disable iff (!rst_n) $rose(req) |-> ##[2:5] ack);`

Q5. Write an assertion that a 4-bit counter increments when enabled. `assert property (@(posedge clk) disable iff (!rst_n) en |=> count == $past(count) + 4'd1);` The sized literal makes the comparison wrap at 15 like the hardware.

Q6. Write an assertion: `valid_out` is asserted exactly one cycle after `valid_in`. Two assertions: `valid_in |=> valid_out` and `!valid_in |=> !valid_out`. Or, looking backwards: `valid_out == $past(valid_in)`, with care for the first cycle after reset.

Q7. Write an assertion that a signal is never X after reset. `assert property (@(posedge clk) disable iff (!rst_n) !$isunknown(sig));`

Q8. How would you check FIFO data integrity? With a scoreboard: push each written word into a reference queue and compare each read word with the front of the queue. It can also be done in SVA with local variables that capture a written word and count the writes and reads until that word must appear, but the scoreboard is simpler and is the normal choice.

Q9. A design-team assertion fires in your simulation. What do you do? Read the property to see which rule was broken, and use the start and failure times to find the cause in the waveform. Then decide whether the stimulus was illegal (a testbench bug, or a missing constraint), the DUT is wrong (file a bug), or the assertion itself is wrong (as with the 32-bit `+ 1`). Do not disable the assertion to make the test pass.

Q10. How do you know that your assertions were actually exercised? From assertion coverage: each assertion should have non-vacuous passes. For each important antecedent a `cover property` is written, and the cover must be hit in the regression.

Part 7: Module 10 Revision and Practice

7.1 Revision Summary

Coverage on one page

Term	One-line meaning
Code coverage	Tool-generated measure of which RTL statements, branches, conditions, toggles and FSM states/transitions were exercised
Functional coverage	User-written measure of which planned features and scenarios were exercised
`covergroup`	Container of a coverage model; a type that must be instantiated with `new`
`coverpoint`	One integral expression to observe, with its bins; optional `iff` guard
Bin	A named counter for a value, set of values, or sequence of values
Automatic bins	Created when no bins are given: one per enum member, or min(2^M, `auto_bin_max` = 64)
`bins b = {..}` / `b[] = {..}` / `b[N] = {..}`	One bin / one bin per value / N bins with values spread evenly
`wildcard bins`	X, Z, `?` in the value are don't-care positions
`ignore_bins`	Values excluded from coverage, silently
`illegal_bins`	Values excluded from coverage; sampling one is a run-time error
`bins b = default`	Catches everything else; not counted
Transition bins	`(a => b)`, `(a => b => c)`, `(a [*3])`, `(a, b => c, d)`; hit by consecutive samples
`cross`	Combinations of bins of two or more coverpoints; size is the product
`binsof` / `intersect`	Select cross products for `bins`, `ignore_bins`, `illegal_bins`
`option` / `type_option`	Per-instance settings / per-type settings
`sample()`	Built-in method that samples the covergroup; can be given arguments with `with function sample`
`get_coverage()` / `get_inst_coverage()` / `$get_coverage()`	Type coverage / instance coverage / overall coverage, as a real from 0 to 100

Coverage rules to remember

- A covergroup collects nothing until it is constructed. In a class, `cg = new();` goes in the class constructor.
- With explicit bins there are no automatic bins; values outside the bins are not tracked.
- Coverage of a coverpoint or cross = covered bins / total bins. Default, ignore and illegal bins are not in the total.
- Coverage of a covergroup = weighted average of its coverpoints and crosses (each is one item).
- A bin is covered when its hit count reaches `option.at_least` (default 1).

- Sample from the monitor side, only when the data is valid.
- Coverage shows what was exercised, never that it was correct.
- Both coverpoints at 100 percent does not imply the cross is at 100 percent.

Assertions on one page

Term	One-line meaning
Immediate assertion	`assert (expr)` : procedural, evaluated when executed, current values, no clock; allowed in classes
Deferred assertion	`assert #0 (expr)` , `assert final (expr)` : immediate assertion whose report waits for the values to settle; avoids glitches
Concurrent assertion	`assert property (...)` : clocked, sampled values, a new attempt every tick; not allowed in classes
Sampled value	The value in the Preponed region: just before the clock edge
Sequence	A pattern of Booleans over clock ticks (`##` , repetition)
Property	A rule built from sequences (implication, `disable iff`)
`assert` / `assume` / `cover` / `restrict`	Check / constrain inputs (formal) / record occurrence / formal-only constraint
`a |-> b`	Overlapped implication: `b` in the tick where `a` ends
`a |=> b`	Non-overlapped: `b` one tick later; same as `a |-> ##1 b`
Vacuous success	The antecedent did not match, so nothing was checked
`##n` , `##[m:n]`	Delay of n ticks; delay window
`[*n]` , `[->n]` , `[=n]`	Consecutive, goto and non-consecutive repetition
`$rose` , `$fell`	LSB changed to 1 / to 0 since the previous tick
`$stable` , `$changed`	Value same as / different from the previous tick
`$past(e, n)`	Value of `e` n ticks ago
`$onehot` , `$onehot0`	Exactly one / at most one bit is 1
`$isunknown` , `$countones`	Any bit X or Z / number of 1 bits
`disable iff (c)`	Do not check, and cancel running attempts, while `c` is true; asynchronous
Action blocks	Pass statement and `else` fail statement; run in the Reactive region; default failure action is `$error`
`bind`	Instantiate a checker module inside a design module without editing the design

Assertion rules to remember

- Regions: sample in Preponed, evaluate in Observed, action blocks in Reactive.
- A signal changing at the clock edge is seen with its old value at that tick.
- Every concurrent assertion needs a clock: explicit, inferred from `always` , or `default clocking` .
- Use `|->` for same-cycle relationships and for consequents that start with `##` ; use `|=>` for registered behaviour.
- Use `$rose(x)` instead of `x` in an antecedent to get one attempt per event.
- `##[1:$]` never fails in simulation. Use a bound.

- `$rose` and `$fell` look at bit 0 only.
- Watch the width in `$past(x) + 1`.
- The reset polarity in `disable iff` must be the condition that disables.
- Reset-value assertions must not be disabled by reset.
- Print `$sampled(sig)` in action blocks.
- Prove non-vacuity with `cover property` or assertion coverage.

The comparison tables of this module

Comparison	Section
Code coverage vs functional coverage	1.2
Covergroup vs coverpoint	1.4
Sampling event vs `sample()`	1.5
`bins b` vs `b[]` vs `b[N]`	2.2
`ignore_bins` vs `illegal_bins`	2.5
Value bins vs transition bins	2.6
Coverpoint vs cross	3.1
`option` vs `type_option`; `get_coverage` vs `get_inst_coverage`	3.2, 3.3
Directed vs coverage-driven verification	3.4
Immediate assertion vs `if`; immediate vs concurrent	4.2, 4.3
Sequence vs property	4.4
`|->` vs `|=>`	5.1
`[*n]` vs `[->n]` vs `[=n]`	5.2
`$rose` vs level vs `posedge`	5.3
`$onehot` vs `$onehot0`	5.4
`disable iff` vs reset in the antecedent	5.5
`and` vs `intersect`	5.7
Assertions vs scoreboard; covergroup vs `cover property`	5.8

Simulation-only or synthesizable

Construct	Status
`covergroup`, `coverpoint`, `cross`, bins	Simulation-only
Immediate and concurrent assertions	Ignored by synthesis; used in simulation and formal verification
`cover property`, `assume property`, `restrict property`	Simulation and/or formal only
`$rose`, `$past`, `$onehot` and the other functions	Not synthesizable in general (`$onehot`, `$countones` and `$clog2` are accepted by many synthesis tools in ordinary RTL expressions)
`bind`	A verification construct; not part of the synthesised design

7.2 Interview Questions and Answers

The sections of Parts 1 to 6 contain the direct questions on each topic. The questions below are additional.

Basic level

Q1. What are the two main types of coverage? Code coverage and functional coverage.

Q2. Which constructs are used for functional coverage? `covergroup` with `coverpoint`, `bins` and `cross` for data-oriented coverage; `cover property` for temporal coverage.

Q3. How do you instantiate a covergroup? With `new`: `cg_type cg = new();`, or `cg_name = new();` in the constructor of a class that embeds it.

Q4. How do you trigger sampling manually? By calling the covergroup's `sample()` method.

Q5. How many automatic bins does a coverpoint on `bit [2:0]` get? On `bit [9:0]`? 8 and 64.

Q6. What are the two kinds of assertions? Immediate and concurrent.

Q7. Which keyword turns a property into a check? `assert property`.

Q8. What does `a |-> b` mean? If `a` is true at a clock tick, `b` must be true at the same tick.

Q9. What does `a |=> b` mean? If `a` is true at a clock tick, `b` must be true at the next tick.

Q10. What does `##2` mean? Two clock ticks later.

Q11. What does `$rose(sig)` return? True if the LSB of `sig` is 1 at this tick and was not 1 at the previous tick.

Q12. What is `disable iff` used for? To switch a property off under a condition, normally during reset.

Q13. What are the severity system tasks? `$fatal`, `$error`, `$warning`, `$info`.

Q14. Can a covergroup be declared inside a class? Can a concurrent assertion? A covergroup can. A concurrent assertion cannot.

Q15. What is the default action when an assertion fails? An error is reported (`$error`); the simulation continues.

Intermediate level

Q16. A coverpoint has `bins a = {[0:3]}; bins b[] = {[4:7]}; bins c[2] = {[8:15]};`. How many bins? 1 + 4 + 2 = 7.

Q17. How do you cover only the legal values of an enum that has two reserved members? Let the tool create automatic bins and exclude the reserved members: `coverpoint op { ignore_bins rsvd = {RSVD1, RSVD2}; }`, or list the legal members in explicit bins.

Q18. What is the difference between `iff` in a coverpoint and `ignore_bins`? `iff` is a run-time condition on other signals: when it is false the coverpoint is not sampled at all for that event. `ignore_bins` permanently removes particular values of the coverpoint itself from the model.

Q19. How do you find out which bins are not covered? From the coverage report generated from the merged coverage database, which lists every bin with its hit count and marks the uncovered ones. `option.cross_num_print_missing` controls how many missing cross bins are listed.

Q20. Why is functional coverage usually sampled in the monitor path and not in the driver? The monitor reports what really occurred at the DUT pins. The driver or generator only knows what was intended; a transaction dropped by the driver or ignored by the DUT would still be counted.

Q21. What is the difference between a transition bin `(1 => 2)` and a cross? A transition bin covers values of **one** coverpoint on **consecutive samples** (a sequence in time). A cross covers values of **several** coverpoints at the **same sample** (a combination).

Q22. How do you write coverage for "all 256 values of an 8-bit field"? `bins all[] = {[0:255]};` Automatic bins would only give 64 bins unless `option.auto_bin_max = 256` is set.

Q23. What does it mean when an assertion "passes vacuously"? Its antecedent was false, so the consequent was not evaluated.

Q24. Write an assertion: if `a` is high, then `b` must be high within the next 3 cycles (not in the same cycle). `assert property (@(posedge clk) a |-> ##[1:3] b);`

Q25. Write an assertion: `req` and `gnt` are never high in the same cycle. `assert property (@(posedge clk) !(req && gnt));`

Q26. Write an assertion: `sig` must be high for at least 2 consecutive cycles once it rises. `assert property (@(posedge clk) $rose(sig) |-> sig [*2]);`

Q27. What is the difference between `$past(a)` and `##1`? `$past(a)` looks backwards: it returns the value of `a` at the previous tick, for use at the current tick. `##1` moves forwards: it advances the sequence by one tick. `a |=> b` and `b |-> $past(a)` relate the same two ticks but check different directions of implication.

Q28. Where would you put the protocol assertions of a bus interface? In the SystemVerilog interface for that bus (or in a checker module bound to it), so that they are active wherever the interface is used.

Q29. How do you turn assertions off for part of a test? `$assertoff` / `$asserton`, optionally with a hierarchy argument; or a control signal used in `disable iff`.

Q30. What is the difference between `assert property` and `cover property`? `assert` reports a failure when the property is violated. `cover` never fails; it counts how often the property or sequence was observed.

Advanced and scenario-based

Q31. Functional coverage is 100 percent and all tests pass, but a bug escaped to silicon. How is that possible? (1) The coverage model did not include the scenario (the plan was incomplete). (2) The scenario was covered but the checker did not check the affected behaviour. (3) Bins were too coarse: the buggy value shared a bin with values that were hit. (4) Coverage was sampled at the wrong time or on the generator side. (5) Exclusions hid a real scenario. Coverage measures the tests against the plan; it cannot find what the plan omits.

Q32. Your regression is stuck at 92 percent functional coverage. Describe your approach. List the uncovered bins and group them by cause. Run more seeds if hits are simply rare. Check constraints for accidental exclusion. Write targeted constraint-based tests or use `dist` for improbable combinations. Write directed tests for specific sequences. Confirm unreachable items with the designer and exclude them with justification. Fix errors in the coverage model itself. Re-run and re-merge.

Q33. An assertion never fails and never passes non-vacuously. What are the possible reasons? The antecedent never occurs because the stimulus does not produce it; the `disable iff` polarity is wrong so the assertion is always disabled; the assertion's clock never ticks or is the wrong clock; the antecedent is written so

that it can never be true (for example using a sampled value function on the clock itself); or assertions are globally switched off.

Q34. An assertion fails at time 0 or at the first clock, before reset. Why, and what do you do? Signals are X before reset, or `$fell` / `$stable` / `$past` compare against an X history at the first tick. Add `disable iff (!rst_n)` (or `default disable iff`), make sure reset is asserted before the first active clock edge, and keep only dedicated reset assertions active during reset.

Q35. Why might the same assertion pass in RTL simulation and fail in gate-level simulation? In gate-level simulation signals change after real delays instead of at the clock edge, there are glitches and X-pessimism, and reset or initialisation X propagates more. Assertions that sample internal signals, or that rely on zero-delay behaviour, see different values. Port-level, clock-synchronous assertions are the ones expected to hold in both.

Q36. How do you decide between a scoreboard check and an assertion for a requirement? If the requirement is about signal behaviour over a few cycles (ordering, timing, stability, mutual exclusion, X), use an assertion. If it is about the correctness of transformed data, especially with queues, reordering or long latency, use the scoreboard. Simple data relationships with fixed latency can be done either way.

Q37. What is the cost of too many covergroups or too large crosses? Slower simulation and larger databases, reports that nobody can read, and coverage that cannot be closed. A coverage item that does not correspond to a plan item should not exist.

Q38. In formal verification, what is the role of `assume`? Assumptions describe the legal behaviour of the inputs. The formal tool explores only input sequences that satisfy them. Too few assumptions give false failures (illegal stimulus); too many, or contradictory ones, hide bugs (over-constraint), which is checked with covers.

Q39. Why should a failing assertion not be "fixed" by adding `disable iff` conditions until it passes? Each added condition removes checking. The failure must first be understood: design bug, illegal stimulus, or wrong assertion. Weakening the assertion without that analysis hides the bug it found.

Q40. How would you verify that a `valid_out` pulse is exactly one cycle wide? `assert property (@(posedge clk) disable iff (!rst_n) $rose(valid_out) |=> !valid_out);` (For the ALU with back-to-back operations this rule does not apply, because consecutive results keep `valid_out` high; the specification decides.)

7.3 Output-Prediction Questions

Work out each answer on paper before reading it.

P1. How many bins, and what is the coverage?

```
bit [2:0] x;
covergroup cg;  cp: coverpoint x;  endgroup
// samples of x: 1, 1, 2, 7
```

Answer: 8 automatic bins. Values 1, 2 and 7 were hit: 3 / 8 = **37.50 percent**.

P2. How many bins, and what is the coverage?

```
bit [3:0] v;
covergroup cg;
  cp: coverpoint v {
    bins a    = {[0:7]};
    bins b[]  = {[8:11]};
    bins c[2] = {[12:15]};
  }
endgroup
// samples of v: 3, 9, 9, 13
```

Answer: 1 + 4 + 2 = 7 bins. Hit: `a` (3), `b[9]` (9, twice), `c[0]` (12 to 13). 3 / 7 = **42.86 percent**.

P3. What is the coverage?

```
bit x;
covergroup cg;
  option.at_least = 3;
  cp: coverpoint x;
endgroup
// samples of x: 0, 0, 0, 1, 1
```

Answer: Two bins. Value 0 has three hits (covered); value 1 has two (not covered). **50 percent**.

P4. What is the coverage? Is there an error?

```
bit [2:0] x;
covergroup cg;
  cp: coverpoint x {
    bins        v[] = {[0:7]};
    ignore_bins ig  = {6, 7};
  }
endgroup
// samples of x: 0, 1, 6, 6
```

Answer: 8 - 2 = 6 counted bins. Hit: 0 and 1. 2 / 6 = **33.33 percent**. No error: ignored values are skipped silently. With `illegal_bins` the two samples of 6 would each produce a run-time error.

P5. What are the coverpoint, cross and covergroup coverages?

```
bit       a;
bit [1:0] b;
covergroup cg;
  cp_a: coverpoint a;
  cp_b: coverpoint b;
  x_ab: cross cp_a, cp_b;
endgroup
// samples (a, b): (0,0), (1,3), (0,0), (1,0)
```

Answer: `cp_a`: both values, 100 percent. `cp_b`: values 0 and 3, 2 / 4 = 50 percent. `x_ab`: 2 × 4 = 8 bins; hit (0,0), (1,3), (1,0): 3 / 8 = 37.5 percent. Covergroup: (100 + 50 + 37.5) / 3 = **62.50 percent**.

P6. Which transition bins are hit, and what is the coverage?

```
bit [1:0] s;
covergroup cg;
  cp: coverpoint s {
    bins t1 = (0 => 1 => 2);
    bins t2 = (1 => 2);
    bins t3 = (2 [*2]);
  }
endgroup
// samples of s: 0, 1, 1, 2, 2
```

Answer: `t1` is not hit: after 0, 1 comes another 1, not 2. `t2` is hit (third and fourth samples). `t3` is hit (fourth and fifth samples). 2 / 3 = **66.67 percent**.

P7. At which tick does the assertion fail?

```
a_t: assert property (@(posedge clk) a |=> b);

tick   1   2   3   4
a      1   1   0   0
b      0   1   0   0
```

Answer: The attempt of tick 1 needs `b` at tick 2: `b` = 1, pass. The attempt of tick 2 needs `b` at tick 3: `b` = 0, **fails at tick 3**. Ticks 3 and 4 are vacuous.

P8. Compare the two assertions on the same stimulus.

```
a_edge : assert property (@(posedge clk) $rose(a) |-> ##[1:2] b);
a_level: assert property (@(posedge clk)       a  |-> ##[1:2] b);

tick   1   2   3   4   5   6
a      0   1   1   1   0   0
b      0   0   0   0   1   0
```

Answer: `a_edge` has one real attempt, at tick 2: `b` is needed at tick 3 or 4, both 0: **fails at tick 4**. `a_level` has three attempts. Tick 2: fails at tick 4. Tick 3: `b` at tick 4 is 0, at tick 5 is 1: passes at tick 5. Tick 4: `b` at tick 5 is 1: passes at tick 5. So `a_level` also reports one failure at tick 4, plus two passes; the level form creates overlapping attempts.

P9. Does the assertion pass or fail at time 15?

```
// clk rises at 5, 15, 25 ...
always @(posedge clk) q <= d;                    // q updated by the same edge
a_q: assert property (@(posedge clk) d |-> q);
// d = 1 from time 10 onwards; q was 0 before time 15
```

Answer: It **fails at time 15**. At the tick at time 15 the sampled values are `d` = 1 and `q` = 0, because `q` is updated by that same edge and the assertion sees the value before the edge. The correct property for a flip-flop is `d |=> q`, which checks at time 25 and passes.

P10. Where does the sequence match?

```
sequence s;  a ##1 b [*2] ##1 c;  endsequence

tick   1   2   3   4   5
a      1   0   0   0   0
b      0   1   1   0   0
c      0   0   0   1   0
```

Answer: `a` at tick 1, `b` at ticks 2 and 3, `c` at tick 4. The sequence **matches, ending at tick 4**. (`b [*2]` binds only to `b`.)

P11. Evaluate the functions.

Expression	Value
`$onehot(4'b1000)`	1
`$onehot(4'b0000)`	0
`$onehot0(4'b0000)`	1
`$onehot0(4'b1010)`	0
`$countones(8'b1011_0010)`	4
`$isunknown(4'b10z1)`	1
`$onehot(4'b1x00)`	1 (X bits are not counted as ones)

P12. A 3-bit counter increments every cycle. Does this assertion ever fail?

```
logic [2:0] cnt;
a_c: assert property (@(posedge clk) disable iff (!rst_n) cnt == $past(cnt) + 1);
```

Answer: Yes, at the wrap-around. When the previous value is 7, the right side is 7 + 1 = 8 in 32 bits, while `cnt` is 0. It also needs care at the first tick after reset, where `$past(cnt)` refers to a tick during reset. The fix for the wrap is `$past(cnt) + 3'd1`.

7.4 Find-the-Error and Debugging Questions

Compile-time errors

E1.

```
class my_cov;
  bit [3:0] v;
  covergroup cg;
    coverpoint v;
  endgroup
  function void build();
    cg = new();
  endfunction
endclass
```

An embedded covergroup must be constructed in the class constructor. Move `cg = new();` into `function new();`.

E2.

```
covergroup cg_a;  cp_a: coverpoint a;  endgroup
covergroup cg_b;
  cp_b: coverpoint b;
  x: cross cp_a, cp_b;
endgroup
```

A cross can only use coverpoints (or variables) of its own covergroup. Put `cp_a` and `cp_b` in the same covergroup.

E3.

```
a_eq: assert (a == b); else $error("mismatch");
```

The semicolon after the condition ends the assertion; the `else` has nothing to attach to. Remove the semicolon: `assert (a == b) else $error("mismatch");`.

E4.

```
class alu_driver;
  virtual alu_if vif;
  a_v: assert property (@(posedge vif.clk) vif.valid_in |=> vif.valid_out);
endclass
```

Concurrent assertions are not allowed in classes. Move the assertion into `alu_if` or into a bound checker module.

E5.

```
module m (input logic clk, req, gnt);
  a1: assert property (req |=> gnt);
endmodule
```

The assertion has no clock. Add `@(posedge clk)` or a `default clocking` declaration.

E6.

```
sequence s_req_gnt;
  req |-> ##1 gnt;
endsequence
```

Implication is a property operator and cannot be used in a sequence. Declare it as a `property`.

E7.

```
logic [3:0] n;
a2: assert property (@(posedge clk) start |-> ##n done);
```

The delay must be a constant. Use a parameter, or a range with constant bounds, or a counter in a local variable.

E8.

```
cp_st: coverpoint state {
  bins t = {IDLE => RUN};
}
```

A transition bin is written with parentheses: `bins t = (IDLE => RUN);`. Braces are for value sets.

Run-time and logic bugs (the code compiles)

D1.

```
cp_cmd: coverpoint cmd {
  bins odd = {4'b???1};
}
```

The `wildcard` keyword is missing, so `?` is a literal Z and the bin can never be hit. Write `wildcard bins odd = {4'b???1};`.

D2.

```
covergroup cg @(posedge clk);
  cp_data: coverpoint data;
endgroup
```

`data` is sampled at every clock, including idle cycles and reset. The coverage fills with meaningless values. Add `iff (valid && rst_n)`, or remove the event and call `sample()` from the monitor.

D3.

```
cp_addr: coverpoint addr {      // addr is 32 bits
  bins all = {[0:$]};
}
```

One bin over the whole range is covered by the first sample. The coverpoint always reports 100 percent and measures nothing. Define bins for the regions and boundaries that matter.

D4.

```
a_rst: assert property (@(posedge clk) disable iff (rst_n) req |=> ack);
```

`rst_n` is active low, so the assertion is disabled whenever the design is running. It should be `disable iff (!rst_n)`.

D5.

```
a_d: assert property (@(posedge clk) en |=> ##1 q == $past(d));
```

`|=>` already moves one tick; `##1` moves another. `q` is checked two ticks after `en` against `d` from one tick earlier, which is probably not the intent. For a simple register: `en |=> q == $past(d)`.

D6.

```
logic [3:0] grant;
a_g: assert property (@(posedge clk) $rose(grant) |-> busy);
```

`$rose` on a vector checks only bit 0, so grants on bits 1 to 3 never trigger the assertion. Use `$rose(|grant)` or `$changed(grant) && grant != 0`.

D7.

```
a_ack: assert property (@(posedge clk) req |-> ##[1:$] ack);
```

With an unbounded window the assertion cannot fail in simulation; a missing `ack` only leaves the attempt pending. Use a finite bound such as `##[1:16]`.

D8.

```
a_rand: assert (txn.randomize()) else $fatal(1, "randomize failed");
```

It works until assertions are disabled; then `randomize()` may not be executed at all. Call it outside the assertion: `if (!txn.randomize()) $fatal(1, "randomize failed");`.

D9.

```
a_fail: assert property (@(posedge clk) valid |-> ready)
  else $error("valid=%b ready=%b", valid, ready);
```

The message prints current values, which may already have changed after the clock edge. Print `$sampled(valid)` and `$sampled(ready)`.

D10.

```
cp_op: coverpoint txn.op { option.weight = 0; }
cp_a : coverpoint txn.a  { option.weight = 0; }
// no cross declared
```

Both coverpoints have weight 0 and there is no cross that uses them, so the covergroup has nothing that counts. Either a cross was forgotten or the weights are wrong.

7.5 Coding Exercises with Solutions

Exercise 1: FIFO fill-level coverage

A testbench tracks the number of entries in a 16-entry FIFO. Write a covergroup that covers the three fill classes (empty, partial, full) and the transitions empty→partial, partial→full, full→partial, partial→empty, and "full for three samples in a row". Sample the levels 0, 5, 16, 16, 3, 0 and compute the coverage.

ex01_fifo_level_cov.sv

```
module ex01_fifo_level_cov;

  covergroup cg_level with function sample(int unsigned level);
    cp_class: coverpoint level {
      bins empty   = {0};
      bins partial = {[1:15]};
      bins full    = {16};
    }
    cp_trans: coverpoint level {
      bins start_fill = (0 => [1:15]);
      bins reach_full = ([1:15] => 16);
      bins leave_full = (16 => [1:15]);
      bins drain      = ([1:15] => 0);
      bins stay_full  = (16 [*3]);
    }
  endgroup

  cg_level cg = new();

  initial begin
    cg.sample(0);
    cg.sample(5);
    cg.sample(16);
    cg.sample(16);
    cg.sample(3);
    cg.sample(0);
    $display("cp_class = %0.2f%%", cg.cp_class.get_coverage());
    $display("cp_trans = %0.2f%%", cg.cp_trans.get_coverage());
    $display("cg_level = %0.2f%%", cg.get_coverage());
  end

endmodule
```

EXPECTED OUTPUT

```
cp_class = 100.00%
cp_trans = 80.00%
cg_level = 90.00%
```

All three classes were sampled. The transitions 0→5, 5→16, 16→3 and 3→0 hit four bins. The FIFO was full for only two consecutive samples, so `stay_full` is the hole: 4 / 5 = 80 percent. The covergroup is the average of the two coverpoints.

Exercise 2: check the width of a pulse

Write an assertion that a pulse on `p` is exactly two clock cycles wide, and test it with one good and one bad pulse.

ex02_pulse_width.sv

```
module ex02_pulse_width;

  logic clk = 0;
  logic p;

  // cycles 0..9 : a 2-cycle pulse at cycles 1-2, a 3-cycle pulse at cycles 5-7
  bit [0:9] p_v = 10'b0110011100;

  always #5 clk = ~clk;

  a_width: assert property (@(posedge clk) $rose(p) |-> p [*2] ##1 !p)
    else $display("[%0t] a_width FAILED: pulse is not exactly 2 cycles wide", $time);

  initial begin
    for (int i = 0; i < 10; i++) begin
      p = p_v[i];
      @(negedge clk);
    end
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[75] a_width FAILED: pulse is not exactly 2 cycles wide
```

`$rose(p)` is true at cycles 1 and 5. From cycle 1: `p` is 1 at cycles 1 and 2 and 0 at cycle 3, a pass. From cycle 5: `p` is 1 at cycles 5 and 6 but still 1 at cycle 7 (time 75), where `!p` was required. A pulse of only one cycle would fail at its second cycle, because `p [*2]` would not match.

Exercise 3: covergroup for a packet length field

A packet has a 6-bit `len` field. Legal lengths are 1 to 60. Lengths 0 and 61 to 63 are illegal. The plan asks for: minimum, maximum, the values next to each boundary, and three ranges in between.

```
covergroup cg_len with function sample(bit [5:0] len);
  cp_len: coverpoint len {
    bins         min       = {1};
    bins         near_min  = {2};
    bins         short_len = {[3:20]};
    bins         mid_len   = {[21:40]};
    bins         long_len  = {[41:58]};
    bins         near_max  = {59};
    bins         max       = {60};
    illegal_bins bad       = {0, [61:63]};
  }
endgroup
```

Seven counted bins. The boundary values get their own bins because off-by-one errors are found there. The illegal values produce an error if the monitor ever observes them (and the same condition should also be checked by an assertion or the scoreboard).

Exercise 4: request/grant rules

Write assertions for: (a) a grant is given within 1 to 4 cycles after a request rises; (b) there is never a grant without a request having been made in one of the previous four cycles or in the current one; (c) `gnt` is a single-cycle pulse.

```
default clocking cb @(posedge clk); endclocking
default disable iff (!rst_n);

// (a) forward: every request is granted in time
a_gnt_in_time: assert property ($rose(req) |-> ##[1:4] gnt);

// (b) backward: every grant has a cause
a_gnt_has_req: assert property (
  gnt |-> (req || $past(req, 1) || $past(req, 2) || $past(req, 3) || $past(req, 4)));

// (c) pulse width of one cycle
a_gnt_pulse: assert property ($rose(gnt) |=> !gnt);

// non-vacuity
c_req: cover property ($rose(req));
c_gnt: cover property ($rose(gnt));
```

The forward assertion alone would be satisfied by a design that holds `gnt` permanently high. The backward assertion and the pulse-width assertion close that gap. Checking both directions is a habit worth mentioning in an interview.

Exercise 5: one-hot state machine

Write assertions for an FSM whose state register `state[3:0]` is one-hot encoded with `state[0]` = IDLE: (a) the state is always one-hot and known; (b) after reset the state is IDLE; (c) from IDLE the FSM leaves only when `start` is high.

```
// (a)
a_onehot: assert property (@(posedge clk) disable iff (!rst_n)
  $onehot(state) && !$isunknown(state));

// (b) not disabled by reset
a_reset_idle: assert property (@(posedge clk) !rst_n |-> state == 4'b0001);

// (c) if we were in IDLE and start was low, we are still in IDLE
a_stay_idle: assert property (@(posedge clk) disable iff (!rst_n)
  (state[0] && !start) |=> state[0]);
```

Exercise 6: coverage for a valid/ready channel

Write coverage showing that the channel was exercised with: immediate acceptance, a stall of one cycle, a stall of two or more cycles, and back-to-back transfers.

```
default clocking cb @(posedge clk); endclocking

c_immediate   : cover property ($rose(valid) ##0 ready);
c_stall_1     : cover property ($rose(valid) ##0 !ready ##1 (valid && ready));
c_stall_2plus : cover property ($rose(valid) ##0 !ready [*2:$] ##1 (valid && ready));
c_back_to_back: cover property ((valid && ready) [*2]);
```

Temporal scenarios on signals are expressed most directly with `cover property`. The same information could be collected by a monitor that measures the wait time of each transfer and samples a covergroup with bins `{0}`, `{1}`, `{[2:$]}`.

Exercise 7: add operand-boundary coverage for the ALU

Extend `alu_coverage` so that it shows, for the addition only, that both operands were at their maximum together and that a carry resulted.

```
// inside covergroup alu_cg of class alu_coverage
cp_add_max: coverpoint (txn.a == 8'hFF && txn.b == 8'hFF) iff (txn.op == ALU_ADD) {
  bins both_max = {1'b1};
}
cp_add_max_carry: coverpoint txn.result[8]
                  iff (txn.op == ALU_ADD && txn.a == 8'hFF && txn.b == 8'hFF) {
  bins carry = {1'b1};
}
```

Each coverpoint has a single bin for the one scenario of interest. This style, a Boolean expression with one bin, is useful for specific corner cases from the verification plan.

Further practice (no solutions given)

1. Write a covergroup for an 8-bit `addr` and a 1-bit `wr` in which the address space has three regions, 0x00-0x3F (RAM), 0x40-0x7F (registers, read-only) and 0x80-0xFF (unused). Cross region with direction, ignore writes to the read-only region, and make any access to the unused region illegal.
2. For the transition example of Section 2.6, write the stimulus that brings `cp_state` to 100 percent with the fewest samples.
3. Write an assertion: once `error` is raised it stays high until `clear` is high.
4. Write an assertion: `busy` is high from the cycle after `start` until, and including, the cycle in which `done` is high.
5. Write an assertion with a local variable: the value on `din` when `push` is high appears on `dout` exactly three cycles later.
6. In `tb_fifo_sva`, change the DUT so that `full` is asserted when the count is 15. Predict which assertions fail and at what times, then run it.
7. Add a covergroup to `handshake_sva` that covers the wait time (in cycles) of each transfer, and compute its coverage for the stimulus of Section 6.2.
8. Bind `alu_sva` to the ALU in the layered testbench of Module 9 and add its cover properties to the end-of-test report.

7.6 Concepts You Must Understand Before Moving Forward

Coverage

- [ ] I can explain code coverage and functional coverage, name the code coverage types, and say what each combination of high/low means.
- [ ] I can write a covergroup in a module and in a class, and I know where it must be constructed.
- [ ] I can explain sampling by event, by `sample()` and by `sample()` with arguments, and why coverage is sampled on the monitor side.
- [ ] I can state how many automatic bins a coverpoint gets.
- [ ] I can explain `bins b`, `bins b[]` and `bins b[N]`.
- [ ] I can write wildcard, ignore, illegal and transition bins and say how each affects the percentage.
- [ ] I can write a cross, restrict it with `binsof` and `intersect`, and count its bins.
- [ ] I can compute the coverage of a coverpoint, a cross and a covergroup by hand, including `at_least` and `weight`.
- [ ] I can explain `option` versus `type_option` and `get_coverage()` versus `get_inst_coverage()`.

- [ ] I can describe the coverage-driven verification loop and what to do with a coverage hole.

Assertions

- [ ] I can explain the difference between immediate, deferred and concurrent assertions.
- [ ] I can explain sampled values and the Preponed, Observed and Reactive regions, and predict what an assertion sees for a signal that changes at the clock edge.
- [ ] I can explain the sequence, property and directive layers, and `assert`, `assume`, `cover`.
- [ ] I can give the four ways of specifying the clock.
- [ ] I can explain `|->` versus `|=>` with a timeline, and vacuous success.
- [ ] I can use `##n`, `##[m:n]`, `[*n]`, `[->n]` and `[=n]`.
- [ ] I can use `$rose`, `$fell`, `$stable`, `$changed` and `$past`, and I know their behaviour at the first tick and on vectors.
- [ ] I can use `$onehot`, `$onehot0`, `$isunknown` and `$countones`.
- [ ] I can explain `disable iff`, its asynchronous nature and the polarity trap.
- [ ] I can explain action blocks, default failure behaviour and the severity tasks.
- [ ] I can write a checker module and attach it with `bind`.
- [ ] I can write a reasonable set of assertions for a FIFO, a valid/ready handshake and a counter from a verbal description.
- [ ] I can explain what belongs in an assertion and what belongs in the scoreboard.

7.7 What Comes Next

Module 11 completes the language with packages and `import`, parameterized and generic classes, process control (`process` handles, named `fork` blocks), `randsequence` and `randcase`, an introduction to DPI, and the commonly used system tasks. It then introduces UVM: how the generator, driver, monitor, scoreboard, coverage collector and environment that you have built by hand in Modules 6 to 10 correspond to UVM sequences, sequencers, drivers, monitors, agents, scoreboards, subscribers and environments. After Module 11, the final section collects more than 150 interview questions across all modules.